NS3SIONNA: MIMO-ABSTRAKTIONSMODELL FÜR IEEE 802.11AX/BE

Stand: 7. Oktober 2026, Phase 4 einschließlich Unterphase 4.8 implementiert;
Unterphase 5.1 für HE MCS 0/BCC/32 Byte/1x1 anhand von Link-Level-Daten validiert;
Unterphasen 5.2.1 bis 5.2.5 mit Kanal-, Konvergenz-, Cache-/IPC-, Arrayabstands- und
CSI-Alterungssensitivität implementiert;
R1 als eine native CPU-Unterphase mit Seed-/Run-/Streams und portablen Bundles abgeschlossen;
erstes R1/R3-Inkrement mit portabler Provenienz, CPU-CI und deaktiviertem Cache implementiert;
zweites R3-Inkrement mit Cachekorrektheitsregressionen, Belegungszählern und inaktiver
Linkbereinigung implementiert;
drittes R3-Inkrement mit Allokations-/Antwort-/Speicherbudgets und Look-ahead-Tiefenprüfung
implementiert;
viertes R3-Inkrement mit gemessener CFR-Kopierreduktion und Getter-/MU-Eigentumsregressionen
implementiert

Arbeitszweig: feature/su-mimo-minimal

Basis: ns-3.40, ns3sionna c896b858; Auftragsbasis 46d73fa; Sionna und Sionna RT 2.0.1

1. ZIEL UND ABGRENZUNG

Sionna RT berechnet die komplexe, frequenzselektive
Kanalübertragungsmatrix. Die eigentliche Link-Abstraktion, also
Post-Processing-SINR, Effective-SINR, MCS/PER und später die Auswahl von
Precoding und MU-MIMO, wird ausschließlich in ns-3 ausgeführt.

Die Entwicklung erfolgt in kleinen, einzeln testbaren Schritten. Der aktuelle
Stand umfasst die Unterphasen 1.1 bis 1.4 sowie die vollständigen Phasen 2,
3 und 4: den numerischen SU-MIMO-Kern, konfigurierbare
Sionna-Arrays, einen
dimensionssicheren MIMO-Datenvertrag, explizit kalibrierbare
802.11ax-EESM-/PER-Abbildung und die
schmale Einkopplung in den SpectrumWifiPhy-Empfangspfad. SionnaMimoHelper legt
den High-Level-API-Vertrag und dessen zentrale Validierung fest und wendet ihn
nun konsistent auf Sionna-IPC, Phase-1-Linkabstraktion und SpectrumWifiPhy an.
Der MCS-bezogene ApplyForHeMcs-Aufruf prüft eine für SU-MIMO erforderliche
EESM-Kalibrierung vor jeder Zustandsänderung. Das reproduzierbare Beispiel
konfiguriert 1x1 und Vollrang-2x2 ausschließlich über SionnaMimoHelper und
deckt mit dem deterministischen Kanalpfad SpectrumWifiPhy end-to-end ab.
Integrationsnahe Negativläufe prüfen widersprüchliche NSS, nicht unterstützte
Modi und eine CFR-/PHY-Dimensionsverletzung. Ein dimensionssicherer,
benutzergegebener Precoder unterstützt nun frequenzflaches und
frequenzselektives festes $W[k]$, einschließlich kontrollierter Streamauswahl
mit $N_s < N_{\mathrm{tx}}$. Darauf aufbauend erzeugt ns-3 nun für den eindeutigen
MISO-Fall aus perfekter CFR ein leistungsnormalisiertes Ein-Stream-MRT-$W[k]$.
Für $N_{\mathrm{tx}} = 1$ ist der bestehende Ein-Stream-MMSE-Pfad als äquivalente
MRC-Kombination validiert. Die neue deterministische SVD-Fabrik wählt pro
Subcarrier explizit die N_s stärksten numerisch tragfähigen Eigenmoden und
erzeugt bei gleicher Leistung oder optionaler räumlicher Wasserfüllung erneut
ein festes $W[k]$. Rang, Singularwerte, Rauschleistung, Referenz-SNR, aktive
Moden und Leistungsallokation bleiben vollständig sichtbar. Die neue
semantische HE-/EHT-SU-Feedbackdarstellung komprimiert gleichleistungs-
SVD-Precoder getrennt in Givens-Winkel, quantisiert sie deterministisch mit
den groben 4/2- oder feinen 6/4-Bit-Codebooks und rekonstruiert daraus erneut
ein auf den festen-$W[k]$-Vertrag normalisiertes Ergebnis. Bitumfang,
Normierungskorrektur sowie phaseninvariante Richtungs- und Koeffizientenfehler
werden ausgewiesen. MimoCsiAgingModel trennt $\hat{H}[k, t_{\mathrm{csi}}]$ von
$H[k, t_{\mathrm{tx}}]$,
bildet periodische Aktualisierung und Feedbackverzögerung explizit ab und
liefert eine gerichtete, array-, modus- und zeitstempelabhängige
Zustandsidentität. SionnaMimoHelper integriert nun festes Precoding, MRT, SVD,

Wasserfüllung, quantisiertes Feedback und CSI-Alterung durch eine explizite,
unveränderliche Precoderwahl. Apply führt das ausgewählte $W[k]$ ohne geänderte
Semantik durch denselben festen SpectrumWifiPhy-Adapterpfad. Das neue
CPU-End-to-End-Beispiel vergleicht Open Loop, festen 2x1-Precoder, MRT und SVD
bei gleicher Gesamtleistung und gibt Konfiguration, Durchsatz, PER,
Stream-SINR, Rang, Precodingnorm, Leistungsanteile, Feedbackdiagnostik und
CSI-Alter maschinenlesbar aus.
Der additive Mehrnutzer-Kanalvertrag ordnet nun gerichtete, zeitgleiche CFRs
stabil nach Nutzer-ID, trägt die RX-Geometrie je Nutzer und übernimmt erst
nach vollständiger Snapshotprüfung Einträge in den bestehenden Cache. Darauf
wertet der neue native Downlink-MU-Kern einen festen gemeinsamen Precoder mit
expliziter Spaltenzuordnung aus. Er erhält die gesamte AP-Sendeleistung,
trennt gewünschte Leistung und Inter-User-Interferenz je Nutzer, Subcarrier und
Stream und berechnet EESM sowie PER mit den nutzerspezifischen MCS- und
Paketgrößenangaben. Die neue wertbasierte Precoderwahl erzeugt für genau einen
Stream und eine Empfangsantenne je Nutzer aus gleichzeitiger perfekter CSI
deterministisches ZF oder RZF mit explizitem Regularisierungsparameter. Beide
Ergebnisse werden auf die gemeinsame Phase-4.2-Gesamtleistung normalisiert;
überlastete und numerisch rangdefiziente Gruppen werden abgelehnt. Unterphase
4.4 integriert diesen Zustand für HE-DL-MU-PPDUs in den Spectrumadapter,
ordnet STA-ID, RU, MCS und NSS zu und begrenzt räumliche Interferenz auf
dieselbe RU. Unterphase 4.5 bindet nun den Laufzeit-CSI-Zustand an die
MultiUserScheduler-Ausgabe und ergänzt feste sowie korrelationsbasierte
Gruppen. Unterphase 4.6 ergänzt die Helper-Installation, den konkreten Downlink-Scheduler und
den
deterministischen CPU-Vergleich. Unterphase 4.7 ergänzt den gemeinsamen
Uplink-ZF-/MMSE-Numerikkern und nutzerweise PER. Unterphase 4.8 bindet reale
HE-Basic-Trigger- und HE-TB-PHY-Metadaten an diesen Kern und berücksichtigt
nur exakt gleichzeitig am AP empfangene Nutzer innerhalb derselben RU.
Unterphase 5.2.1 ergänzt einen festen 2x2-Kanalsensitivitätsbenchmark mit
Abstands-/Orientierungssweep, wiederholbarem CPU-Raytracing und eingefrorenen
Kanal-/SINR-Referenzen. Die Auswertung bleibt innerhalb des kalibrierungsfreien
Kanal-/MMSE-/MI-Messvertrags.
Unterphase 5.2.2 ergänzt gepaarte Seed-/Samplebudgetvergleiche für vier
repräsentative Konfigurationen mit vorab definierten Toleranzen,
byteidentischen Wiederholungen und unabhängiger Referenzvalidierung.
Unterphase 5.2.3 misst den bestehenden Cache und tatsächliche lokale
Prozess-IPC mit eingefrorenen Kanalantworten, getrennt von Raytracing.
Unterphase 5.2.4 ergänzt getrennte TX-/RX-Abstände und ausgewählte Azimute
im bestehenden Benchmark. Neue Messartefakte bleiben lokal und unversioniert.
Unterphase 5.2.5 vergleicht aktuelles und verzögertes Gleichleistungs-SVD-CSI
auf denselben explizit entwickelten Sendekanälen zweier repräsentativer Links.

2. REPRODUZIERBARE AUSGANGSBASIS

Die lokale Referenzplattform ist CPU-only:

- Ubuntu 24.04, GCC 13.3, CMake 3.28
- ns-3.40 mit Tests und Beispielen
- ZeroMQ 4.3.5 und Protocol Buffers 3.21.12
- Python 3.12.12, Sionna 2.0.1, Sionna RT 2.0.1 und NumPy 2.5.2
- Mitsuba 3.8.0, Dr.Jit 1.3.1 und LLVM-Backend llvm_ad_mono_polarized
- PyTorch 2.9.1+cpu; TensorFlow ist keine Laufzeitabhängigkeit mehr

Ohne explizite Arraykonfiguration verwendet der Server weiterhin exakt die
bisherige 1x1-Sende- und Empfangsantenne. Die normalisierte SISO-CFR erfüllt

$$
  \frac{1}{N_\mathrm{sc}}\sum_k |h[k]|^2 = 1,
$$

während der Wideband-Pfadverlust separat in dB übertragen wird. Unterphase 1.2 verallgemeinert diese Trennung auf alle Antennenpaare: $H[k]$ enthält Small-Scale-Fading und der skalare Wideband-Verlust bestimmt die Referenz-SNR.

3. ENTWICKLUNGSPLAN

PHASE 0: BASELINE UND MESSVERTRAG

1. Versionen und CPU-Ausführung reproduzierbar festhalten.
2. Bestehende SISO-Import-, Unit- und Beispieltests ausführen.
3. Einheiten, Antennenreihenfolge, CFR-Normalisierung und Subcarrier-Reihenfolge als Datenvertrag festlegen.

Abnahmekriterium: SISO-Verhalten bleibt unverändert und alle alten Tests laufen.

PHASE 1: BELASTBARES SU-MIMO-MINIMALMODELL

UNTERPHASE 1.1: NUMERISCHER KERN (IMPLEMENTIERT)

- CFR-Tensor $H[k]$ mit expliziten Dimensionen $N_\mathrm{sc}\times N_\mathrm{rx}\times N_\mathrm{tx}$.
- Open-Loop Spatial Multiplexing, ein Stream je Sendeantenne.
- Gleiche Gesamtleistungsaufteilung über $N_s = N_{\mathrm{tx}}$ Streams.
- Perfekte CFR-Kenntnis am Empfänger und linearer MMSE-Detektor.
- Post-MMSE-SINR pro Subcarrier und Stream.
- Mutual-Information-basierte Kompression über Subcarrier.
- Analytische Unit-Tests für SISO, orthogonale 2x2-Kanäle und frequenzselektive Kanäle.

Abnahmekriterium: analytische Werte werden mit einer Toleranz von 1e-10 reproduziert.

UNTERPHASE 1.2: MIMO-DATENVERTRAG SIONNA -> NS-3 (IMPLEMENTIERT)

- Antennendimensionen rückwärtskompatibel in message.proto ergänzen.
- Flache Reihenfolge sc/rx/tx verbindlich festlegen.
- Sionna-Arrays konfigurierbar machen und die CFR ohne Verlust der Antennendimensionen serialisieren.
- CacheEntry auf MimoChannelMatrix umstellen.
- Python-Shape-Tests und C++-Deserialisierungstests hinzufügen.

Abnahmekriterium: 1x1 ist binär/funktional kompatibel; synthetische 2x2-Werte kommen indexgenau in ns-3 an.

UNTERPHASE 1.3: 802.11AX-LINK-ABSTRAKTION (IMPLEMENTIERT)

- Referenz-SNR aus Sendeleistung, Wideband-Verlust, Bandbreite, Rauschzahl und thermischem Rauschen berechnen.
- Effektive SINR mit einer explizit kalibrierbaren EESM-Abbildung je MCS bestimmen; ohne belegte Kalibrierung existieren bewusst keine Default-Betas.
- Vorhandene ns-3 ErrorRateModel-Schnittstellen soweit möglich wiederverwenden.
- SISO-Parität und AWGN-Kurven gegen ns-3 validieren.

Abnahmekriterium: PER-vs-SNR-Kurven sind monoton, reproduzierbar und im SISO-Grenzfall deckungsgleich zur festgelegten Referenz. Alle Kriterien wurden mit den in Abschnitt 7 beschriebenen automatischen Tests erfüllt.

UNTERPHASE 1.4: WIFIPHY-INTEGRATION UND BEISPIEL (IMPLEMENTIERT)

- Nur den schmalsten notwendigen Empfangspfad in SpectrumWifiPhy anbinden.
- NSS aus WifiTxVector verwenden und ungültige Antennenkonfigurationen früh

  ablehnen.
  - Beispiel ns3_wifi_sumimo_minimal.cc mit 1x1/2x2-Vergleich anlegen.
  - Durchsatz, PER, Stream-SINR und Kanalrang ausgeben.

Abnahmekriterium: 2x2 gewinnt auf gut konditionierten Kanälen und kollabiert
auf Rang-1-Kanälen nachvollziehbar gegen 1x1.
Die automatisierten Ergebnisse in Abschnitt 7 erfüllen dieses Kriterium.

  PHASE 2: HIGH-LEVEL-API

Geplant ist ein SionnaMimoHelper, der mindestens folgende Aspekte gemeinsam
konfiguriert:

  - Arraygeometrie: Zeilen, Spalten, Elementabstand und Polarisation
  - Ausrichtung: Azimut, Elevation und Rollwinkel
  - Betriebsart: SISO, SU-MIMO Spatial Multiplexing, SU-Beamforming, MU-MIMO
  - Zahl der TX-/RX-Antennen, maximale NSS und Empfängertyp
  - Kanalnormalisierung, Rauschzahl und Effective-SINR-Modell

Die API validiert Wi-Fi-, Sionna- und Abstraktionsparameter zentral, sodass
Beispiele keine internen Objekte manuell verdrahten müssen.

Phase 2 wird in drei Unterphasen nach API-Vertrag, interner Integration und
Benutzbarkeit getrennt. Damit bleiben API-Entscheidungen ohne Änderung des
PHY-Verhaltens prüfbar, während Kompatibilitäts- und End-to-End-Regressionen
den nachfolgenden Integrationsschritten eindeutig zugeordnet werden können.

  UNTERPHASE 2.1: API-VERTRAG (IMPLEMENTIERT)

  - SionnaMimoHelper sowie Konfigurationstypen, dokumentierte Defaults und
  unterstützte Betriebsarten definieren.
  - Arraygeometrie, knotenspezifische Ausrichtung, TX-/RX-Antennenzahlen,
  maximale NSS, Empfängertyp, Kanalnormalisierung, Rauschzahl und
  Effective-SINR-Modell in einem konsistenten Vertrag zusammenführen.
  - Zentrale Validierung für vollständige, endliche und untereinander
  konsistente Parameter bereitstellen.
  - In Phase 2 nur SISO und SU-MIMO Spatial Multiplexing als funktionale Modi
  unterstützen. Beamforming und MU-MIMO werden entweder noch nicht angeboten
  oder brechen, falls sie für API-Stabilität bereits repräsentiert werden,
  eindeutig als nicht unterstützt ab.
  - Reine API-Unit-Tests für Defaults sowie gültige und ungültige
  Konfigurationen hinzufügen, ohne das bestehende PHY-Verhalten zu ändern.

Abnahmekriterium: Der API-Vertrag ist unabhängig von einer Simulation
testbar, rückwärtskompatible Defaults sind festgeschrieben und kein Modus aus
Phase 3 oder 4 erscheint fälschlich funktionsfähig.

  UNTERPHASE 2.2: INTEGRATION (IMPLEMENTIERT)

  - SionnaMimoHelper mit den vorhandenen Phase-1-Komponenten für Kanal,
  Link Budget, MMSE, EESM und SpectrumWifiPhy verbinden.
  - Knotenbezogene Arraygeometrie und Ausrichtung über den Sionna-IPC- und
  Serverpfad anwenden.
  - Antennenzahlen, maximale Spatial Streams, Rauschzahl, Empfängertyp,
  Normalisierung und Effective-SINR-Konfiguration konsistent an die
  beteiligten ns-3-Objekte weiterreichen.
  - Den bisherigen SionnaHelper-Pfad und die globalen 1x1-Defaults unverändert
  nutzbar halten.
  - Integrationstests ergänzen, welche manuelle Phase-1-Konfiguration und
  Helper-Konfiguration für 1x1 und 2x2 vergleichen.

Abnahmekriterium: Äquivalente manuelle und High-Level-Konfigurationen liefern dieselben Phase-1-Ergebnisse; bestehende 1x1-Beispiele und Tests bleiben unverändert erfolgreich.

UNTERPHASE 2.3: BENUTZBARKEIT UND VALIDIERUNG (IMPLEMENTIERT)

- Minimale 1x1- und 2x2-Beispiele ausschließlich über die High-Level-API anlegen.
- Integrationsnahe Negativtests für widersprüchliche Laufzeitparameter und Fehlerweitergabe vom Helper bis zum PHY ergänzen.
- End-to-End-Tests vom Helper über Sionna beziehungsweise den deterministischen Kanalpfad bis SpectrumWifiPhy ausführen.
- API, Defaults, unterstützte Modi, Fehlermeldungen und Migration von der manuellen Konfiguration dokumentieren.

Abnahmekriterium: Die 1x1-/2x2-Beispiele laufen ohne manuelle Verdrahtung interner Objekte, ungültige Konfigurationen brechen früh und eindeutig ab und Dokumentation sowie PDF beschreiben den abgeschlossenen Stand von Phase 2. Die helperbasierten End-to-End- und Laufzeit-Negativtests in Abschnitt 7 erfüllen dieses Kriterium.

PHASE 3: SU-MIMO MIT PRECODING UND BEAMFORMING

Sionna liefert weiterhin ausschließlich die komplexe CFR $H[k]$. Auswahl und Anwendung des Precoding $W[k]$, Streamauswahl, Leistungsaufteilung und CSI-Zeitbezug bleiben vollständig in ns-3. Die bestehende skalare Grenze zu SpectrumWifiPhy bleibt erhalten: Erst nach der matrixwertigen Kanal-, Precoding- und Detektorberechnung wird auf eine Effective-SINR beziehungsweise äquivalente Empfangs-PSD komprimiert.

Phase 3 ist inhaltlich größer als Phase 1 oder Phase 2 jeweils für sich: Sie erweitert gleichzeitig Datenvertrag, numerischen Kern, High-Level-API und PHY-Integration und führt zusätzlich rückgemeldete sowie zeitabhängige CSI ein. Die folgenden fünf Unterphasen beschreiben deshalb vertikale, abnehmbare Funktionen; die einzelnen Aufzählungspunkte bleiben interne Implementierungsschritte. Unterphasen 3.1 bis 3.3 bilden den idealen Perfect-CSI-Kern, Unterphase 3.4 ist der eigene Realismus- und Standardisierungsblock und Unterphase 3.5 schließt die Phase end-to-end ab.

UNTERPHASE 3.1: FESTES PRECODING ALS VOLLSTÄNDIGE GRUNDLAGE (IMPLEMENTIERT)

- Einen dimensionssicheren Vertrag für $W[k] \in \mathbb{C}^{N_{tx} \times N_s}$ mit verbindlicher sc/tx/stream-Reihenfolge definieren; frequenzflache Matrizen werden explizit auf alle Subcarrier abgebildet.
- Das Signalmodell auf $y[k] = \sqrt{P/N_s}\, H[k]W[k]x[k] + n[k]$ verallgemeinern und den bestehenden MMSE-Kern auf den effektiven Kanal $H_{\text{eff}}[k] = H[k]W[k]$ anwenden.
- N_s unabhängig von N_tx im Bereich $1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$ führen.
- Die Leistungsnormierung $\|W[k]\|_F^2 = N_s$ festlegen, sodass P weiterhin die gesamte Sendeleistung bezeichnet und kein unbeabsichtigter Precoding-Gewinn entsteht.
- Analytische Tests für SISO, Identitäts-Precoding, Streamauswahl, komplexwertige feste Matrizen und Dimensions-/Normierungsfehler ergänzen.

Numerisches Zwischenkriterium: $W[k] = I$ reproduziert den Phase-1-Kern innerhalb 1e-10;
synthetische $H[k]W[k]$-Referenzen liefern indexgenau dieselben Post-MMSE-SINRs und die Gesamtleistung bleibt invariant.

Integrationsschritte innerhalb derselben Unterphase:

- Benutzergegebenes frequenzflaches und frequenzselektives $W[k]$ durch SionnaMimoHelper, Link-Abstraktion und SpectrumWifiPhy-Adapter führen.
- WifiTxVector::GetNss an die Spaltenzahl von $W[k]$ statt weiterhin zwingend an N_{tx} koppeln und $N_s < N_{tx}$ erstmals kontrolliert zulassen.
- Subcarrierzahl, Antennenmaße, NSS, endliche Koeffizienten und Leistungsnormierung vor jeder Zustandsänderung zentral validieren.
- Ohne explizites Precoding den bisherigen SISO-/Open-Loop-Pfad byte- und funktional kompatibel halten.
- API-, Adapter- und End-to-End-Tests für 1x1, 2x2-Identität sowie $N_{tx} = 2$, $N_{rx} = 2$, $N_s = 1$ ergänzen.

Abnahmekriterium: Manuell und über SionnaMimoHelper konfiguriertes festes $W[k]$ liefern dieselben SINR-, EESM- und PER-Ergebnisse; alle Phase-2-Tests bleiben unverändert erfolgreich. Die analytischen, Adapter- und SpectrumWifiPhy-End-to-End-Ergebnisse in Abschnitt 7 erfüllen dieses Kriterium.

UNTERPHASE 3.2: EIN-STREAM-MRT/MRC (IMPLEMENTIERT)

- MRT für den eindeutig definierten MISO-Fall $N_{rx} = 1$ aus perfekter CFR berechnen und auf Einheitsnorm normieren.
- Für $N_{tx} = 1$ die bestehende Ein-Stream-MMSE-Lösung als äquivalente MRC-Kombination ausweisen und validieren.
- Den allgemeinen Fall $N_{tx} > 1$ und $N_{rx} > 1$ nicht mehrdeutig als MRT/MRC bezeichnen; dessen optimale Ein-Stream-Lösung gehört zur SVD-Unterphase.
- Analytische MISO-/SIMO-Kanäle, gemeinsame Phasendrehung, Nullkanal und erwarteten Arraygewinn testen.

Abnahmekriterium: MRT/MRC erreicht auf analytischen Ein-Stream-Kanälen die Post-Combining-SNR $\rho\|h\|_2^2$, bleibt leistungsnormalisiert und fällt im 1x1-Grenzfall exakt auf das bestehende Ergebnis zurück. Der analytische MRT-/MRC-Test in Abschnitt 7 erfüllt dieses Kriterium einschließlich Phaseninvarianz und eindeutiger Ablehnung der Null- und allgemeinen MIMO-Fälle.

UNTERPHASE 3.3: SVD-PRECODING UND LEISTUNGSALLOKATION (IMPLEMENTIERT)

- Pro Subcarrier $H[k] = U[k]\Sigma[k]V^H[k]$ zerlegen und die N_s stärksten rechten Singulärvektoren bei zunächst gleicher Streamleistung als $W[k]$ verwenden.
- N_s explizit wählen und gegen Antennenzahlen sowie einen dokumentierten numerischen Rangschwellwert validieren.
- Sortierung, Vorzeichen-/Phasenkonvention und Verhalten bei gleichen Singularwerten deterministisch festlegen.
- Diagonale, unitär gedrehte, frequenzselektive und rangdefiziente Referenzkanäle gegen analytische Eigenmode-SINRs testen.

Zwischenkriterium für SVD und Streamauswahl: Die Eigenmode-SINRs stimmen innerhalb 1e-10 mit
den Singularwertreferenzen überein; Rangverlust wird durch eine explizite Streamwahl behandelt und nicht durch versteckte Regularisierung kaschiert.

Optionale Leistungsallokation innerhalb derselben Unterphase:

- Gleiche Leistung als reproduzierbaren Standard beibehalten und Wasserfüllung als explizit auswählbare Forschungsoption ergänzen.
- Den Umfang zunächst auf räumliche SVD-Moden je Subcarrier begrenzen, damit spektrale Maske und Gesamtleistung pro Subcarrier unverändert bleiben.
- Rauschleistung, Referenz-SNR, aktive Moden und die Summe der zugewiesenen

Leistung vollständig ausgeben und validieren.
- Analytische Zwei-Moden-Fälle unterhalb und oberhalb der Aktivierungsschwelle sowie Leistungserhaltung und KKT-Bedingungen testen.

Abnahmekriterium der optionalen Erweiterung: Die Leistungsanteile sind endlich, nicht negativ und
summieren sich exakt zur vorgegebenen Gesamtleistung; Wasserfüllung erreicht auf den Referenzkanälen mindestens die ideale Rate der geprüften Gleichleistungsreferenz, ohne Sendeleistung zu erzeugen.

UNTERPHASE 3.4: QUANTISIERTE RÜCKMELDUNG UND VERALTETE CSI (IMPLEMENTIERT)

- Vor der Implementierung das konkret unterstützte HE-/EHT-Feedbackformat, Winkel-/Codebookdarstellung, Bitbreiten und Rekonstruktionsreihenfolge verbindlich dokumentieren; nicht unterstützte Standardvarianten klar abgrenzen.
- Ideales SVD-Precoding, Quantisierung und Rekonstruktion als getrennte, einzeln testbare Schritte modellieren.
- Quantisiertes $W[k]$ erneut leistungsnormalisieren und Feedbackumfang sowie Quantisierungsfehler als Diagnosegrößen bereitstellen.
- Bekannte Vektoren, Grenzwinkel, minimale/maximale Quantisierung und den unquantisierten Grenzfall testen.

Zwischenkriterium für die Quantisierung: Bitgenaue Referenzvektoren werden deterministisch
rekonstruiert, die Precoder bleiben leistungsnormalisiert und feinere Quantisierung verschlechtert die Referenzmetrik auf den Testkanälen nicht.

CSI-Alterung innerhalb derselben Unterphase:

- Die zur Precoderwahl verwendete Schätzung $\hat{H}[k, t_{\mathrm{csi}}]$ strikt von der bei der Paketübertragung wirksamen CFR $H[k, t_{\mathrm{tx}}]$ trennen.
- CSI-Alter, Aktualisierungsintervall und Feedbackverzögerung explizit und reproduzierbar konfigurieren; perfektes aktuelles CSI bleibt der Nullalter-Grenzfall.
- Richtung, Arraykonfiguration, Precodingmodus und CSI-Zeitstempel in Cache- beziehungsweise Zustandsidentität berücksichtigen.
- Statische Kanäle, kontrollierte Phasen-/Subraumrotationen und wiederholte Läufe mit identischen Seeds testen.

Abnahmekriterium: CSI-Alter null reproduziert das ideale Precoding exakt; kontrolliert veraltete Schätzungen werden nachweislich nur zur Wahl von W verwendet, während SINR und PER stets mit dem aktuellen H berechnet werden. Die bitgenauen Quantisierungs-, Leistungs-, Timing-, Subraum-, SINR- und PER-Regressionen in Abschnitt 7 erfüllen dieses Kriterium für den in Abschnitt 5.12 abgegrenzten SU-Feedbackumfang.

UNTERPHASE 3.5: BENUTZBARKEIT UND PHASENVALIDIERUNG (IMPLEMENTIERT)

- SionnaMimoHelper erst nach erfolgreicher Implementierung die jeweiligen Beamforming-/Precodingmodi als unterstützt melden lassen.
- Das Beispiel ns3_wifi_sumimo_precoding.cc anlegen und Open-Loop, festes Precoding, MRT sowie SVD bei gleicher Gesamtleistung vergleichen; Wasserfüllung, Quantisierung und CSI-Alter werden als explizite Optionen zugeschaltet.
- Durchsatz, PER, Stream-SINR, Kanalrang, Precodingnorm, Leistungsanteile und CSI-Alter maschinenlesbar ausgeben.
- Negative Konfigurationstests und vollständige CPU-End-to-End-Regressionen ergänzen sowie API-Migration und Modellgrenzen dokumentieren.

Abnahmekriterium: Die Beispiele benötigen keine manuelle Verdrahtung interner Objekte, analytisch überlegene Precoder zeigen den erwarteten Gewinn ohne zusätzliche Sendeleistung, ungültige Konfigurationen brechen früh ab und alle Phase-1-/2-Regressionen bleiben erfolgreich.

Dieses Kriterium ist erfüllt: Der Helper meldet SU_BEAMFORMING erst mit dieser Integration als unterstützt, das neue Vergleichsbeispiel verwendet nur die öffentliche High-Level-Konfiguration und SelectPrecoding, alle vier Modi halten dieselbe Gesamtleistungsdefinition ein, ungültige Kombinationen brechen vor Zustandsänderungen ab und die CPU-Regressionen in Abschnitt 7 decken die alten sowie die neuen Pfade ab.

  PHASE 4: MU-MIMO

Sionna liefert auch in Phase 4 ausschließlich gerichtete komplexe CFRs \(H_u[k]\) für die einzelnen Verbindungen. Gemeinsame Nutzerwahl, Precoding, Leistungsaufteilung, Inter-User-Interferenz, Scheduling, Empfängermodell und PER-Abbildung bleiben vollständig in ns-3. Eine MU-Übertragung darf nur Kanäle aus einem konsistenten gemeinsamen Zeitschnappschuss verwenden.

Phase 4 ist größer als die vorherigen Einzelphasen. Insbesondere sind Downlink und Uplink nicht symmetrisch: Im Downlink sendet ein AP mit einem gemeinsamen Precoder, während im Uplink mehrere unabhängige Sender am AP gemeinsam detektiert werden. Die Phase wird daher in acht vertikale Unterphasen geteilt. Die Unterphasen 4.1 bis 4.6 liefern zuerst einen vollständig nutzbaren Downlink-MU-MIMO-Pfad; 4.7 und 4.8 ergänzen danach den Uplink, ohne den Abschluss des Downlink-Pfads zu blockieren.

  UNTERPHASE 4.1: MEHRNUTZER-KANALSCHNAPPSCHUSS UND IDENTITÄT (IMPLEMENTIERT)

  - Einen dimensionssicheren Vertrag für gerichtete CFRs \(\{H_u[k]\}_{u=1}^{U}\) mit stabiler Nutzer-ID, Sender, Empfänger, Richtung und gemeinsamem CSI-Zeitstempel definieren.
  - Subcarrierzahl, Frequenzraster, Normalisierung und die Trennung zwischen CFR und skalarem Wideband-Pfadverlust über alle Nutzer konsistent halten.
  - Unterschiedliche Empfangsarraygeometrien und Antennenzahlen je Nutzer repräsentieren; die bisherige gemeinsame RX-Geometrie darf nicht stillschweigend auf alle Stationen übertragen werden.
  - Vorhandene gebündelte Sionna-Anfragen und Cachepfade soweit möglich wiederverwenden und gemischte Zeitstempel, doppelte Nutzer, inkonsistente Dimensionen oder Richtungen vor jeder Zustandsänderung ablehnen.
  - Python- und C++-Vertragstests für Nutzerreihenfolge, Serialisierung, Tensorindizes, heterogene RX-Dimensionen und Zeitidentität ergänzen.

Abnahmekriterium: Synthetische Mehrnutzer-CFRs kommen unabhängig von ihrer Eingabereihenfolge nutzer- und indexgenau in ns-3 an; jeder inkonsistente Schnappschuss wird früh abgelehnt und alle bisherigen SU-Verträge bleiben unverändert erfolgreich.

Dieses Kriterium ist erfüllt: Die vorhandene `ChannelState`-/`RxNodeInfo`-Bündelung trägt eine additive `UserChannelIdentity`; Python und C++ prüfen den gesamten Schnappschuss vor Verwendung, C++ sortiert nach stabiler Nutzer-ID und der Cache übernimmt erst danach alle Links gemeinsam. Die neuen synthetischen Verträge decken heterogene RX-Arrays ab. Sämtliche Phase-1- bis Phase-3-SU-Regressionen bleiben bestanden; Unterphase 4.2 wurde nicht vorweggenommen.

  UNTERPHASE 4.2: FESTES DOWNLINK-MU-PRECODING UND INTERFERENZMODELL (IMPLEMENTIERT)

  - Einen festen gemeinsamen Precoder

```
  \[
    W[k] = \left[W_1[k],\ldots,W_U[k]\right]
  \]

  mit expliziter Zuordnung seiner Spalten zu Nutzer und Stream definieren.
  - Die Phase-3-Leistungskonvention auf die Gesamtzahl
  \(N_{s,\mathrm{ges}}\) aller gleichzeitig gesendeten Streams erweitern:
  \(\Vert W[k]\Vert_F^2=N_{s,\mathrm{ges}}\), sodass \(P\) weiterhin die
  gesamte AP-Sendeleistung bezeichnet. Jede abweichende Nutzer- oder
  Streamgewichtung muss explizit sein und in Summe dieselbe Gesamtleistung
  erhalten.
  - Für jeden Nutzer gewünschten Anteil und Inter-User-Interferenz getrennt
  berechnen:

  \[
    y_u[k] = H_u[k]W_u[k]x_u[k]
    + \sum_{v\ne u} H_u[k]W_v[k]x_v[k] + n_u[k].
  \]

  - Den bestehenden linearen Empfänger mit der vollständigen
  Interferenzkovarianz auswerten und gewünschte Leistung,
  Interferenzleistung sowie Post-Processing-SINR je Nutzer, Subcarrier und
  Stream ausgeben.
  - EESM und PER anschließend separat mit dem MCS und der Paketgröße des
  jeweiligen Nutzers bestimmen.

Abnahmekriterium: Ein einzelner Nutzer reproduziert Phase 3 innerhalb
\(10^{-10}\); orthogonale Nutzer erzeugen keine Inter-User-Interferenz,
korrelierte Referenzkanäle liefern die analytische Interferenz und die
Gesamtsendeleistung bleibt invariant.

Dieses Kriterium ist erfüllt: `DlMuMimoPrecodingMatrix` ordnet jede
Spalte des unveränderten Phase-3-Precoders explizit einem Nutzer und lokalen
Stream zu. `DlMuMimoLinkModel` wertet die vollständige Empfangskovarianz
aus und weist gewünschte Leistung, Inter-User-Interferenz und SINR getrennt
aus. `HeDlMuMimoLinkAbstraction` wendet den bestehenden EESM- und
ErrorRateModel-Vertrag anschließend nutzerweise an. Analytische native Tests
erfüllen alle vier Invarianten; Unterphase 4.3 wurde nicht vorweggenommen.

  UNTERPHASE 4.3: IDEALES DOWNLINK-ZF UND REGULARISIERTES ZF (IMPLEMENTIERT)

  - Zunächst den eindeutigen Minimalfall mit einem Stream je Nutzer und
  perfekter gleichzeitiger CSI implementieren.
  - ZF aus dem gestapelten Mehrnutzerkanal und RZF mit explizit
  konfigurierbarem Regularisierungsparameter berechnen; es gibt keinen
  versteckten Regularisierungsdefault.
  - Rangschwelle, Nutzerreihenfolge, Pseudoinverse und deterministische
  Behandlung gleicher beziehungsweise nahezu gleicher Singulärwerte
  dokumentieren.
  - Den gemeinsamen Precoder nach jeder Auswahl auf die Konvention aus
  Unterphase 4.2 normalisieren und überlastete oder rangdefiziente
  Nutzergruppen eindeutig ablehnen beziehungsweise als nicht planbar melden.
  - Orthogonale, korrelierte, überlastete und rangdefiziente analytische
  Referenzkanäle testen.

Abnahmekriterium: ZF unterdrückt auf vollrangigen Referenzkanälen die
Inter-User-Interferenz innerhalb der numerischen Toleranz; RZF bleibt auf
schlecht konditionierten Kanälen endlich, deterministisch und
leistungsnormalisiert. Mehrere Streams je Nutzer werden erst nach einem
explizit dokumentierten Blockdiagonalisierungs- oder gemeinsamen
Transceiververtrag freigeschaltet und bis dahin klar abgelehnt.
```

Dieses Kriterium ist erfüllt: `DlMuMimoPrecodingMatrix::ZeroForcing` bildet die Moore-Penrose-Pseudoinverse des nach stabiler Nutzer-ID gestapelten Einantennenkanals; `RegularizedZeroForcing` verwendet ausschließlich das explizit übergebene positive RZF-α. Beide Fabriken prüfen den vollständigen Phase-4.1-Snapshot und die Ein-Stream-Zuordnung vor Konstruktion, lehnen Überlast und Rangverlust ab und skalieren erst das fertige Ergebnis auf die Phase-4.2-Gesamtleistung. Die schnellen nativen Referenzfälle in Abschnitt 7.5 erfüllen das Abnahmekriterium.

  UNTERPHASE 4.4: HE-DOWNLINK-MU-PHY-INTEGRATION (KORRIGIERT UND VALIDIERT)

  - Den gemeinsamen MU-Zustand in den schmalsten geeigneten SpectrumWifiPhy-Pfad für `WIFI_PREAMBLE_HE_MU` einkoppeln.
  - Die vorhandenen STA-ID-, RU-, MCS- und NSS-Angaben aus `WifiTxVector::HeMuUserInfoMap` verwenden, anstatt parallele Wi-Fi-Metadaten im Sionna-Modul einzuführen.
  - Inter-User-Interferenz nur zwischen räumlich gemultiplexten Nutzern innerhalb derselben RU berücksichtigen; verschiedene RUs bleiben orthogonal.
  - Für jeden Empfänger erst nach der matrixwertigen MU-Berechnung auf dessen skalare äquivalente Empfangs-PSD komprimieren. Pfadverlust, AP-Sendeleistung, Rauschen und Empfangsgewinn dürfen dabei jeweils genau einmal eingehen.
  - Positive und negative Adaptertests für Nutzer-/STA-ID-Zuordnung, RU, NSS, Antennendimensionen und unbekannte Empfänger ergänzen.

Abnahmekriterium: Ein echter HE-DL-MU-PPDU liefert je adressiertem Empfänger die analytisch erwartete SINR und PER; nicht adressierte oder inkonsistente Empfänger verändern keinen MU-Zustand. Der bestehende SU-Pfad bleibt bit- und funktionskompatibel. EHT bleibt bis zu einer expliziten EHT-Link-/PER-Abstraktion als nicht unterstützt gekennzeichnet.

Die erste Implementierung vom 3. September prüfte die direkte PPDU-/Adapterverdrahtung. Der Review vom 5. September zeigte Fehler in der RU-Frequenzauswahl, RU-Leistungsnormierung, gemeinsamen Präambel und Behandlung mithörender Stationen. Die Korrektur ergänzt explizite belegte Tonindizes, gruppenweise Leistungsnormierung und getrennte Nicht-HE-/Nutzdatenpfade. Die neuen numerischen und echten Kanal-/PHY-Empfangstests sowie der tatsächliche Validierungsstand stehen in Abschnitt 7.9. Der genaue Arbeitsplan steht in `doc/mu-mimo-4.4-improvement-plan-2026-09-07.md`.

  UNTERPHASE 4.5: LAUFZEIT-CSI UND SCHEDULER-SCHNITTSTELLE (IMPLEMENTIERT)

  - Einen gerichteten, zeitgestempelten CSI- und Precoderzustand pro AP, Nutzergruppe, RU und Übertragungsrichtung verwalten; veraltete oder für eine andere Gruppe erzeugte Zustände dürfen nicht wiederverwendet werden.
  - An die vorhandene ns-3-`MultiUserScheduler`-Schnittstelle anbinden und zunächst deterministische Nutzergruppen sowie RU-/NSS-Zuordnungen unterstützen.
  - Danach eine explizit auswählbare korrelationsbasierte Gruppierung ergänzen, welche unbelegbare Gruppen verwirft und ihre Auswahlmetrik ausgibt.
  - CSI-Aktualisierungsintervall, Feedbackverzögerung und den tatsächlichen Sendezeitpunkt aus Phase 3 übernehmen. Quantisierte MU-Rückmeldung erhält einen eigenen Vertrag; ein rekonstruierter SU-Precoder darf nicht stillschweigend als vollständige MU-Kanalschätzung behandelt werden.
  - Wiederholte Läufe mit identischen Seeds sowie kontrollierte Änderungen von Nutzerreihenfolge, Korrelation und CSI-Alter testen.

Abnahmekriterium: Scheduler und PHY verwenden nachweislich denselben gerichteten Kanalschnappschuss; unpassende oder veraltete Zustände werden erkannt und identische Eingaben erzeugen exakt dieselbe Nutzergruppe,

RU-/NSS-Zuordnung und denselben Precoder.

Dieses Kriterium ist erfüllt: DlMuMimoRuntime übernimmt Kandidaten und
RU-/NSS-Metadaten direkt aus MultiUserScheduler::DlMuInfo, bindet STA-IDs
explizit an stabile Kanalnutzer und liefert eine deterministische feste oder
korrelationsbasierte Gruppenauswahl. Erst wenn der Scheduler genau diese
Auswahl für denselben Sendezeitpunkt ausgibt, installiert Install den
unveränderten Snapshot und ZF-/RZF-Precoder im Phase-4.4-Adapter. Falsche,
veraltete, anders gerichtete oder nicht unterstützte Zustände werden vor
jeder Änderung abgelehnt. Unterphase 4.6 verwendet den getrennten Übertragungsschnappschuss und
ersetzt den CSI-Zustand dabei nicht.

  UNTERPHASE 4.6: DOWNLINK-BENUTZBARKEIT UND PHASENVALIDIERUNG (IMPLEMENTIERT)

  - `SionnaMimoHelper` den Modus `MU_MIMO` erst nach erfolgreicher
  End-to-End-Integration als unterstützt melden lassen.
  - Das Beispiel `ns3_wifi_mumimo.cc` anlegen und SU-Round-Robin, festes
  MU-Precoding, MU-ZF und MU-RZF bei gleicher Gesamtleistung und vergleichbarer
  Sendezeit gegenüberstellen.
  - Orthogonale und stark korrelierte Nutzergruppen sowie aktuelles und
  veraltetes CSI vergleichen.
  - Pro Nutzer Durchsatz, PER, Stream-SINR, gewünschte Leistung,
  Inter-User-Interferenz, RU, NSS und CSI-Alter sowie für die Gruppe Rang,
  Precodingnorm, Gesamtleistung und Fairness maschinenlesbar ausgeben.
  - Vollständige CPU-End-to-End- und Negativtests sowie alle Regressionen der
  Phasen 1 bis 3 ausführen.

Abnahmekriterium: MU-ZF erzielt bei ausreichend unkorrelierten Nutzern den
erwarteten gleichzeitigen Übertragungsgewinn gegenüber SU-Round-Robin, ohne
zusätzliche AP-Sendeleistung zu erzeugen; bei hoher Korrelation verschlechtert
sich das Ergebnis nachvollziehbar oder die Gruppe wird abgelehnt. Beispiele
benötigen keine manuelle Verdrahtung interner Objekte.

Dieses Kriterium ist erfüllt: `InstallDownlink` übernimmt einen AP, stabile
Nutzerbindungen, Spectrumadapter und einen zeitabhängigen Snapshotanbieter in
ein AP-eigenes `MultiUserScheduler`-Objekt. Die Installation verlangt zwei bis
acht AP-Sendeantennen, eine Empfangsantenne und NSS eins je Station, eine
explizite MU-Laufzeitkonfiguration sowie ein EESM-Beta für den verwendeten MCS.
Am tatsächlichen `PhyTxPsduBegin`-Zeitpunkt wählt sie CSI erneut, behält die
ausgegebene Gruppe bei und installiert den physischen Downlink-Snapshot getrennt
vom CSI-Snapshot. Der Adapter verwendet damit auch für SU-Empfang und
Mithörer die gerichtete Übertragungs-CFR.

Das CPU-Beispiel verwendet einen Zweiantennen-AP und zwei Einantennen-HE-
Stationen auf Kanal 36 mit 20 MHz, 800 ns GI, 7 dB Rauschzahl, 0 dBm AP-Leistung,
MCS 5, statischem ARP sowie einem MPDU je Nutzer und PPDU. Sein
frequenzflacher Kanal lautet

$$
 h_1(t)=[1,e^{j\theta(t)}],\qquad
 h_2(t)=[1,e^{j(\theta(t)+\phi)}],\qquad
 \theta(t)=2\pi f t .
$$

Dabei sind $\phi=\pi$ und $\phi=0{,}2$ die orthogonale und korrelierte
Variante; $f=25\,\mathrm{Hz}$. Aktuelles CSI verwendet Nullintervall und
Nullverzögerung, altes CSI 5 ms und 2 ms. Der Verifikationslauf mit
`--eesmBetaLinear=2 --rzfAlpha=0.25 --verify=1` bestätigt die konstante
AP-Leistung von 0,001 W, zwei adressierte Nutzer pro HE-MU-PPDU, endliches
normalisiertes RZF, nichtverschwindende Alterungsinterferenz und den erwarteten
ZF-Unterschied: orthogonales aktuelles ZF erreicht 75,226685740 Mbit/s je

Airtime-Sekunde gegenüber 49,586776860 für SU-MRT. Die Ausgabe trennt tatsächliche PHY-Drops und empirische PER von der numerischen EESM/PER-Prognose.

  UNTERPHASE 4.7: UPLINK-MU-NUMERIKKERN (IMPLEMENTIERT)

  - Das Uplink-Signalmodell

  \[
    y[k] = \sum_{u=1}^{U} H_u[k]x_u[k] + n[k]
  \]

  mit getrennten Nutzer-Sendeleistungen, Pfadverlusten und Streamzuordnungen definieren.
  - Zunächst ideale Zeit- und Frequenzsynchronisation sowie einen Stream je Nutzer annehmen und diese Grenze ausdrücklich dokumentieren.
  - Gemeinsame ZF- und lineare MMSE-Detektion am AP implementieren und Post-Processing-SINR, Inter-User-Interferenz und PER je Nutzer bestimmen.
  - Nah-Fern-Fälle, orthogonale Nutzer, Rangverlust, unterschiedliche Sendeleistungen und den Einzelnutzer-Grenzfall analytisch testen.

Abnahmekriterium: Der Einzelnutzerfall reproduziert die vorhandene SU-Empfangsabstraktion; analytische Mehrnutzerkanäle und Nah-Fern-Fälle liefern die erwarteten ZF-/MMSE-SINRs, ohne Nutzerleistungen zu vermischen.

Dieses Kriterium ist erfüllt: `UlMuMimoLinkModel` stapelt die gerichteten Ein-Stream-Uplinkkanäle am gemeinsamen AP, hält jede Nutzer-SNR getrennt und liefert für ZF beziehungsweise lineares MMSE gewünschte Leistung, Inter-User-Interferenz, verstärkte Rauschleistung und SINR je Subcarrier. `HeUlMuMimoLinkAbstraction` verwendet danach nutzerweise den vorhandenen EESM-/ErrorRateModel-Vertrag. Der Einzelnutzer-, Orthogonal-, Rangverlust- und Nah-Fern-Fall stimmen mit den analytischen Referenzen überein. HE-TB-/Trigger- und Gleichzeitigkeitseinbindung bleiben ausschließlich Unterphase 4.8.

  UNTERPHASE 4.8: UPLINK-PHY-INTEGRATION UND GESAMTABSCHLUSS (IMPLEMENTIERT)

  - Den Uplink-Kern an koordinierte HE-TB-/UL-MU-Übertragungen, Triggerrahmen, STA-ID-, RU- und MCS-Zuordnungen anbinden.
  - Sicherstellen, dass nur tatsächlich gleichzeitig empfangene Nutzer in denselben Detektionszustand eingehen; fehlende, verspätete oder inkonsistent konfigurierte Teilnehmer werden explizit behandelt.
  - Das Phasenbeispiel um UL-MU gegenüber sequenziellen SU-Übertragungen erweitern und die ideale Synchronisationsannahme in der Ausgabe sichtbar machen.
  - Alle API-, Numerik-, PHY-, Scheduler-, CPU-End-to-End- und Legacytests ausführen sowie Dokumentation und PDF abschließend aktualisieren.

Abnahmekriterium: Eine koordinierte UL-MU-Gruppe liefert pro Nutzer reproduzierbare SINR-, PER- und Durchsatzwerte; der erwartete Gleichzeitigkeitsgewinn sowie Nah-Fern- und Rangverlusteffekte sind sichtbar, und alle Phase-1- bis Phase-3-Regressionen bleiben erfolgreich.

Dieses Kriterium ist erfüllt: `HeUlMuMimoPhyAdapter` validiert einen HE-Basic-Trigger gemeinsam mit den tatsächlich am AP beobachteten `HePpdu`-Objekten. Stabile Nutzer-, STA- und Senderbindungen sowie Trigger-UID, HE-TB-Präambel, PPDU-Dauer, NSS, MCS-Beta und AP-Antennendimension werden vor der Rechnung geprüft. Nur PPDUs mit exakt dem expliziten gemeinsamen Empfangsintervall werden nach RU gruppiert; verschiedene RUs bleiben orthogonal. Fehlende und nicht gleichzeitige STA-IDs werden getrennt im Ergebnis ausgewiesen. Das erweiterte CPU-Beispiel vergleicht dieselben orthogonalen Nutzer sequenziell und gleichzeitig, kennzeichnet die ideale Synchronisation maschinenlesbar und zeigt außerdem Nah-Fern- sowie

Rangverlustverhalten.

  PHASE 5: KALIBRIERUNG, SKALIERUNG UND REPRODUZIERBARKEIT

  UNTERPHASE 5.1: HE-EESM-/PER-KALIBRIERUNG GEGEN LINK-LEVEL-REFERENZEN

  - Die AWGN-Abbildung des tatsächlich eingesetzten ns-3-
  `TableBasedErrorRateModel` für genau einen HE-MCS, Codeart, Paketlänge,
  NSS, RX-Antennenzahl, Kanalbreite und Guard Interval als CSV exportieren.
  - Unabhängige Link-Level-Messungen für AWGN und frequenzselektive Kanäle mit
  Paketfehlern, Paketanzahl und den zugehörigen linearen Post-Processing-SINRs
  aller übertragenen Subcarrier/Streams einlesen. Datensatzquelle, Version
  und PHY-Konfiguration müssen explizit vorhanden sein.
  - Zuerst einen expliziten AWGN-SNR-Korrekturwert auf AWGN-Trainingspunkten
  anhand der Binomial-Likelihood bestimmen und an zurückgehaltenen
  AWGN-Punkten prüfen. Danach das lineare EESM-Beta auf
  frequenzselektiven Trainingskanälen fitten und auf einer anderen
  Kanalrealisation prüfen. Ein Beta allein kann eine abweichende
  AWGN-Basiskurve nicht korrigieren.
  - Die AWGN-Korrektur nur bei exakt passender PHY-Konfiguration auf die
  `ErrorRateModel`-Eingabe anwenden; EESM und die physikalische SINR bleiben
  unverändert. Synthetische Selbsttests und abgelehnte Fits geben nur
  Diagnosekandidaten aus. Es gibt weiterhin keine Default-Koeffizienten.

`tools/generate_he_link_level_reference.m` erzeugt nun eine unabhängige
Bit-Level-Referenz mit MATLAB WLAN Toolbox R2026a, idealem Timing und
perfekter RX-CSI, HE-SU-BCC-Decodierung sowie gemessenen Post-OFDM-SINRs.
Die ersten 20 Messpunkte mit je 1000 Paketen sind in `data/` versioniert.
Der Workflow hat für HE MCS 0, BCC, 32 Byte, 1x1, 20 MHz und 800 ns GI
eine Konfiguration mit zurückgehaltenen AWGN- und Zweitap-Kanälen
validiert. Die Koeffizienten sind nur für diese genaue PHY-Konfiguration
und die dokumentierten Kanalannahmen belegt; weitere MCS, Paketlängen,
Codearten, Antennenmaße und TGax-Kanalmodelle bleiben zu kalibrieren.

  UNTERPHASE 5.2.1: 2X2-SU-MIMO, ABSTAND UND ORIENTIERUNG (IMPLEMENTIERT)

Der erste Teil von Phase 5.2 untersucht einen gerichteten, statischen
2x2-Open-Loop-Link bei fester Szene, Knotenposition, Leistung, Rauschen und
Seeds. Die vollständige kleine Vergleichsmatrix enthält die gemeinsamen
horizontalen TX-/RX-Abstände 0,25, 0,5 und 1 Wellenlänge sowie unabhängig
TX- und RX-Azimut 0 oder 90 Grad: insgesamt zwölf Konfigurationen mit je zwei
frischen Raytracing-Läufen. Elevation und Roll bleiben im Sweep null.
Ein separater Integrationstest prüft zusätzlich unterschiedliche, nichtnullige
Azimut-/Elevations-/Rollwinkel an beiden tatsächlichen Sionna-Radioobjekten.

`examples/ns3_wifi_sumimo_benchmark.cc` exportiert über die vorhandene
`SionnaMimoHelper::Apply`-/`PopulateMimoConfiguration`-Kette die echten
Array-/Orientierungsfelder. Seine zweite Aktion liest eine eingefrorene
Serverantwort und berechnet ausschließlich in ns-3 Linkbudget, Open-Loop-MMSE,
MI-Kompression und SVD-Diagnosen. `tools/benchmark_sumimo.py` ruft den
unveränderten `SionnaEnv` direkt auf, orchestriert die zwölf Fälle und erstellt
CSV-/Protobuf-/JSON-Exporte sowie Vergleichsplots. Ein laufender ZMQ-Server,
MAC-Verkehr und eine EESM-/PER-Konfiguration sind für diesen Messvertrag nicht
erforderlich. Die bestehenden Framework-/Serverpfade wurden nicht geändert.

Die versionierte CPU-Referenz liegt unter `data/phase5_2_spacing_orientation/`.
`manifest.json` enthält sämtliche festen Kontrollen, die zwölf vollständigen
IPC-Konfigurationen, tatsächlich beobachtete Arraykoordinaten und Orientierungen,
Softwareversionen, Backend sowie SHA-256-Hashes von Quellcode, Szenendateien,
Konfigurationen, Kanälen und nativem Programm. Pro Fall sind
`configuration.pb` als rohe `SimInitMessage` und zwei `channel_XX.pb` als rohe

`ChannelStateResponse` gespeichert. `tones.csv` enthält alle 6144 Tonzeilen; `summary.csv` enthält 24 Läufe mit beiden Stream-SINRs, minimalem Rang, Singularwertdiagnosen, Linkbudget und getrennten Laufzeiten. Drei lokal erzeugte PNG-Plots vergleichen Kanal, SINR und Laufzeit über den Antennenabstand. Generierte Plots werden nicht versioniert; `.gitignore` schließt `plots/` sowie PNG-/SVG-/PDF-Dateien unter `data/` aus. Die Referenzvalidierung benötigt keine Plotdateien.

Abnahmekriterium: wiederholte CPU-Kanalantworten sind byteidentisch; Geometrie und Grad-/Radiantumrechnung erreichen die tatsächlichen PlanarArrays und Radioobjekte; analytische sowie eingefrorene reale Kanäle liefern die Referenz-Singularwerte und -SINRs. Die elf automatisierten neuen Prüfungen und die relevanten Regressionen sind in Abschnitt 7.14 dokumentiert. Dieser Teil berichtet keine absolute PER oder Wi-Fi-Datenrate: die gemessene Kalibrierung aus Unterphase 5.1 gilt ausschließlich für die dokumentierte 1x1-Konfiguration und darf nicht auf 2x2 übertragen werden.

  UNTERPHASE 5.2.2: RAYTRACING-SAMPLEKONVERGENZ (IMPLEMENTIERT)

`tools/benchmark_rt_convergence.py` erweitert den bestehenden 2x2-Messvertrag um 10000, 100000 und 1000000 Samples je Quelle. Die vorab anhand der 5.2.1-Referenz ausgewählten Fälle 03, 05, 06 und 10 decken schwache, mittlere und gute Konditionierung sowie alle drei Arrayabstände ab. Jeder Fall nutzt dieselben Seeds 42, 43 und 44 bei allen Budgets und jeweils zwei frische CPU-Raytraces: 72 Läufe insgesamt. Szene, Positionen, Arraymuster, Polarisation, Orientierungen, Leistung, Rauschen und sämtliche anderen Solveroptionen bleiben je Fall unverändert.

Der Runner schreibt `plan.json` mit festen Toleranzen vor dem ersten Trace. `convergence.csv` enthält alle gepaarten Budgetvergleiche; die Entscheidung in `convergence.json` verlangt die Einhaltung jedes Kriteriums für jeden Ton, Stream, Fall und Seed. Die vollständigen Konfigurationen, Kanalantworten, Ton-/Zusammenfassungsmetriken und Laufzeiten liegen in `data/phase5_2_sample_convergence/`. Vier Vergleichsplots bleiben lokal und unversioniert. Messvertrag und Formeln stehen in Abschnitt 5.21, Ergebnisse und automatisierte Validierung in Abschnitt 7.15. PER wird weiterhin nicht abgeleitet: die 5.1-Kalibrierung gilt ausschließlich für die dokumentierte HE-MCS-0/BCC/32-Byte/1x1-Konfiguration.

  UNTERPHASE 5.2.3: CACHE-/IPC-SKALIERUNG (IMPLEMENTIERT)

`examples/ns3_sionna_cache_ipc_benchmark.cc` nutzt die bestehenden `SionnaPropagationCache`-Getter, deren Misspfad und den unveränderten `SionnaHelper`-REQ-Socket. `tools/benchmark_cache_ipc.py` startet einen separaten Python-REP-Prozess mit vorserialisierten eingefrorenen Antworten. Der Sweep umfasst 64/256/1024 Subcarrier, TX/RX-Dimensionen 1/1, 2/2, 4/4, 2/4 und 8/8 sowie 1/16/64 gleichzeitig gecachte Links: 45 Fälle mit je drei frischen nativen Prozessen. Seed 42 und Run 1 bleiben fest.

Pro Fall werden Serialisierung, Deserialisierung, einzelne CFR-Dekodierung, reale lokale ZMQ-Roundtrips, Cache-Hits und frische Cache-Misses gemessen. Logischer Koeffizienten-/Frequenzspeicher, glibc-Live-Heap und Linux-RSS werden getrennt ausgewiesen. Die vorab exportierten Abnahmekriterien verlangen exakte Daten-/Byteerhaltung, erwartete Request- und Hit-/Misszahlen sowie den inklusiven Gültigkeitsrand und die anschließende Cacheexpiration. Laufzeit- und Allokatordeltas werden als Verteilungen berichtet.

Die CPU-Referenz liegt in `data/phase5_2_cache_ipc/`. Sie enthält 15 kompakte Protobuf-Vorlagen, vollständige Messkonfigurationen und SHA-256-Nachweise, Zusammenfassungen, sämtliche Zeitstichproben und Requesttranskripte. Drei Vergleichsplots bleiben lokal und unversioniert.

Abschnitt 5.22 definiert Messvertrag und Formeln; Abschnitt 7.16 berichtet
Ergebnisse und automatisierte Validierung. Die Produktionsdateien von
Cache, Helper, Protobuf und Server werden nicht verändert.

  UNTERPHASE 5.2.4: GETRENNTE TX-/RX-ABSTÄNDE UND AZIMUTE (IMPLEMENTIERT)

Der bestehende Runner `benchmark_rt_convergence.py --independent-spacing`
misst fünf vorab festgelegte Konfigurationen mit Seeds 42/43/44 und zwei
frischen CPU-Traces. Alle Fälle verwenden 100000 Samples; die beiden
Fälle mit neuen Azimuten werden zusätzlich mit 1000000 Samples geprüft.
Die bestehenden nativen Optionen erhalten unabhängige TX-/RX-Overrides,
und die Geometrieprüfung liest den Abstand des jeweiligen Endpunkts.
Produktionshelper, Server, Protobuf und Linkmodelle bleiben unverändert.
Vorhandene Tests werden erweitert; zusätzliche Testsuites, Fixturedateien
oder Messergebnisse werden nicht eingecheckt. Abschnitt 5.23 beschreibt
den Messvertrag, Abschnitt 7.17 die Ergebnisse.

  UNTERPHASE 5.2.5: CSI-ALTERUNGSSENSITIVITÄT (IMPLEMENTIERT)

`benchmark_rt_convergence.py --csi-aging` verwendet die bestehenden Fälle
03/06 mit Seeds 42/43/44, 100000 Samples und gepaarten 1000000-Samplechecks.
Für vier vorab definierte Alter werden frisches und verzögertes,
unquantisiertes Gleichleistungs-SVD verglichen. Die explizite analytische
TX-Basisrotation entwickelt die initiale RT-CFR bei fester Geometrie;
Zeitstempel allein verändern den Kanal nicht. Die vorhandenen
MimoCsiAgingModel-/SVD-/MMSE-APIs übernehmen Auswahl und Linkrechnung,
perfekte RX-CSI gilt für den aktuellen Sendekanal. Abschnitt 5.24 beschreibt
Formeln und Messvertrag, Abschnitt 7.18 Ergebnisse und Grenzen. Vorhandene
Tools/Tests werden erweitert; neue Datensätze und Fixtures bleiben lokal.

  WEITERE PHASE-5-ARBEITSPAKETE

Der folgende Restplan basiert auf dem Implementierungsstand `2425c7c` und
dem Vergleich vom 1. Oktober 2026. R1 bis R6 bezeichnen offene Arbeitspakete,
keine neuen abgeschlossenen Unterphasen. Der ausführliche Vergleich mit dem
ursprünglichen Plan und seine Prüfnachweise stehen im
[Phase-5-Implementierungsplan](doc/phase-5-implementation-plan.md).

Die korrigierte HE-DL-MU-Rezeption, Laufzeit-/Helperintegration, Uplinkkern,
HE-TB-/Triggeradapter und Kohärenzzeitumrechnung sind bereits implementiert
und historisch auf CPU validiert; sie werden nicht erneut als offene
Phase-4-Voraussetzungen eingeplant. Die bestehenden Grenzen bei perfekter
RX-CSI, Synchronisation, Nutzerstreams und automatischer Ablaufsteuerung
bleiben Bestandteil des jeweiligen Messvertrags.

Die abgeschlossenen Phase-5-Studien werden innerhalb ihrer belegten Grenzen
weiterverwendet. Die Kalibrierung gilt ausschließlich für HE MCS 0/BCC/32
Byte/1x1. Der selektive Validierungsfehler 0,054886 erfüllt die implementierte
Grenze 0,10, aber nicht die ursprünglich vorgeschlagene Grenze 0,05. Eine
strengere oder breitere Freigabe benötigt neue, vorab festgelegte Kriterien
und unabhängige Testdaten. Die 2x2-Studien liefern MMSE-/SVD-/MI-Diagnostik;
die analytische CSI-Rotation ist keine physikalische Dopplerreferenz.
Der Cache-/IPC-Benchmark liefert eine eingefrorene lokale Prozessreferenz,
keine Live-RT- oder Langzeit-Speicherfreigabe.

Die bestehenden Werkzeuge `calibrate_eesm.py`, `benchmark_sumimo.py`,
`benchmark_rt_convergence.py` und `benchmark_cache_ipc.py` werden erweitert.
Gemeinsame kleine Hilfsfunktionen sollen doppelte Manifest-/Prozesslogik
reduzieren; ein zweites Benchmarkframework ist nicht erforderlich.

  R1: NATIVE REPRODUZIERBARKEIT (ABGESCHLOSSEN)

R1: native Reproduzierbarkeit (abgeschlossen). Der feste Umfang umfasst Seed/Run/Random-Streams, vollständige portable Studienbundles und leichte Lock-/Systemprüfungen im bestehenden Conda-CPU-Environment. Abnahme: ein verschobenes Bundle verifiziert und reproduziert einen kleinen RT-/Native-Fall; Seedwiederholung und die etablierten CPU-Regressionen bestehen. Container/Docker, Resume, neue Studien und GPU-Abnahme gehören nicht dazu; R2 bleibt geschlossen.

`SionnaHelper::AssignStreams(s)` reserviert RT-ID s und Mobilitäts-ID $s+1$ (Default 0/1, Rückgabewert 2). Andere ns-3-Zufallsobjekte benötigen disjunkte IDs. `SimInitMessage.randomness` überträgt Mappingversion 1, tatsächlichen ns-3-Seed, Run und beide IDs. Für Domäne q (`rt` oder `mobility`) gilt

$$
\begin{aligned}
&D_q=\operatorname{SHA256}(\operatorname{ASCII}(\\
&\texttt{ns3sionna-rng-v1:seed:run:stream:q})),\qquad\\
&s_q=\operatorname{uint32be}(D_q[0:4]).
\end{aligned}
$$

Alle Zahlen sind dezimal, ohne Leerzeichen/Endzeile. RT bei Run 1/Stream 0 verwendet direkt den ns-3-Seed: 42/43/44 bleiben erhalten. Beide PathSolver-Pfade verwenden s_{rt}; jeder Mobilitätsknoten verwendet `SeedSequence([s_mobility,node_id,0x51A0])`. Ziehungen anderer Knoten/RT verändern diesen Generator nicht. Das emuliert keine ns-3-Zufallsfolge und beweist keine statistische Unabhängigkeit. Wiederholte gleiche Tupel sind gepaarte Wiederholungen. Ohne neue Felder bleibt der Legacyvertrag (Version 0) erhalten; unbekannte Versionen/ungültige Tupel scheitern. Sidecars erfassen Tupel und effektive Seeds.

`experiment_bundle.py` kopiert Konfigurationen, notwendige Szenen-/Mesh-/Material- und Laufinputs sowie Ergebnisse/Logs unverändert. Der externe [Artefaktkatalog](experiments/r1/catalog.json) hält den vertrauenswürdigen Indexhash und relative lokale Zugriffspfade; ein gehosteter Download wird nicht behauptet. Verifikation prüft das vollständige Inventar, relative Pfade, Referenzabschluss und Rohbinär-/LF-Texthashes. Historische Messzeitpunkt-Quellhashes bleiben erhalten; fehlende Ausführungsprovenienz bleibt unbekannt. Reproduktion schreibt neue Ausführungs-/Validierungsnachweise außerhalb des unveränderten Bundles. Native Metriken werden mit rtol $1O^{-4}$ und atol $1O^{-7}$ verglichen; Rang, Tonindex und Frequenz sind exakt. Laufzeit und maschinenübergreifende Bytegleichheit gehören nicht zur numerischen Abnahme. `--mode rt` rechnet frisch, `--mode frozen` analysiert den gespeicherten Kanal. 10000 Samples sind ein Reproduktionsprüfsatz, kein Konvergenznachweis.

`check_cpu_environment.py --strict` prüft Python 3.12.12, pip 26.0.1, alle 55 bestehenden Paketpins, `pip check`, Compiler/CMake/Protobuf/ZMQ/LLVM, aktuelle Bindings und tatsächliche LLVM-Ausführung. Systemversionen sind dokumentiert, nicht hermetisch gesperrt. Befehle und Format stehen im [nativen Vertrag](doc/provenance-cpu-increment.md).

  R2: VOLLSTÄNDIGER KALIBRIERPROFILVERTRAG UND ERWEITERTE PER-VALIDIERUNG

Abhängigkeit: R1-Identitäten und geeignete unabhängige Referenzdaten.

  - Die MCS-0/BCC-Pilotkalibrierung und ihre historischen Grenzen 0,05/0,10
  erhalten. Den nächsten Profilumfang anhand verfügbarer Referenzen
  auswählen. LDPC, 1458 Byte, weitere MCS, 2x2 und MU sind Erweiterungen,
  keine bereits validierten Kombinationen.
  - Die Bedeutung von `packet_size_bytes` eindeutig festlegen. MATLAB verwendet
  `APEPLength`/`getPSDULength`, der AWGN-Exporter `8*packetSizeBytes`, der
  Empfang PSDU-/Chunkgrößen. Anwendungs-, MPDU-, PSDU- und Aggregatlängen
  sowie Fehlerzählung vor einem Paketvergleich konsistent zuordnen.
  - Ein explizites typisiertes Profil für Beta, optionale AWGN-Korrektur,

PHY-Kontext, Fehlermodellklasse/-attribute, Detektor-/CSI-Annahme,
Ton-/RU-Vertrag und Profil-/Provenienzidentität ergänzen. Derzeit ist Beta
nur MCS-bezogen, während der Offset einen engeren Kontext besitzt.
Manuelle Setter bleiben als unvalidierte Experimente nutzbar.
- Das bestehende JSON-Ergebnis oder ein kleines versioniertes Laufzeitformat
außerhalb der Mathematikklassen einlesen. Fehlerhafte, abgelehnte,
synthetische, unvollständige oder inkompatible Profile und widersprüchliche
manuelle Werte vor Zustandsänderungen zurückweisen.
- Anwendbarkeit sowohl bei Helperkonfiguration als auch für jeden tatsächlichen
PPDU/Nutzer, `WifiTxVector`, Codeart, NSS, Bandbreite/RU und Paketlänge
prüfen. Fehlermodellattribute und Fallbackzweig gehören zum Profil.
Ein Lauf darf nach einem Teilfallback nicht weiterhin als vollständig
kalibriert erscheinen.
- Den skalaren 1x1-Kompatibilitätspfad getrennt prüfen: Er umgeht die
MIMO-Abstraktion. Ein Offline-/Numerikfit belegt daher nicht automatisch
dessen Anwendung bei realem 1x1-Netzwerkempfang. Den vorgesehenen
kalibrierten Empfangspfad explizit festlegen und Ende-zu-Ende validieren.
- Einen nativen Batchauswerter für gespeicherte CFRs, Precoder, Tonmasken,
Linkbudget und PHY-Kontext ergänzen, der die Produktionsrechnung
MMSE/EESM/Fehlermodell benutzt. Der vorhandene Benchmark prüft MMSE/MI;
der AWGN-Exporter prüft nur das Fehlermodell.
- Unberührte finale Testdaten von Training/Modellauswahl trennen; sämtliche
Punkte einer Kanalrealisation gehören in dieselbe Partition.
Binomialintervalle pro festem Kanal und Unsicherheit über unabhängige
Realisationen berichten. Paketanzahlen anhand eines Piloten vorab
festlegen; 1000 Pakete garantieren keine 95-Prozent-Intervallhalbbreite
von höchstens 0,02. Unzureichende Präzision ausdrücklich kennzeichnen.

Abnahmekriterium: Vollständige Profile werden explizit gewählt und beim
tatsächlichen Einsatz geprüft; Fit, native Rechnung und PHY-Empfang stimmen
für ihren belegten Bereich überein. Für neue Daten gelten als vorgeschlagene
Ziele selektive PER-MAE höchstens 0,05 und SNR-Verschiebung höchstens 1 dB
bei PER 0,1/0,5, sofern die Kurven diese Übergänge einschließen. Abweichende
Kriterien müssen vor Einsicht in neue finale Testdaten begründet werden.
Das vorhandene Ergebnis 0,054886 wird nicht nachträglich als Bestehen von
0,05 ausgegeben; Nullfehlerpunkte werden als obere Schranke behandelt.

R3: CACHEKORREKTHEIT UND BEGRENZTE SPEICHERUNG (ABGESCHLOSSEN)

Die bestehenden Abnahmen umfassen neue ungecachte Antworten, inklusive
Gültigkeitsränder, atomare Batchvalidierung, inaktive Linkbereinigung,
Belegungs-/Kapazitätszähler, Dimensions-/Koeffizienten-/Antwort-/Snapshotbudgets
und unabhängige Getterwerte (7.19–7.22). Die lokalen Zeit-/Link-/Tiefensweeps
bleiben als getrennte historische Messungen erhalten.

Die verbleibende Arbeit wird als eine vollständige Unterphase abgeschlossen:
begrenzte knotenbezogene Mobilitätshistorien mit Arraywiederverwendung,
historiesparende Look-ahead-Checkpoints mit Wiederherstellung bei Vorhersagefehlern
und ein begrenzter nativer Pool für leere Batch-Staging-Slots. Feste Abnahme:
unveränderte deterministische Trajektorien und Look-ahead-Werte, endliche
Retention bei Langläufen, tatsächliche Allokationswiederverwendung, unabhängige
Getterwerte, atomare Datenvalidierung bei Fehlern und sequenzielle CPU-Regressionen.
R1/R2 bleiben geschlossen. Live-RT/PHY-Skalierung, allgemeine CFR-/Protobuf-
Allokatorersetzung und GPU-Nachweise gehören nicht dazu. Methoden stehen in 5.26,
die Abschlussabnahme in 7.24. Logische Bytes sind keine harte RSS-Schranke.

R4: LIVE-SERVER-/PHY-SKALIERUNG UND GEMESSENE OPTIMIERUNGEN

Abhängigkeit: R3 für belastbare Cachevergleiche; 5.2.3 liefert die Basis.

- Bestehende Matrizen und Requestprüfungen für echte Sionna-Serverberechnung

in P2P/P2MP/Look-ahead und unterstützte PHY-/Verkehrsfälle erweitern.
Den abgebrochenen `performance-sionna-mimo.sh` reparieren, Seeds/Run
weiterreichen und getrennte Ausgabeverzeichnisse verwenden.
- PathSolver, CIR/CFR, NumPy-Konvertierung/Normalisierung, Protobufaufbau,
Serialisierung, Transport, C++-Parsing/Validierung, Cacheübernahme und
Linkrechnung getrennt messen. Ereignisse über Anfrageidentität verbinden;
keine Differenz unsynchronisierter Prozessuhren als Laufzeit ausgeben.
- Import-/JIT-Kosten von warmen Messungen trennen. RSS/Spitzen-RSS und
backendbezogenen GPU-Speicher messen; Torch-Allokatorwerte erfassen nicht
den gesamten Mitsuba-/Dr.Jit-Bedarf. Validierte Live-RT-Optionen für Samples,
Tiefe und Solverflags ergänzen; bisher setzt der Runner sie direkt im
Prozess.
- Transportstress um ausgewählte 4096-Ton-/Eckfälle und zeitliche Batches
innerhalb eines vorab geprüften Budgets ergänzen. Synthetische Tensorgrößen
bleiben von unterstützten HE-Ressourcengittern getrennt.
- MU-Gruppenrechnungen pro Empfänger/PPDU zählen und erst dann reduzieren.
Wiederverwendung erfordert vollständige CSI-/Gruppen-/RU-/Precoder-/
Rausch-/TX-Vector-Identität und korrekten empfängerbezogenen Zustand.
- Baseline auf einem benannten Host mit vielen Anfragestichproben und
unabhängigen Prozessen messen. Median/p95 und Schwankungen zwischen
Läufen berichten; Leistungsgrenzen vor Kandidatenvergleich festlegen.
Eine höchstens zehnprozentige Regression auf unbetroffenen Fällen ist
ein vorgeschlagenes Prüfziel, keine bestehende Garantie.

Abnahmekriterium: Live-CPU und später GPU besitzen belegte Grenzen für
Speicherverstärkung, Latenz und simulierte Sekunden pro Wandzeitsekunde.
Ressourcenüberschreitungen werden mitberichtet; Optimierungen besitzen
gemessenen Nutzen oder eine belegte Korrektheits-/Speichergrenzenwirkung.

R5: PHYSIKALISCHE CSI-ALTERUNG UND BREITERE SENSITIVITÄT

Abhängigkeit: R1; kalibrierte PER-Ausgaben zusätzlich R2.

- Vorhandene Abstands-/Winkel-/Konvergenzmatrizen wiederverwenden.
Gezielte 0,75-Lambda-, Elevations-/Rollfälle und eine weitere
Multipath-/NLOS-Szene mit konkreter Untersuchungsfrage ergänzen.
Gesamtleistung, Rauschen und Solveroptionen kontrollieren sowie
tatsächliche Geometrie und Apertur ausgeben.
- Einen physikalischen Bewegungs-/Dopplervertrag mit getrennten Sounding-
und Sendekanälen festlegen. Frisches und verzögertes CSI gegen denselben
tatsächlichen Sendekanal auswerten; Updateintervall, Feedbackdauer und
Zeitstempel von der Propagationcache-Gültigkeit unterscheiden.
- Die bestehende analytische Rotation als Kontrolle erhalten. Ihre
Energie-/Singularwertinvarianz prüft Numerik, nicht Doppler.
Gleichleistungs-SVD unterstützt Alterung; MRT benötigt aktuelles
perfektes CSI, Wasserfüllung schließt Alterung/Quantisierung aus.
- Unsicherheit über Seeds/Geometrien ergänzen. Die finale Anzahl unabhängiger
Realisationen anhand eines Piloten bestimmen; ein Start mit zehn gepaarten
Seeds ist ein Planungswert. Deterministische Wiederholungen zählen nicht
als unabhängige statistische Stichproben.
- Historische Toleranzen erhalten: 0,5 dB Verlust/effektive SINR, 1 dB
Ton-SINR, die Rang-/Singularwertkriterien und 0,2 bit/s/Hz MI-Effizienz.
Neue Szenen oder strengere Ziele benötigen vorab festgelegte Kriterien.
Nach R2 kalibrierte Nutzdaten-PER und beobachtete PHY-Ergebnisse ergänzen;
MI, vorhergesagte PER, Präambelfehler, Payloadfehler, Retries und Goodput
getrennt berichten.

Abnahmekriterium: Physikalische Alterung besitzt reproduzierbare Bewegung,
CSI-Zeiten, unabhängige Kontrollen, Unsicherheit und erklärten Gültigkeitsbereich.
Eine 1x1-Kalibrierung wird nicht ungeprüft auf 2x2 oder MU übertragen.

R6: CUDA-CI, BACKENDVERGLEICH UND PHASE-5-ABSCHLUSS

GPU-Zugang früh konfigurieren; finale Evidenz folgt den benötigten R1-R5-Paketen.

- Einen unterstützten GPU-Runner mit exakten Abhängigkeiten, GPU-/Treiberdaten und tatsächlicher CUDA-Mitsuba-Variante einrichten. Torch-Gerätewahl und RT-Backend unabhängig prüfen.
- Backendinterne Wiederholung und CPU-/GPU-Vergleich für analytische Kontrollen sowie repräsentative RT-/Serverfälle automatisieren. Physikalische Invarianten und vorab definierte numerische/statistische Äquivalenzgrenzen verwenden. Fehlender GPU-Zugang ist ein offener Nachweis, kein erfolgreicher CPU-Fallback eines GPU-Jobs.
- PR-Prüfungen auf native, Vertrags-, Profil-, Policy- und eingefrorene Daten beschränken; vollständige RT-/Referenz-/Live-Skalierungs-/Langläufe bei relevanten Änderungen oder Freigabekandidaten ausführen.
- Unabhängige Referenzdeckung zuerst für das gewählte SU-Profil, danach für festes/ZF/RZF-DL-MU und ideales synchronisiertes UL-MU erweitern. Automatisches Sounding, Triggersteuerung und Synchronisationsfehler sind getrennte Erweiterungen der bestehenden Phase-4-Annahmen.
- Ein Reproduktionspaket für eine saubere Maschine mit Manifesten, Befehlen, gesperrten Versionen, Artefaktindex, Profilen, Rohdaten und Genauigkeits-/Ressourcenberichten veröffentlichen. README, Modell und PDF nach tatsächlichen Abnahmen konsistent aktualisieren.

Abnahmekriterium: Für beide gewählten Backends liegen Nachweise derselben Freigaberevision vor. Alle kalibrierten Aussagen gehören zu bestandenen Profilen; offene Hardware-, Daten- oder Featuregrenzen bleiben benannt.

REIHENFOLGE, AUFWAND UND NÄCHSTES INKREMENT

Paket	Hauptabhängigkeit	Geschätzte Entwicklertage
R1: Provenienz und CPU-CI	vorhandene Exporte	3-5
R2: Profile und eine Erweiterung	R1-Identitäten, unabhängige Referenz	5-9
R3: Cachekorrektheit und Speicherung	bestehender Cachebenchmark	3-5
R4: Live-Skalierung und Optimierung	R3 für Cachevergleiche	3-6
R5: Physikalische Alterung/Sensitivität	R1, für kalibrierte PER zusätzlich R2	4-7
R6: CUDA und Reproduktionsabschluss	GPU-Zugang, benötigte R1-R5-Pakete	3-5

Der geschätzte Restaufwand beträgt 21-37 Entwicklertage. Referenzbeschaffung/Lizenzen, unbeaufsichtigte Experiment-/Runnerwartezeit und vollständige Kalibrierung sämtlicher MCS-/FEC-/Längen-/SU-/MU-Kombinationen sind nicht enthalten. Die Schätzung ersetzt den früheren Gesamtphasenansatz 27-47 Tage; bereits geleistete Arbeit wird damit nicht rückwirkend bewertet.

Erstes R1/R3-Inkrement (implementiert): portable Provenienz mit additivem versioniertem Laufmanifest, erste CPU-CI sowie Regression und Korrektur des deaktivierten Caches. Vertrag, Schemamigration und verbleibende Grenzen stehen in [der Inkrementdokumentation](doc/provenance-cpu-increment.md), Prüfergebnisse in Abschnitt 7.19. R1 ist nun durch die native Abnahme in 7.23 abgeschlossen. Der oben referenzierte ausführliche Phase-5-Plan war im aktuellen Checkout und im durchsuchten Arbeitsbereich nicht vorhanden; maßgeblich war der hier enthaltene Restplan. Weitere R3-/R4-/R5-/R6-Arbeit bleibt außerhalb der R1-Abnahme. R2 wird durch R1 nicht erweitert. Korrektheit, Provenienz, Kalibrierung und Optimierung bleiben getrennt prüfbare Änderungen.

Die Prüfung vom 1. Oktober bestätigte acht Kalibrier-Unit-Tests sowie alle 135 referenzierten binären Konfigurations-/Kanal-/Templatehashes der drei versionierten Studien. Native, RT-, GPU- und MATLAB-Läufe wurden dabei nicht erneut ausgeführt. Die alten Abnahmen in Abschnitt 7 bleiben historische Nachweise; die lokal gehaltenen Rohdaten von 5.2.4/5.2.5 waren in diesem

Checkout nicht vorhanden. Der Restplan ist daher kein neuer wissenschaftlicher Abschlussnachweis.

4. DATENMODELL UND IPC-VERTRAG DER UNTERPHASEN 1.1/1.2

4.1 TENSOR UND FLACHE REIHENFOLGE

Für jeden Subcarrier k ist

$$
\mathbf{H}[k] \in \mathbb{C}^{N_\mathrm{rx}\times N_\mathrm{tx}}.
$$

Der C++-Container und die beiden Protocol-Buffer-Felder csi_real und csi_imag speichern die Werte flach. Der Index für den Eintrag h[k,r,t] lautet

$$
i(k,r,t) = (k N_\mathrm{rx} + r)N_\mathrm{tx} + t.
$$

Damit ist k die langsamste und t die schnellste Dimension. Es gelten die verbindlichen Längen

$$
|\mathrm{frequencies}| = N_\mathrm{sc},
\qquad
|\mathrm{csi_real}| = |\mathrm{csi_imag}|
= N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}.
$$

Die neuen skalaren Felder num_rx_antennas und num_tx_antennas wurden mit den Feldnummern 9 und 10 an RxNodeInfo angehängt. Der Proto3-Standardwert null wird beim Empfänger als eins interpretiert. Für eine neue 1x1-Antwort lässt der Python-Server beide Felder bewusst ungesetzt; ihre serialisierte Darstellung ist dadurch bytegleich zur alten Nachricht. Alte Sender werden als 1x1 gelesen, alte Empfänger ignorieren die unbekannten neuen Felder einer MIMO-Antwort.

4.2 TRANSFORMATION AUS SIONNA

Sionna RT 2.0.1 liefert die CFR weiterhin in der Form

$$
H_\mathrm{raw}
[\ell,r,q,t,n,k]
$$

mit Linkindex ℓ, RX-Antenne r, TX-Index q, TX-Antenne t, Zeitindex n und Subcarrier k. Eine ns3sionna-Anfrage enthält genau einen Sender und einen Zeitpunkt. Deshalb wird ohne squeeze-Operation explizit ausgewählt:

$$
H_\ell[r,t,k] =
H_\mathrm{raw}[\ell,r,0,t,0,k].
$$

Vor der Serialisierung wird nach [k,r,t] transponiert und anschließend in C-Reihenfolge abgeflacht. Dadurch bleiben auch Singleton-Dimensionen sichtbar und 2x2-Werte landen indexgenau im C++-Container.

4.3 WIDEBAND-VERLUST UND MIMO-NORMALISIERUNG

Für die rohe Link-CFR wird die mittlere Leistung gemeinsam über Subcarrier und Antennenpaare gebildet:

$$
\bar P =
\frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
\sum_{k=0}^{N_\mathrm{sc}-1}
\sum_{r=0}^{N_\mathrm{rx}-1}
\sum_{t=0}^{N_\mathrm{tx}-1}
|H_\ell[r,t,k]|^2.
$$

Der übertragene skalare Wideband-Verlust und die normalisierte Matrix lauten

$$
L_\mathrm{wb} = -10\log_{10}(\bar P)\ \mathrm{dB},
\qquad
H_\mathrm{norm}[r,t,k] =
\frac{H_\ell[r,t,k]}{\sqrt{\bar P}}.
$$

Damit gilt

$$
\frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
\sum_{k,r,t}|H_\mathrm{norm}[r,t,k]|^2 = 1.
$$

Für $N_{\mathrm{rx}} = N_{\mathrm{tx}} = 1$ ist dies exakt die bisherige SISO-Normalisierung. Die Aufteilung
der gesamten Sendeleistung auf N_s Streams bleibt davon getrennt und erfolgt weiterhin im numerischen Modell mit dem Faktor $1/N_s$.

4.4 ARRAYKONFIGURATION UND NS-3-CACHE

SimInitMessage besitzt optionale tx_array- und rx_array-Nachrichten mit Zeilen, Spalten, vertikalem und horizontalem Abstand in Wellenlängen, Elementmuster und Polarisation. Ohne Nachricht werden exakt die alten Arrays verwendet: 1x1/tr38901/V am Sender und 1x1/dipole/V am Empfänger. SionnaHelper bietet dafür die kleinen Methoden ConfigureTransmitArray und ConfigureReceiveArray; die umfassende knotenbezogene High-Level-API bleibt Phase 2 vorbehalten.

Die Antwortdimensionen werden aus der tatsächlich von Sionna gelieferten Form abgeleitet und berücksichtigen damit auch die Polarisation. CacheEntry hält die CFR nun als MimoChannelMatrix. GetPropagationChannelMatrix erhält die Dimensionen; GetPropagationCSI bleibt als flache, für 1x1 unveränderte Kompatibilitätsansicht bestehen.

4.5 MEHRNUTZER-SNAPSHOT UND GERICHTETE IDENTITÄT DER UNTERPHASE 4.1

Die bestehende gebündelte Antwort bleibt der Transportcontainer. Eine `ChannelState`-Nachricht bezeichnet genau einen CSI-Zeitpunkt und enthält weiterhin einen gemeinsamen Sender sowie mehrere `RxNodeInfo`-Nachrichten. Feld 11 von `RxNodeInfo` ergänzt additiv die Nachricht `UserChannelIdentity` mit

$$
I_u=(u, n_{\mathrm{tx}}, n_{\mathrm{rx}}, d, t_{\mathrm{csi}}, A_{\mathrm{rx},u}).
$$

Hier sind u die stabile Nutzer-ID, n_{tx} und

n_{rx} die gerichteten Endpunkte, d die explizite Richtung
und $A_{\mathrm{rx},u}$ die vollständige Empfangsarraygeometrie. Unterphase
4.1 erzeugt ausschließlich `CHANNEL_DIRECTION_DOWNLINK`; Uplink wird damit
nicht vorweggenommen. Fehlt `UserChannelIdentity`, bleibt die vorhandene
SU-/Legacy-Deserialisierung unverändert.

Für einen gültigen Snapshot gelten gleichzeitig

$$
  t_{\mathrm{csi},u}=t_{\mathrm{start}},\qquad
  f_{u,k}=f_k,\qquad
  N_{\mathrm{sc},u}=N_{\mathrm{sc}},\qquad
  N_{\mathrm{tx},u}=N_{\mathrm{tx}}.
$$

Die Empfangsdimension darf dagegen nutzerabhängig sein. Für ein planares
Array mit R_u Zeilen, C_u Spalten und $P_u \in \{1,2\}$
Polarisationsports gilt verbindlich

$$
  N_{\mathrm{rx},u}=R_u\,C_u\,P_u.
$$

Jeder Nutzer trägt sein eigenes $A_{\mathrm{rx},u}$; die bisherige globale
RX-Konfiguration wird nicht mehr stillschweigend aus einem anderen Link
abgeleitet. Der aktuelle Server schreibt für jeden von ihm erzeugten
Mehrnutzer-Link die tatsächlich konfigurierte globale Geometrie explizit aus;
Einzelnutzerantworten bleiben identitätsfrei. Der IPC- und
C++-Vertrag kann bereits unterschiedliche Geometrien und
$N_{\mathrm{rx},u}$ innerhalb desselben synthetischen Snapshots abbilden;
eine knotenbezogene High-Level-Konfiguration des Sionna-Raytracings gehört
nicht zu dieser Unterphase.

Die CFR jedes Nutzers verwendet unverändert die Reihenfolge
$i_u(k,r,t) = (kN_{\mathrm{rx},u} + r)N_{\mathrm{tx}} + t$, einen eigenen skalaren
Wideband-Pfadverlust und die Einheitsleistungsnormalisierung

$$
  \frac{1}{N_{\mathrm{sc}}N_{\mathrm{rx},u}N_{\mathrm{tx}}}
  \sum_{k,r,t}|H_u[k,r,t]|^2=1.
$$

`validate_multi_user_snapshot` prüft denselben Vertrag auf der Python-Seite.
`DeserializeMultiUserChannelSnapshot` prüft in C++ zunächst sämtliche Nutzer,
ohne Cachezustand zu verändern, und baut erst danach
`MultiUserChannelSnapshot` auf. Dessen Nutzervektor ist unabhängig von der
Eingabereihenfolge streng aufsteigend nach u sortiert. Gemischte oder
fehlende Identitäten, doppelte Nutzer, falsche Endpunkte/Richtungen,
Zeitstempel, Frequenzraster, TX-Dimensionen, nicht passende RX-Geometrien,
nicht endliche Werte und abweichende Normalisierung werden abgelehnt.

Auch Antworten mit mehreren Look-ahead-Zeitpunkten werden vollständig in
temporäre Einträge dekodiert, bevor der bestehende Cache einen davon
übernimmt. Explizit identifizierte Einträge dürfen nur in ihrer gespeicherten
TX-RX-Richtung getroffen werden. Identitätsfreie alte SU-Einträge behalten
dagegen die bisherige reziproke Schlüsselverwendung. Damit bleibt der
SU-Datenpfad erhalten, während Mehrnutzerzustände weder partiell noch in
umgekehrter Richtung sichtbar werden.

5. NUMERISCHES SU- UND DOWNLINK-MU-MIMO-MODELL

5.1 SIGNALMODELL

Ohne Precoding gilt je Subcarrier

$$
\mathbf{y}[k] =
\sqrt{\frac{P}{N_s}}\,\mathbf{H}[k]\mathbf{x}[k] + \mathbf{n}[k],
$$

mit $N_s = N_{\mathrm{tx}}$, $E[xx^H] = I$ und räumlich weißem Rauschen $E[nn^H] = NI$. Die Eingangsgröße

$$
\rho = \frac{P}{N}
$$

ist die gesamte Referenz-SNR vor der Small-Scale-CFR-Gewichtung. Das Modell verlangt in dieser Unterphase $N_{\mathrm{rx}} \geq N_{\mathrm{tx}}$. Overloaded MIMO wird erst mit einer expliziten Streamauswahl zugelassen.

5.2 LINEARER MMSE-EMPFÄNGER

Mit $\alpha = \rho / N_s$ und $G[k] = H[k]^H H[k]$ folgt die Fehlerkovarianz des linearen MMSE-Detektors:

$$
\mathbf{E}[k] =
(\mathbf{I} + \alpha\mathbf{G}[k])^{-1}.
$$

Die Post-Processing-SINR des Streams s ist

$$
\gamma_s[k] = \frac{1}{E_{s,s}[k]} - 1.
$$

Die Formel berücksichtigt gewünschte Leistung, Inter-Stream-Interferenz und Rauschverstärkung gemeinsam. Sie ist auch für rangdefiziente H numerisch definiert, weil I + alpha G positiv definit ist.

5.3 SISO-GRENZFALL

Für $N_{\mathrm{tx}} = N_{\mathrm{rx}} = 1$ ergibt sich

$$
E = \frac{1}{1 + \rho |h|^2},
\qquad
\gamma = \rho |h|^2.
$$

Damit ist der neue Kern im skalaren Grenzfall exakt kompatibel mit dem üblichen SISO-SNR-Modell.

5.4 FREQUENZKOMPRESSION

Vor einer MCS-spezifischen Kalibrierung wird je Stream eine Mutual-Information-erhaltende AWGN-SNR ausgewiesen:

$$
\gamma_{\mathrm{eff},s} =
$$

$$
  \exp\left(
    \frac{1}{N_\mathrm{sc}}\sum_k \ln(1+\gamma_s[k])
  \right)-1.
$$

Die mittlere ideale spektrale Effizienz über alle Streams ist

$$
  \eta =
  \frac{1}{N_\mathrm{sc}}
  \sum_k\sum_s \log_2(1+\gamma_s[k]).
$$

Diese MI-Größe bleibt eine Validierungsmetrik und ist keine unmittelbare
802.11-Datenrate oder PER. Die HE-PER-Abbildung verwendet stattdessen die
MCS-spezifische EESM-Kompression aus Abschnitt 5.6.

5.5 REFERENZ-SNR AUS DEM LINK BUDGET

Die skalare Wideband-Dämpfung L_wb bleibt von der normalisierten
Small-Scale-CFR getrennt. Aus der gesamten Sendeleistung $P_\mathrm{tx,dBm}$
folgen Empfangsleistung und lineare Empfangsleistung

$$
  P_\mathrm{rx,dBm} = P_\mathrm{tx,dBm} - L_\mathrm{wb},
  \qquad
  P_\mathrm{rx} =
  10^{(P_\mathrm{rx,dBm}-30)/10}\ \mathrm{W}.
$$

Mit Rauschbandbreite B in Hz, Temperatur T in Kelvin und
Empfängerrauschzahl NF in dB gelten

$$
  N_\mathrm{th} = kTB,
  \qquad
  F = 10^{NF/10},
  \qquad
  N_\mathrm{rx} = FkTB,
$$

und damit

$$
  \rho =
  \frac{P_\mathrm{rx}}{N_\mathrm{rx}}
  =
  \frac{10^{(P_\mathrm{tx,dBm}-L_\mathrm{wb}-30)/10}}
    {kTB\,10^{NF/10}}.
$$

WifiLinkBudget gibt Empfangsleistung, thermisches Rauschen,
Empfängerrauschen und ρ separat aus. dB-/Linear- und dBm-/Watt-
Umrechnungen delegieren an DbToRatio, RatioToDb und DbmToW aus
ns-3.40 wifi-utils. Für $k=1{,}3803\cdot10^{-23}\ \mathrm{J/K}$ wird bewusst
derselbe Zahlenwert wie in ns-3.40 InterferenceHelper verwendet. Das ist die
Provenienz dieser physikalischen Konstante; es ist keine Fit-Konstante.
Anders als InterferenceHelper ist T hier ein expliziter positiver
Eingabeparameter und nicht fest auf 290 K gesetzt.

P_tx ist die Gesamtleistung über alle Spatial Streams; die

Aufteilung durch N_s erfolgt erst im MMSE-Modell. B ist die Empfänger-Rauschbandbreite. Link Budget und MMSE addieren selbst keine externe Interferenz; der SpectrumWifiPhy-Pfad bewertet sie anschließend skalar gemäß Abschnitt 5.8. Ein zusätzlicher pauschaler RX-Diversity-Gewinn wird nicht aufgeschlagen: Array- und Detektorgewinn sind bereits in $\mathbf H[k]$ und den Post-MMSE-SINRs enthalten.

5.6 MCS-ABHÄNGIGE EESM-KOMPRESSION

Für HE MCS m komprimiert EESM alle $N_\mathrm{sc}N_s$ Post-MMSE-SINRs mit gleicher Gewichtung:

$$
\gamma_{\mathrm{EESM},m}
=
-\beta_m \ln\left(
  \frac{1}{N_\mathrm{sc}N_s}
  \sum_{k=0}^{N_\mathrm{sc}-1}
  \sum_{s=0}^{N_s-1}
  \exp\left(-\frac{\gamma_s[k]}{\beta_m}\right)
\right).
$$

$\beta_m>0$ hat wie γ lineare SINR-Einheiten und ist MCS-abhängig. SetEesmBeta erwartet den linearen Wert; SetEesmBetaDb ist die dB-Komfortfunktion. Zur Vermeidung numerischen Unterlaufs wird mit $\gamma_\min=\min_{k,s}\gamma_s[k]$ äquivalent gerechnet:

$$
\gamma_{\mathrm{EESM},m}
=
\gamma_\min
-\beta_m \ln\left(
  \frac{1}{N_\mathrm{sc}N_s}
  \sum_{k,s}
  \exp\left(-\frac{\gamma_s[k]-\gamma_\min}{\beta_m}\right)
\right).
$$

Für einen flachen Kanal mit identischen SINRs ist das Ergebnis unabhängig von β_m exakt diese SINR. Das ermöglicht die SISO-AWGN-Paritätsprüfung.

Es werden bewusst keine Default-Betas ausgeliefert. Jeder HE MCS von 0 bis 11 muss vor der Nutzung explizit konfiguriert werden. Damit enthält der Code keine unbelegten Kalibrierkonstanten. Die in analytischen Tests verwendeten positiven Werte sind ausschließlich Formel-Sentinels und ausdrücklich keine 802.11ax-Kalibrierung. Unterphase 5.1 versioniert erstmals einen Link-Level-Fit für HE MCS 0/BCC/32 Byte/1x1 einschließlich einer getrennten AWGN-SNR-Korrektur. Andere MCS und PHY-Konfigurationen benötigen eigene Referenzen. Die bisherige MI-Kompression je Stream bleibt als modellunabhängige Diagnosemetrik erhalten und wird nicht als kalibrierte PER-Abbildung ausgegeben.

5.7 ABBILDUNG AUF NS-3-PER

HeSuMimoLinkAbstraction übernimmt WifiTxVector einschließlich HE-MCS, NSS und Codeart. NSS muss der Zahl der übertragenen Streams entsprechen. Nach EESM wird die vorhandene Schnittstelle verwendet:

$$
\gamma_{\mathrm{model}}=
$$

$$
  \gamma_{\mathrm{EESM},m}10^{\delta_c/10},\qquad
  P_\mathrm{succ} =
  \mathrm{ErrorRateModel::GetChunkSuccessRate}
  (m,\mathrm{TxVector},\gamma_{\mathrm{model}},8L,N_\mathrm{rx}),
  \qquad
  PER = 1-P_\mathrm{succ},
$$

wobei L die Paketlänge in Byte ist. Ohne eine exakt passende Phase-5.1-Kalibrierung gilt $\delta_c=0$. ErrorRateModel ist als ns-3-Attribut und über SetErrorRateModel konfigurierbar. Standard ist TableBasedErrorRateModel, also das ns-3.40-Defaultmodell für 802.11n/ac/ax.

Die Kalibrierprovenienz bleibt vollständig bei ns-3: Laut src/wifi/doc/source/wifi-design.rst stammen die AWGN-PER-Tabellen aus End-to-End-Linksimulationen mit MATLAB WLAN Toolbox, wurden gegen IEEE-TGn-Ergebnisse validiert und verwenden die Paketlängenextrapolation aus IEEE P802.11 TGax 11-14/0803r1. BCC besitzt Referenztabellen für 32 und 1458 Byte mit einer 400-Byte-Schwelle und MCS 0 bis 9; MCS 10/11 fällt auf YansErrorRateModel zurück. LDPC besitzt eine 1458-Byte-Referenz für MCS 0 bis 11. Die Abstraktion kopiert oder verändert keine dieser Kurven. Die Phase-5.1-Korrektur verschiebt ausschließlich die ErrorRateModel-Eingabe für eine exakt passende PHY-Konfiguration. Da sie Erfolgswahrscheinlichkeiten und keine Zufallsentscheidung ausgibt, ist die Berechnung bei identischen Eingaben deterministisch.

5.8 EINKOPPLUNG IN SPECTRUMWIFIPHY

Die Integration bleibt auf SionnaSpectrumPropagationLossModel::DoCalcRxPowerSpectralDensity beschränkt. Für einen HE-SU-PPDU liest sie N_s ausschließlich mit WifiTxVector::GetNss. Vor der Link-Abstraktion müssen die physikalischen TX-/RX-Antennenzahlen gleich den CFR-Dimensionen, die maximal unterstützten Spatial Streams mindestens N_s und $N_\mathrm{ant,rx}\geq N_s$ sein. Ohne konfiguriertes Precoding bleibt zusätzlich unverändert $N_s=N_\mathrm{tx}$. Mit festem Precoding muss N_s dagegen exakt der Spaltenzahl von W[k] entsprechen. Inkonsistenzen brechen damit vor MMSE, EESM und Paketempfang ab. Der neue kontrollierte Fall $N_s<N_\mathrm{tx}$ ist ausschließlich mit explizitem festem W[k] zulässig.

Der SpectrumChannel hat den skalaren Wideband-Pfadverlust bereits auf params->psd angewendet. Die MIMO-Integration verwendet daher

$$
  P_\mathrm{in} =
  \int S_\mathrm{params}(f)\,\mathrm df \; G_\mathrm{rx}
$$

als bereits pfadverlustbehaftete Empfangsleistung. Der Wideband-Verlust aus dem Cache wird an dieser Stelle nur noch für den bestehenden Diagnose-Tag gelesen und nicht erneut multipliziert. Empfangsverstärkung, Rauschbandbreite, 290 K und Empfängerrauschzahl gehen genau einmal in das Link Budget ein. Weil ns-3.40 die Rauschzahl nicht lesbar als WifiPhy-Attribut bereitstellt, verlangt SetReceiverNoiseFigureDb einen expliziten Wert; er muss mit SpectrumWifiPhy::RxNoiseFigure identisch konfiguriert werden. Ein fehlender Wert bricht den MIMO-Pfad früh ab.

MMSE und EESM liefern γ_eff. SpectrumWifiPhy erwartet dagegen weiterhin eine skalare PSD und wendet seine InterferenceHelper-Berechnung später selbst an. Für das AWGN-Fehlermodell ist dessen interner Diversity-Faktor

$$
  G_\mathrm{AWGN} =
  \begin{cases}
    N_\mathrm{ant,rx}/N_s, & N_\mathrm{ant,rx}>N_s,\\
    1, & \text{sonst}.
  \end{cases}
$$

Da der MMSE-Schritt den Arraygewinn bereits enthält, wird die äquivalente skalare Empfangsleistung auf

$$
  P_\mathrm{eq} =
  \frac{\gamma_\mathrm{eff} P_N}{G_\mathrm{AWGN}},
  \qquad P_N = kTB\,10^{NF/10}
$$

gesetzt. InterferenceHelper erhält damit bei zusätzlicher skalarer Interferenz I

$$
  \gamma_\mathrm{phy}
  = \frac{P_\mathrm{eq}}{P_N+I}G_\mathrm{AWGN}
  = \gamma_\mathrm{eff}\frac{P_N}{P_N+I}.
$$

Ohne Interferenz gilt exakt $\gamma_\mathrm{phy}=\gamma_\mathrm{eff}$; Pfadverlust, thermisches Rauschen, Rauschzahl und Arraygewinn werden nicht doppelt angewendet. Zeitvariable Interferenz wirkt weiterhin über SpectrumWifiPhy, jedoch als skalare Nachbewertung und nicht als subcarrierabhängige Interferenzkovarianz im MMSE-Detektor.

Der bisherige 1x1-Pfad bleibt unverändert skalar und benötigt kein EESM-Beta. Nicht-HE-Kontrollrahmen einer MIMO-PHY verwenden während Phase 1 das erste Antennenpaar, damit Beacon, Association und ACK ohne vorweggenommene Streamauswahl funktionieren. Der letzte Guard-Bin und das bestehende CFR-Tag-Format bleiben im 1x1-Fall kompatibel. Für automatisierte Beispiele kann eine normalisierte feste CFR den Sionna-Cache ersetzen; dieser deterministische Test-Hook ändert den Produktionspfad nicht. Der MimoRx-Trace meldet PPDU-UID, Knoten, Effective-SINR, PER und den kleinsten Kanalrang über alle Subcarrier.

Das Beispiel ns3_wifi_sumimo_minimal.cc erzeugt drei flache, normierte Kanäle bei HE MCS 5 und 20 dB Referenz-SNR: 1x1 mit $h=1$, gut konditioniertes 2x2 mit $\mathrm{diag}(\sqrt{2},\sqrt{2})$ und Rang-1-2x2 mit ausschließlich Einsen. Zusätzlich führt es den gut konditionierten Kanal mit explizitem Identitäts-Precoding und mit einem 2x1-Precoder zur Auswahl der ersten TX-Antenne aus. Es misst realen UDP-Durchsatz über SpectrumWifiPhy und gibt zusätzlich PER, Stream-SINR, EESM-SINR und Mindestkanalrang aus. Der Aufruf muss ein positives --eesmBetaLinear liefern. Der im Regressionstest verwendete Wert 2 ist nur ein expliziter Formel-Sentinel; bei diesen flachen Kanälen ist das Ergebnis betaunabhängig und stellt keine Produktionskalibrierung dar.

Die deterministische Referenzausgabe lautet:

- 1x1: Rang 1, Stream-/Effective-SINR 20,000 dB, PER 0,000016 und 68,1408 Mbit/s;
- gut konditioniertes 2x2: Rang 2, beide Stream-SINRs und Effective-SINR 20,000 dB, PER 0,000016 und 134,3904 Mbit/s;

  - Rang-1-2x2: Rang 1, beide Stream-SINRs und Effective-SINR -0,043214 dB,
  PER 1 und 0 Mbit/s;
  - Identitäts-Precoding 2x2: dieselben analytischen SINR-, EESM- und
  PER-Ergebnisse wie der offene Vollrangpfad und 133,4784 Mbit/s im
  sequenziellen Netzwerkregressionslauf;
  - 2x1-Streamauswahl auf dem 2x2-Kanal: Rang 1, Stream-/Effective-SINR
  23,010300 dB, PER 0 und 68,1504 Mbit/s. Der 3,0103-dB-Unterschied zum
  offenen Einzelstreamvergleich folgt aus der vollen Gesamtleistung für
  \(N_s=1\), nicht aus zusätzlicher Precoderleistung.

  5.9 FESTES PRECODING DER UNTERPHASE 3.1

MimoPrecodingMatrix ist ein unveränderlicher, dimensionssicherer Werttyp für

\[
  \mathbf W[k]\in\mathbb C^{N_\mathrm{tx}\times N_s},
  \qquad
  1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx}).
\]

Seine flache Reihenfolge ist verbindlich subcarrier/tx/stream:

\[
  j(k,t,s)=(kN_\mathrm{tx}+t)N_s+s.
\]

Der frequenzselektive Konstruktor erwartet deshalb genau
\(N_\mathrm{sc}N_\mathrm{tx}N_s\) komplexe Koeffizienten. FrequencyFlat
erwartet genau \(N_\mathrm{tx}N_s\) Werte in tx/stream-Reihenfolge und
expandiert sie beim Erzeugen des Werttyps explizit auf alle \(N_\mathrm{sc}\)
Subcarrier. Intern existiert damit kein implizites Broadcasting und jeder
Zugriff besitzt dieselben drei Dimensionen. Alle Real- und Imaginärteile
müssen endlich sein.

Für jeden Subcarrier gilt verpflichtend

\[
  \Vert\mathbf W[k]\Vert_F^2
  =\sum_{t=0}^{N_\mathrm{tx}-1}\sum_{s=0}^{N_s-1}|w_{t,s}[k]|^2
  =N_s.
\]

Die absolute Prüftoleranz beträgt \(10^{-10}\). Eine verletzte Norm wird
abgelehnt und nicht stillschweigend korrigiert. Mit
\(\mathrm E[\mathbf x\mathbf x^H]=\mathbf I\) lautet das erweiterte
Signalmodell

\[
  \mathbf y[k]=
  \sqrt{\frac{P}{N_s}}\,\mathbf H[k]\mathbf W[k]\mathbf x[k]
  +\mathbf n[k].
\]

Damit bleibt \(P\) die gesamte Sendeleistung und

\[
  \mathbf H_\mathrm{eff}[k]=\mathbf H[k]\mathbf W[k]
  \in\mathbb C^{N_\mathrm{rx}\times N_s}.
\]

Der bestehende MMSE-Kern wird ohne neuen Detektortyp auf

$\mathbf H_\mathrm{eff}$ angewendet. Mit
$\alpha=\rho/N_s$ gilt weiterhin

$$
  \mathbf E[k]=
  \left(\mathbf I_{N_s}+\alpha
  \mathbf H_\mathrm{eff}^H[k]\mathbf H_\mathrm{eff}[k]\right)^{-1},
  \qquad
  \gamma_s[k]=\frac{1}{E_{s,s}[k]}-1.
$$

SuMimoLinkModel, HeSuMimoLinkAbstraction und der Spectrumadapter besitzen dafür explizite Precoding-Pfade. Ihre bisherigen Überladungen ohne W[k] bleiben bestehen und rechnen unverändert direkt mit H[k]. Insbesondere nimmt der bytegleiche 1x1-IPC-Vertrag kein neues Feld auf: Sionna liefert weiterhin ausschließlich H[k], W[k] existiert nur in ns-3.

SionnaMimoConfiguration enthält optional precodingMatrix. Validate prüft vor jeder Helper-Zustandsänderung die TX-Dimension, die Gleichheit von maxNss und W-Spalten sowie $N_s\leq N_\mathrm{rx}$. Apply ersetzt den Adapterzustand entweder durch den vollständig validierten Precoder oder löscht ihn explizit, sodass erneutes Anwenden einer Phase-2-Konfiguration kein altes W[k] zurücklässt. Am Laufzeitpfad werden zusätzlich CFR-Subcarrierzahl und beide CFR-Antennendimensionen geprüft, bevor ErrorRateModel- oder Empfangszustand verändert wird. WifiTxVector::GetNss muss bei vorhandenem W[k] exakt dessen Spaltenzahl, andernfalls weiterhin exakt N_tx sein.

Unterphase 3.1 implementiert benutzergegebenes festes Precoding, Unterphase 3.2 ergänzt die CFR-abhängige MISO-MRT-Fabrik und Unterphase 3.3 die deterministische allgemeine SVD-Fabrik mit gleicher oder wassergefüllter räumlicher Leistung. Alle erzeugen unveränderliche W[k]-Schnappschüsse, die anschließend denselben H[k]W[k]-Pfad durchlaufen. CSI-Rückmeldung, CSI-Alterung, automatische Laufzeit-Beamformingmodi und MU-MIMO bleiben außerhalb dieses Vertrags.

5.10 EIN-STREAM-MRT/MRC DER UNTERPHASE 3.2

Für den eindeutig definierten MISO-Fall $N_\mathrm{rx}=1$ sei die CFR je Subcarrier der Zeilenvektor

$$
  \mathbf h[k]\in\mathbb C^{1\times N_\mathrm{tx}}.
$$

Bei $\Vert\mathbf h[k]\Vert_2>0$ erzeugt MimoPrecodingMatrix::MaximumRatioTransmission aus perfekter CFR genau einen Precodervektor

$$
  \mathbf w_\mathrm{MRT}[k]
  =\frac{\mathbf h^H[k]}{\Vert\mathbf h[k]\Vert_2},
  \qquad
  \Vert\mathbf w_\mathrm{MRT}[k]\Vert_2^2=1=N_s.
$$

Damit erfüllt das Ergebnis direkt den Leistungsvertrag aus Abschnitt 5.9 und der effektive Kanal ist

$$
  \mathbf h[k]\mathbf w_\mathrm{MRT}[k]
  =\Vert\mathbf h[k]\Vert_2.
$$

\]

Der unveränderte Ein-Stream-MMSE-Kern liefert folglich

\[
  E[k]=\frac{1}{1+\rho\Vert\mathbf h[k]\Vert_2^2},
  \qquad
  \gamma[k]=\rho\Vert\mathbf h[k]\Vert_2^2.
\]

Die Normierung erfolgt unabhängig für jeden Subcarrier; das zurückgegebene
W[k] besitzt deshalb dieselbe sc/tx/stream-Reihenfolge wie jeder feste
Precoder. Eine gemeinsame Phasendrehung
\(\mathbf h'[k]=e^{j\phi}\mathbf h[k]\) führt zu
\(\mathbf w'[k]=e^{-j\phi}\mathbf w[k]\) und lässt den effektiven Kanal sowie
die SINR invariant. Besitzt auch nur ein Subcarrier Norm null, wird der gesamte
Erzeugungsaufruf eindeutig mit std::invalid_argument abgelehnt. Es wird weder
eine beliebige Strahlrichtung gewählt noch Energie erzeugt. Nicht endliche
CFR-Koeffizienten werden ebenfalls abgelehnt.

Für \(N_\mathrm{tx}=1\) ist
\(\mathbf H[k]=\mathbf h[k]\in\mathbb C^{N_\mathrm{rx}\times1}\). Dann ist die
Richtung des linearen MMSE-Kombinierers kollinear zu \(\mathbf h[k]\) und die
bestehende Lösung reduziert sich ohne neuen Empfängertyp auf MRC:

\[
  \gamma[k]=\rho\,\mathbf h^H[k]\mathbf h[k]
  =\rho\Vert\mathbf h[k]\Vert_2^2.
\]

Im 1x1-Fall entfernt MRT nur die Kanalphase; der Betrag und damit das
bisherige skalare Ergebnis bleiben exakt erhalten. Für
\(N_\mathrm{tx}>1\) und \(N_\mathrm{rx}>1\) weist die MRT-Funktion den Aufruf
bewusst ab. Eine optimale Ein-Stream-Richtung in diesem allgemeinen Fall ist
der stärkste rechte Singulärvektor und gehört ausschließlich in Unterphase
3.3.

Der ideale numerische Aufruf lautet:

```
    const auto mrt =
    MimoPrecodingMatrix::MaximumRatioTransmission(channelAtCsiTime);
    const auto result = SuMimoLinkModel::Calculate(channelAtCsiTime, mrt, rho);
```

Das Ergebnis ist ein unveränderlicher W[k]-Schnappschuss für die übergebene
perfekte CFR. Unterphase 3.2 führt absichtlich keinen automatischen
Helper-/SpectrumWifiPhy-Beamformingmodus, keine CSI-Zeitachse und keine
periodische Aktualisierung ein. Der erzeugte Precoder kann wie jeder feste
2x1-Precoder durch die bereits validierten Pfade aus Unterphase 3.1 geführt
werden; die darauf aufbauende end-to-end Modusauswahl ist nun in Abschnitt
6.7 als Teil der abgeschlossenen Unterphase 3.5 beschrieben.

5.11 DETERMINISTISCHES SVD-PRECODING UND LEISTUNGSALLOKATION DER UNTERPHASE 3.3

Für jeden Subcarrier berechnet
MimoPrecodingMatrix::SingularValueDecomposition aus der explizit übergebenen
perfekten CFR eine dünne Jacobi-SVD

\[
  \mathbf H[k]=\mathbf U[k]\boldsymbol\Sigma[k]\mathbf V^H[k],
  \qquad

$$
  \sigma_0[k]\geq\sigma_1[k]\geq\ldots\geq0.
$$

Die Streamzahl wird nicht aus dem Kanal erraten, sondern als N_s explizit übergeben und muss

$$
  1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})
$$

erfüllen. Der numerische Rang je Subcarrier ist

$$
  r[k]=\#\left\{i:\sigma_i[k]>
  \tau_\mathrm{rank}\sigma_0[k]\right\}.
$$

Der dokumentierte Standard ist $\tau_\mathrm{rank}=10^{-12}$; ein benutzergegebener Wert muss endlich und strikt in (0,1) liegen. Gilt auf auch nur einem Subcarrier $r[k]<N_s$, wird der gesamte Fabrikaufruf mit Subcarrierindex, geforderter Streamzahl und gefundenem Rang abgelehnt. Es gibt weder eine versteckte Regularisierung noch eine stille Verringerung von N_s. Nicht endliche CFR-Koeffizienten werden vor der Zerlegung zurückgewiesen.

Die von Eigen absteigend gelieferten Singularwerte erhalten zusätzlich eine explizite deterministische Vektorkonvention. Zwei Werte derselben Zerlegung gelten bei

$$
  |\sigma_i-\sigma_j|\leq10^{-12}\sigma_0
$$

als numerisch gleich. Für jede solche entartete Gruppe wird nicht die vom SVD-Algorithmus beliebig gewählte Basis übernommen. Stattdessen bildet ns-3 den eindeutigen Unterraumprojektor $\mathbf P=\mathbf V_g\mathbf V_g^H$, projiziert die kanonischen TX-Koordinatenachsen und orthonormalisiert sie deterministisch. Je Schritt wird die Achse mit der größten Residualnorm gewählt; numerisch gleiche Kandidaten verwenden den kleinsten TX-Index. Abschließend wird jeder Singulärvektor so phasengedreht, dass sein betragsgrößter Eintrag reell und nicht negativ ist; bei gleichem Betrag gewinnt ebenfalls der kleinste TX-Index. Damit sind Reihenfolge, Vorzeichen beziehungsweise komplexe Phase und die Basis bei gleichen Singularwerten vollständig festgelegt.

  GLEICHE LEISTUNG ALS STANDARD

Im Standardmodus MimoPowerAllocationMode::EQUAL_POWER gilt für alle gewählten Moden

$$
  f_s[k]=\frac{1}{N_s},
  \qquad
  \mathbf W[k]=[\mathbf v_0[k],\ldots,\mathbf v_{N_s-1}[k]].
$$

Damit besitzt jede Spalte Norm eins und der Vertrag aus Abschnitt 5.9 bleibt unverändert:

$$
  \Vert\mathbf W[k]\Vert_F^2=N_s.
$$

\]

Die SVD-Moden sind orthogonal. Der unveränderte MMSE-Kern liefert deshalb
ohne Inter-Stream-Interferenz

\[
  \gamma_s[k]=\frac{\rho}{N_s}\sigma_s^2[k].
\]

Der 1x1-Grenzfall verwendet den einzigen rechten Singulärvektor mit
Einheitsnorm und reproduziert das bisherige skalare Ergebnis exakt. Die
Optionen verwenden für die normierten Leistungsdiagnosen standardmäßig
\(\rho=1\) und \(N=1\); die gleichmäßige Richtungsauswahl ist davon
unabhängig.

  OPTIONALE RÄUMLICHE WASSERFÜLLUNG

Wasserfüllung wird nur durch die explizite Auswahl
MimoPowerAllocationMode::WATER_FILLING aktiviert. Sie verteilt die Leistung
ausschließlich über die \(N_s\) bereits gewählten räumlichen SVD-Moden eines
Subcarriers. Zwischen Subcarriern wird keine Leistung verschoben; spektrale
Maske und Gesamtleistung je Subcarrier bleiben daher unverändert. Mit
Rauschleistung \(N>0\), Referenz-SNR \(\rho>0\), Gesamtleistung
\(P=\rho N\) und Modengewinn \(g_s[k]=\sigma_s^2[k]\) lauten die
Leistungsanteile

\[
  f_s[k]=\left(
    \nu[k]-\frac{1}{\rho g_s[k]}
  \right)^+,
  \qquad
  \sum_{s=0}^{N_s-1}f_s[k]=1.
\]

Der aktive Modensatz und der normierte Wasserstand \(\nu[k]\) werden
deterministisch aus den absteigend sortierten Moden bestimmt. Eine Mode am
exakten Aktivierungsschwellwert bleibt inaktiv. Der Precoder skaliert danach

\[
  \mathbf w_s[k]=\sqrt{N_s f_s[k]}\,\mathbf v_s[k].
\]

Somit gilt auch bei inaktiven Nullspalten exakt

\[
  \sum_s\Vert\mathbf w_s[k]\Vert_2^2
  =N_s\sum_s f_s[k]=N_s.
\]

Die physikalisch ausgewiesene Modenleistung und der Wasserstand sind

\[
  p_s[k]=P f_s[k],
  \qquad
  \mu[k]=P\nu[k].
\]

Für jede aktive Mode erfüllen sie die KKT-Bedingung

\[
  p_s[k]+\frac{N}{g_s[k]}=\mu[k],

\]

während inaktive Moden keinen negativen Leistungsanteil erhalten. Eine abschließende Rundungskorrektur wird ausschließlich auf den letzten aktiven Anteil angewendet, sodass die Anteile wieder exakt zur normierten Gesamtleistung summieren. Alle Gewinne, Anteile und Wasserstände werden auf Endlichkeit und Nichtnegativität geprüft.

MimoSvdPrecodingResult hält den festen Precoder sowie pro Subcarrier numerischen Rang, gewählte Singularwerte, aktive Modenzahl, Leistungsanteile, absolute Modenleistungen, deren Summe und den Wasserstand. Zusätzlich werden Rauschleistung, Referenz-SNR, Gesamtleistung und Rangtoleranz unverändert ausgewiesen. Ein idealer Aufruf mit Wasserfüllung lautet:

```
    MimoSvdPrecodingOptions options;
    options.powerAllocation = MimoPowerAllocationMode::WATER_FILLING;
    options.referenceSnrLinear = rho;
    options.noisePower = noisePower;

    const auto svd = MimoPrecodingMatrix::SingularValueDecomposition(
    channelAtCsiTime, numStreams, options);
    const auto result = SuMimoLinkModel::Calculate(
    channelAtCsiTime, svd.GetPrecodingMatrix(), options.referenceSnrLinear);
```

Wie MRT erzeugt auch SVD ausschließlich einen unveränderlichen perfekten-CSI-W[k]-Schnappschuss. Unterphase 3.3 aktiviert weder SU_BEAMFORMING im Helper noch automatische Aktualisierung, Feedback, CSI-Zeitbezug oder einen neuen SpectrumWifiPhy-Pfad. Der vorhandene feste-W[k]-Vertrag ist die einzige Integration; Unterphase 3.5 verwendet genau diesen Vertrag nun für die benutzerseitige end-to-end Modusauswahl, ohne die SVD- oder Leistungsallokationssemantik zu verändern.

5.12 QUANTISIERTE HE-/EHT-SU-RÜCKMELDUNG UND CSI-ALTERUNG DER UNTERPHASE 3.4

UNTERSTÜTZTER FEEDBACKUMFANG UND ABGRENZUNG

MimoCompressedBeamformingFeedback bildet die semantische komprimierte Beamformingmatrix $\mathbf V[k]$ aus der HE-/EHT-SU-Rückmeldung ab. IEEE 802.11ax beschreibt diese Matrix als geordnete Folge von ϕ-Phasenwinkeln und reellen ψ-Givens-Winkeln. EHT übernimmt dieselbe komprimierte Darstellung mit erweiterten Grenzen für N_c und N_r; dies ist auch in der öffentlichen [IEEE-802.11be-Beamforming-Korrespondenz](https://www.ieee802.org/11/email/stds-802-11-tgbe/msg01987.html) festgehalten. Die öffentlich gelisteten [HE-Comment-Resolution-Dokumente](https://www.ieee802.org/11/Reports/tgax_update.htm) verweisen auf die HE Compressed Beamforming Report- und SU-MIMO-Beamforming-Klauseln. Die Implementierung unterstützt daraus bewusst folgenden schmalen Vertrag:

- ausschließlich Single-User-Feedback für eine semi-unitäre Gleichleistungs-SVD-Matrix mit $1\leq N_s\leq N_\mathrm{tx}$;
- HE mit höchstens acht und EHT mit höchstens 16 TX-Antennen;
- das grobe SU-Codebook mit $b_\phi=4,\ b_\psi=2$ sowie das feine SU-Codebook mit $b_\phi=6,\ b_\psi=4$;
- eine Matrix je modelliertem Subcarrier und einen unquantisierten Forschungsgrenzfall.

Der Typ ist absichtlich kein serialisierter IEEE-802.11-Action-Frame. MIMO-Control-, Average-SNR-/CQI-Felder, MAC-Header, Segmentierung, Subcarriergruppierung N_g, Interpolation, RU-/Partial-Bandwidth-Auswahl, MU-Exclusive-Feedback und die feineren MU-Codebooks sind nicht implementiert.

Wassergefülltes W[k] wird ebenfalls nicht als $\mathbf V[k]$ ausgegeben, weil dessen Spalten nicht semi-unitär sind; die Richtungsrückmeldung und eine separate Leistungsallokation werden nicht fälschlich vermischt. Damit wird weder MU-MIMO noch die Laufzeitintegration aus Unterphase 3.5 vorweggenommen.

  GIVENS-DARSTELLUNG UND VERBINDLICHE WINKELREIHENFOLGE

Sei $m=\min(N_s,N_\mathrm{tx}-1)$. Für jede Stufe $i=0,\ldots,m-1$ enthält $\mathbf D_i$ auf den Diagonaleinträgen $i,\ldots,N_\mathrm{tx}-2$ die Phasen $\exp(j\phi_{r,i})$; der letzte Eintrag und die Einträge vor i sind eins. $\mathbf G_{l,i}^{T}(\psi_{l,i})$ ist die reelle Rotation, deren (i,l)-Block

$$
  \begin{bmatrix}
    \cos\psi_{l,i} & -\sin\psi_{l,i}\\
    \sin\psi_{l,i} & \cos\psi_{l,i}
  \end{bmatrix}
$$

lautet. Die Rekonstruktion verwendet exakt

$$
  \widehat{\mathbf V}[k]
  =
  \left[
  \prod_{i=0}^{m-1}
    \mathbf D_i
    \left(
    \mathbf G_{N_\mathrm{tx}-1,i}^{T}
    \cdots
    \mathbf G_{i+1,i}^{T}
    \right)
  \right]
  \mathbf I_{N_\mathrm{tx}\times N_s}.
$$

Die flache Reihenfolge je Subcarrier ist für jede Stufe zuerst

$$
  \phi_{i,i},\ldots,\phi_{N_\mathrm{tx}-2,i}
$$

und danach

$$
  \psi_{i+1,i},\ldots,\psi_{N_\mathrm{tx}-1,i}.
$$

Die getrennten ganzzahligen ϕ- und ψ-Arrays sind innerhalb ihres Typs subcarrier-major. Die Zahl der Winkel jedes Typs lautet

$$
  A =
  \sum_{i=0}^{m-1}(N_\mathrm{tx}-i-1).
$$

Encode entfernt die für die Linkmetrik irrelevante Phase jeder SVD-Spalte deterministisch, extrahiert danach die idealen Winkel stufenweise und quantisiert nur diese Winkel. FromQuantizedAngles überspringt die Extraktion

und erzeugt aus bekannten Integerindizes eine bitgenaue Referenzdarstellung. Reconstruct ist davon getrennt und verwendet ausschließlich die gespeicherten Indizes. Die idealen SVD-, Feedback- und Rekonstruktionsobjekte bleiben damit einzeln prüfbar.

DETERMINISTISCHE QUANTISIERUNG UND GRENZWERTE

Für $\phi\in[0,2\pi)$ und 2^{b_ϕ} Zellen gelten

$$
q_\phi =
\left\lfloor
  \frac{2^{b_\phi}\phi}{2\pi}
\right\rfloor,
\qquad
\widehat\phi =
\frac{2\pi(q_\phi+\tfrac12)}{2^{b_\phi}}.
$$

Vor der Quantisierung wird ϕ modulo 2π normalisiert; der obere Rand 2π fällt damit eindeutig auf Index null. Für $\psi\in[0,\pi/2]$ gilt

$$
q_\psi =
\min\left(
  2^{b_\psi}-1,
  \left\lfloor
  \frac{2^{b_\psi+1}\psi}{\pi}
  \right\rfloor
\right),
\qquad
\widehat\psi =
\frac{\pi(q_\psi+\tfrac12)}{2^{b_\psi+1}}.
$$

Damit rekonstruieren auch die minimalen und maximalen Indizes stets die Zellmitte und nie einen mehrdeutigen periodischen Rand. Nicht passende Indexanzahlen und Werte außerhalb der gewählten Bitbreite werden vor der Rekonstruktion mit std::invalid_argument abgelehnt. Der unquantisierte Modus speichert das bereits validierte feste W[k] direkt, rekonstruiert es koeffizientenexakt und weist null Feedbackbits aus; null bedeutet hier „kein endlicher Drahtumfang modelliert“ und nicht ein kostenloses Standardfeedback.

Der reine Winkelumfang ohne PHY-/MAC-Header und ohne CQI ist

$$
B_\mathrm{fb}
=
N_\mathrm{sc} A (b_\phi+b_\psi).
$$

Für eine 2x1-Matrix sind dies je Subcarrier sechs Bit im groben und zehn Bit im feinen SU-Codebook. Eine 3x2-Matrix besitzt je Typ drei Winkel und benötigt im feinen Profil 30 Bit je Subcarrier.

LEISTUNGSRENORMIERUNG UND DIAGNOSEGRÖSSEN

Obwohl das Produkt idealer Rotationen semi-unitär ist, wird jede quantisiert rekonstruierte Matrix vor dem bestehenden festen-W[k]-Konstruktor explizit

erneut normalisiert. Mit

$$
  P_\mathrm{vor}[k]
  =
  \Vert\widehat{\mathbf V}[k]\Vert_F^2,
  \qquad
  a[k]=\sqrt{\frac{N_s}{P_\mathrm{vor}[k]}},
  \qquad
  \mathbf W_q[k]=a[k]\widehat{\mathbf V}[k]
$$

gilt anschließend wieder

$$
  \Vert\mathbf W_q[k]\Vert_F^2=N_s.
$$

MimoFeedbackReconstruction weist $P_\mathrm{vor}[k]$, $a[k]$ und die endgültige Leistung getrennt aus. Nullleistung, nicht endliche Werte oder ein Ergebnis außerhalb des festen-W[k]-Vertrags werden abgelehnt.

MimoFeedbackDiagnostics meldet neben B_fb zwei phaseninvariante Fehler je Subcarrier. Für normierte ideale und rekonstruierte Spalten sind dies der mittlere Spalten-Chordalfehler

$$
  e_\mathrm{ch}[k]
  =
  \sqrt{
    \frac1{N_s}\sum_s
    \left(
    1-
    \frac{
    |\mathbf w_s^H[k]\mathbf w_{q,s}[k]|^2
    }{
    \Vert\mathbf w_s[k]\Vert_2^2
    \Vert\mathbf w_{q,s}[k]\Vert_2^2
    }
    \right)
  }
$$

und der RMS-Koeffizientenfehler nach optimaler gemeinsamer Phasenausrichtung jeder Spalte. Zusätzlich stehen die Mittelwerte über alle Subcarrier bereit. Damit kann der Fehler ohne willkürliche SVD-Spaltenphase verglichen werden. Auf den Referenzvektoren ist das feine Codebook nicht schlechter als das grobe.

  EXPLIZITE CSI-ZEITACHSE

MimoCsiAgingModel::Select erhält ausschließlich die zur Auswahl verwendete Schätzung

$$
  \widehat{\mathbf H}[k,t_\mathrm{csi}]
$$

sowie t_csi, t_tx und eine MimoCsiSelectionConfiguration. Mit Aktualisierungsintervall $T_u\geq0$ und Feedbackverzögerung $d_\mathrm{fb}\geq0$ ist der neueste zum Sendezeitpunkt

verfügbare Schätzzeitpunkt verbindlich

$$
  t_\mathrm{csi}
  =
  \begin{cases}
    t_\mathrm{tx}-d_\mathrm{fb}, & T_u=0,\\
    T_u\left\lfloor
    \dfrac{t_\mathrm{tx}-d_\mathrm{fb}}{T_u}
    \right\rfloor, & T_u>0.
  \end{cases}
$$

Ein Sendezeitpunkt vor d_fb besitzt noch kein verfügbares Feedback. Ein explizit übergebener Zeitstempel, der nicht der Gleichung entspricht, wird abgelehnt. Das ausgewiesene CSI-Alter ist

$$
  \Delta t_\mathrm{CSI}=t_\mathrm{tx}-t_\mathrm{csi}.
$$

$T_u=0$ und $d_\mathrm{fb}=0$ bilden den perfekten Nullalter-Grenzfall. Select berechnet aus $\widehat{\mathbf H}[k,t_\mathrm{csi}]$ zuerst das deterministische Gleichleistungs-SVD-W[k], danach Feedback und Rekonstruktion. CalculateLink verlangt dagegen die davon getrennt übergebene aktuelle CFR

$$
  \mathbf H[k,t_\mathrm{tx}]
$$

und berechnet ausschließlich $\mathbf H[k,t_\mathrm{tx}]\mathbf W_q[k]$. Der CSI-Schnappschuss wird in diesem Schritt nicht gespeichert oder wiederverwendet. Derselbe feste Precoder kann an HeSuMimoLinkAbstraction übergeben werden; dadurch verwenden auch EESM und PER explizit die aktuelle CFR. Eine kontrollierte orthogonale Subraumrotation liefert im Test mit altem W null SINR und eine entsprechend schlechtere PER, während ein statischer Kanal trotz positiven Alters unverändert bleibt.

MimoCsiStateIdentity ist der vollständige wertbasierte Schlüssel des Auswahlzustands. Er enthält gerichtete TX- und RX-Node-IDs, beide Arraygeometrien einschließlich Abstand, Muster, Polarisation und Orientierung, den Precodingmodus, HE-/EHT-Feedbackprofil und t_csi. Vertauschte Richtung, eine geänderte Arraykonfiguration, ein anderes Profil oder ein anderer CSI-Zeitstempel sind damit verschiedene Zustände. Diese Identität ist von der bestehenden rohen Sionna-PropagationCache-Ablage getrennt und verhindert, dass ein späterer Laufzeitcache versehentlich Precoderzustände verschiedener Richtungen oder Konfigurationen teilt.

Der ideale Aufruf ohne Quantisierung lautet:

```
    MimoCsiSelectionConfiguration configuration;
    configuration.numStreams = numStreams;
    configuration.feedback.codebook =
    MimoFeedbackCodebook::UNQUANTIZED;
    configuration.timing.updateInterval = MilliSeconds(5);
    configuration.timing.feedbackDelay = MilliSeconds(2);

    const auto selection = MimoCsiAgingModel::Select(
    channelEstimateAtCsiTime,
    csiTimestamp,
```

```
    transmissionTimestamp,
    configuration);
    const auto currentLink = selection.CalculateLink(
    channelAtTransmissionTime,
    referenceSnrLinear);
```

Der Vertrag bleibt bewusst keine automatische Simulationstaktung. Unterphase 3.5 bindet ihn nun durch einen expliziten SelectPrecoding-Aufruf an SionnaMimoHelper und SpectrumWifiPhy: Der Anwender übergibt CSI- und Sendezeitpunkt, der Helper hält das unveränderliche Ergebnis und Apply nutzt anschließend den vorhandenen festen-W[k]-Pfad. Periodische Ereignisse oder MAC-Feedbackframes werden dadurch nicht vorgetäuscht.

5.13 FESTES DOWNLINK-MU-PRECODING UND INTERFERENZMODELL DER UNTERPHASE 4.2

`DlMuMimoPrecodingMatrix` bettet den unveränderten, leistungsvalidierten `MimoPrecodingMatrix`-Tensor ein. Der Index eines Eintrags im Zuordnungsvektor ist die globale Precoder-Spalte c; der Eintrag

$$
  a_c=(u,s_u)
$$

nennt die stabile Nutzer-ID u und den nutzerlokalen Stream s_u. Die lokalen Streamindizes jedes Nutzers beginnen bei null und sind lückenlos. Jeder Nutzer des Phase-4.1-Snapshots erhält mindestens einen Stream, und kein Stream darf auf einen nicht vorhandenen Nutzer zeigen.

Die Phase-3-Normierung gilt jetzt über alle gleichzeitig gesendeten Spalten:

$$
  \Vert W[k]\Vert_F^2=N_{s,\mathrm{ges}}.
$$

Mit der expliziten Spaltenleistung

$$
  q_c[k]=\sum_t |W_{t,c}[k]|^2,\qquad
  \sum_c q_c[k]=N_{s,\mathrm{ges}}
$$

beträgt der Anteil der gesamten AP-Leistung für Spalte c genau $q_c[k]/N_{s,\mathrm{ges}}$. Gleiche Spaltennormen ergeben gleiche Leistung; abweichende Nutzer- oder Streamgewichte stehen sichtbar in $W[k]$, ohne die Gesamtleistung zu ändern.

Für Nutzer u wird zunächst der vollständige effektive Kanal gebildet:

$$
  G_u[k]=H_u[k]W[k]
    =\left[g_{u,0}[k],\ldots,g_{u,N_{s,\mathrm{ges}}-1}[k]\right].
$$

Die nutzerspezifische Referenz-SNR ρ_u enthält den getrennten Wideband-Pfadverlust und die Empfängerrauschleistung. Für jede globale Spalte gilt derselbe Symbolleistungsfaktor

$$
  \alpha_u=\frac{\rho_u}{N_{s,\mathrm{ges}}}.
$$

Der lineare Empfänger verwendet die vollständige Kovarianz aller gewünschten und fremden Spalten,

$$
  R_u[k]=I+\alpha_u G_u[k]G_u^H[k],
  \qquad
  f_{u,c}[k]=R_u^{-1}[k]g_{u,c}[k].
$$

Die Skalierung von $f_{u,c}$ ist für die SINR ohne Bedeutung. Der ausgegebene gewünschte Anteil und die Inter-User-Interferenz nach Kombination lauten

$$
  D_{u,c}[k]=\alpha_u|f_{u,c}^H[k]g_{u,c}[k]|^2,
$$

$$
  I_{u,c}^{\mathrm{MU}}[k]
    =\alpha_u\sum_{d:a_d=(v,\cdot),\,v\ne u}
    |f_{u,c}^H[k]g_{u,d}[k]|^2.
$$

Mit dem analogen Intra-User-Anteil über die anderen Streams desselben Nutzers und $N_{u,c}[k]=\|f_{u,c}[k]\|_2^2$ folgt

$$
  \gamma_{u,c}[k]
    =\frac{D_{u,c}[k]}
    {N_{u,c}[k]+I_{u,c}^{\mathrm{intra}}[k]
    +I_{u,c}^{\mathrm{MU}}[k]}.
$$

Für genau einen Nutzer ist dies algebraisch derselbe lineare MMSE-Vertrag wie in Phase 3. Das gilt auch für den 1x1-Grenzfall und für null SNR beziehungsweise einen verschwindenden effektiven Kanal.

`HeDlMuMimoLinkAbstraction` komprimiert anschließend ausschließlich die SINR-Werte des jeweiligen Nutzers:

$$
  \gamma_{\mathrm{eff},u}
   =-\beta_{m_u}\ln\left(
    \frac{1}{N_{\mathrm{sc}}N_{s,u}}
    \sum_{k,s_u}\exp\left(-\frac{\gamma_{u,s_u}[k]}{\beta_{m_u}}\right)
    \right).
$$

MCS m_u, EESM-β_{m_u}, `WifiTxVector` und Paketgröße gehören explizit zum Nutzer. Die anschließende ErrorRateModel-Abfrage ist daher getrennt und kann für zwei Nutzer unterschiedliche PER liefern.

Vor der Berechnung werden Snapshotrichtung und -ordnung, CSI-Zeitidentität, Frequenzraster, CFR-/Precoderdimensionen, endliche Koeffizienten, die vollständige Spaltenzuordnung, nutzerweise Stream-/RX-Grenzen sowie genau eine SNR- und HE-Eingabe je Nutzer geprüft. Konstruktion und Berechnung sind wertbasiert; eine Ablehnung verändert weder Snapshot noch Precoder oder EESM-Konfiguration. Der Kern konsumiert den Phase-4.1-Snapshot nur lesend und ändert weder Protocol Buffers, Batching noch Cache.

5.14 IDEALES DOWNLINK-ZF UND REGULARISIERTES ZF DER UNTERPHASE 4.3

Die wertbasierten Fabriken `DlMuMimoPrecodingMatrix::ZeroForcing` und
`RegularizedZeroForcing` konsumieren denselben vollständig validierten
Phase-4.1-Snapshot wie der feste Phase-4.2-Kern. Der absichtlich schmale
Vertrag umfasst gleichzeitige perfekte Downlink-CSI, genau eine
Empfangsantenne und genau den lokalen Stream null je Nutzer. Für U Nutzer
und N_{tx} AP-Antennen gilt $1 \le U \le N_{\mathrm{tx}}$.
Mehrere RX-Antennen oder Streams je Nutzer benötigen eine explizite
Empfängerauswahl, Blockdiagonalisierung oder einen gemeinsamen
Transceiververtrag und werden in dieser Unterphase nicht stillschweigend
angenommen.

Die Nutzer des Snapshots liegen bereits streng aufsteigend nach stabiler
Nutzer-ID vor. Die als Menge behandelten Streamanforderungen dürfen in
beliebiger Reihenfolge eingegeben werden, müssen aber exakt $(u, 0)$ einmal
für jeden Snapshotnutzer enthalten. Die ausgegebenen Spalten und
`DlMuMimoStreamMapping`-Einträge folgen immer der stabilen Snapshotreihenfolge.
Pro Subcarrier entsteht damit eindeutig

$$
  \bar H[k]=\begin{bmatrix}H_{u_0}[k]\\ \vdots\\ H_{u_{U-1}}[k]\end{bmatrix}
  \in\mathbb C^{U\times N_\mathrm{tx}}.
$$

ns-3 berechnet eine dünne Jacobi-SVD

$$
  \bar H[k]=U_H[k]\Sigma[k]V_H^H[k],
  \qquad \sigma_0[k]\geq\ldots\geq\sigma_{U-1}[k]\geq0.
$$

Wie in Phase 3 ist die dokumentierte Standardrangschwelle
$\tau_{\mathrm{rank}} = 10^{-12}$; ein expliziter Wert muss endlich und strikt
in $(0, 1)$ liegen. Der numerische Rang zählt genau die Werte
$\sigma_i[k] > \tau_{\mathrm{rank}}\sigma_0[k]$. Auf jedem Subcarrier muss er
U betragen. Es gibt weder abgeschnittene schwache Nutzermoden noch eine
stille Nutzerreduktion oder versteckte Regularisierung.

Für ZF wird auf dem vollständig akzeptierten Zeilenraum die ungekürzte
Moore-Penrose-Pseudoinverse verwendet:

$$
  \widetilde W_\mathrm{ZF}[k]
    =V_H[k]\,\mathrm{diag}(\sigma_i^{-1}[k])\,U_H^H[k].
$$

RZF besitzt keinen Defaultparameter. Der Aufrufer muss ein endliches, strikt
positives dimensionsloses α angeben; für die gemeinsam normierte CFR
gilt

$$
  \widetilde W_\mathrm{RZF}[k]
    =V_H[k]\,\mathrm{diag}\!\left(
    \frac{\sigma_i[k]}{\sigma_i^2[k]+\alpha}\right)U_H^H[k]
    =\bar H^H[k]\left(\bar H[k]\bar H^H[k]+\alpha I\right)^{-1}.
$$

Singulärwerte und zugehörige linke/rechte Vektoren bleiben als gekoppelte,
absteigend gelieferte Tripel zusammen. Die vollständige Pseudoinversen-
beziehungsweise Filtermatrix ist bei exakt gleichen Singularwerten invariant
gegen Basisrotationen und gemeinsame Spaltenphasen; nahezu gleiche, aber

verschiedene Werte behalten ihren eigenen Filterfaktor. Da keine einzelnen Moden als Nutzerstrahlen exportiert werden, gelangt keine beliebige SVD-Basiswahl in die Nutzerordnung. Wiederholte Auswahl und umgekehrte Anforderungsreihenfolge liefern im nativen Test koeffizientengleiche Ergebnisse.

Erst nach vollständiger Auswahl wird pro Subcarrier ein einziger reeller Skalierungsfaktor angewendet:

$$
W[k]=\sqrt{\frac{U}{\Vert\widetilde W[k]\Vert_F^2}}\,
  \widetilde W[k],
\qquad \Vert W[k]\Vert_F^2=U.
$$

Damit bleibt P exakt die gesamte AP-Sendeleistung aus Unterphase 4.2. Die globale Skalierung erhält die ZF-Nullstellen. RZF bleibt auf vollrangigen, schlecht konditionierten Kanälen endlich; auch RZF akzeptiert absichtlich keinen numerischen Rangverlust, weil Regularisierung in Unterphase 4.3 keine nicht planbare Nutzergruppe kaschiert.

Vor SVD oder Ergebniskonstruktion werden Richtung, Zeit, Nutzerordnung, Frequenzraster, Dimensionen, Endlichkeit, Last, Streamanforderungen, RX-Antennenzahl, Rangtoleranz und gegebenenfalls α geprüft. Jede Ablehnung ist `std::invalid_argument` und verändert keinen Zustand. Das fertige Objekt läuft unverändert durch `DlMuMimoLinkModel` sowie die bestehenden nutzerweisen MMSE-, EESM- und PER-Pfade. Protocol Buffers, Serialisierung, Batching, Cache, ZMQ, Sionna RT und SpectrumWifiPhy werden durch Unterphase 4.3 nicht geändert.

5.15 HE-DOWNLINK-MU-SPECTRUMINTEGRATION DER UNTERPHASE 4.4

`SetDlMuMimoState` übernimmt einen wertbasierten Downlink-Snapshot und dessen gemeinsamen Precoder. Vor der atomaren Installation werden numerische Dimensionen, Zeit-/Nutzerordnung und Precoder geprüft. Zusätzlich müssen ein gemeinsamer Sender, eindeutige Empfänger ohne Selbstverbindung und endliche Widebandverluste vorliegen. Diese Installationsprüfung ersetzt nicht die Array-/Normalisierungsprüfung des Protobuf-Deserialisierers; die direkte numerische API bleibt für synthetische Referenzkanäle nutzbar.

Die installierte Gruppe gilt nur für ihre gerichteten AP-/Empfängerlinks und tatsächlich adressierte PSDUs. Die MAC-Zieladresse wird eindeutig auf die STA-ID abgebildet; RU, MCS und NSS stammen aus `WifiTxVector::HeMuUserInfoMap`. Mithörer und andere Sender verwenden ihren physikalischen Cache-/Festkanalpfad und ändern den installierten MU-Zustand nicht. Auch ein Teilnehmer einer zuvor installierten Gruppe darf ein aktuell nicht an ihn adressiertes PPDU mithören. Für diese Verbindungen muss deshalb ein physikalischer Kanal konfiguriert sein.

Belegte Frequenzen. Die Laufzeit-CFR verwendet das Sender-PSD-Gitter mit 78,125 kHz Abstand, abzüglich des abschließenden zusätzlichen Guardbins. `HeRu::GetSubcarrierGroup` liefert die einzelnen belegten Teilspannen; `HePhy::ConvertHeRuSubcarriers` bildet sie auf dieses Sendergitter ab. RX-Bandindizes dürfen bei unterschiedlicher Betriebsbandbreite nicht zur Indizierung des Sendespektrums verwendet werden.
Datentöne und Piloten gehen gleich gewichtet in die Abstraktion ein; DC, Nulltöne, Guardtöne und andere RUs werden ausgeschlossen. Dies modelliert keine gesonderten Pilotschätzfehler. Für die belegte Tonmenge K_u gilt

$$
 \gamma_{\mathrm{eff},u}=-\beta_{m_u}\log\left(
 \frac{1}{|K_u|N_{s,u}}\sum_{k\in K_u}\sum_s
$$

 e^{-\gamma_{u,s}[k]/\beta_{m_u}}\right).
\]

Auch eine Vollband-RU benötigt eine explizite Tonauswahl. Die optionale Liste
`subcarrierIndices` darf nur in der direkten numerischen API leer bleiben und
bezeichnet dort wie bisher das gesamte übergebene Gitter. Im Laufzeitpfad ist
sie stets nicht leer. Doppelte oder außerhalb der CFR liegende Indizes werden
abgelehnt. Matrixdiagnostik bleibt auf dem vollständigen Gitter verfügbar;
nur die ausgewählten Töne gehen in die EESM-/PER-Reduktion ein.

Leistung je RU. Gleiche `HeRu::RuSpec`-Werte bilden eine Gruppe g. Auf
jedem Ton wird die Energie aller in dieser Gruppe aktiven Spalten berechnet:

\[
 E_g[k]=\sum_{j\in g}\sum_t |W_{t,j}[k]|^2,\qquad
 \rho_u=\frac{P_{\mathrm{RU}(u)}L_uG_{\mathrm{rx},u}}{N_u},\qquad
 \rho_{u,s}[k]=\frac{\rho_u}{E_{g(u)}[k]}.
\]

Dieser letzte Faktor wird konsistent in gewünschter Leistung, MMSE-Kovarianz
und Interferenz verwendet. Damit gilt $\sum_{j\in g} P_g\|w_j[k]\|^2/E_g[k]=P_g$, auch für ungleiche Spaltennormen
und ungleiche RU-Leistungen. Relative Gewichte innerhalb einer RU bleiben
erhalten. Beim bisherigen Ein-Gruppen-Vertrag gilt $E_g[k]=N_{s,\mathrm{ges}}$,
also die bisherige Skalierung. Gruppenenergie muss auf jedem numerisch
ausgewerteten Ton endlich und positiv sein; vollständig abgeschaltete
Gruppenspalten werden explizit abgelehnt. Andere RUs gehen nicht in die
MMSE-Kovarianz oder IUI ein.

Der skalare SpectrumChannel-Pfadverlust liegt beim Eintritt bereits in der
PSD. RU-Leistung wird über die vom PHY verwendete umschließende RU-Bandspanne
auf dem Sendergitter integriert. Der Adapter rekonstruiert die AP-RU-Leistung
und wendet pro Snapshotnutzer dessen Widebandverlust und Empfangsgewinn an.
Rauschen verwendet die nominelle ns-3-RU-Bandbreite, 290 K und die konsistente
PHY-Rauschzahl. Die skalare Nutzdatenleistung ist

\[
 P_{\mathrm{eq},u}=\gamma_{\mathrm{eff},u}N_u/G_{\mathrm{AWGN},u}.
\]

Nur die RU-Bandspanne des aktuellen Nutzers wird auf diese Leistung skaliert;
der nachgelagerte PHY-Empfangsgewinn wird vorher herausgerechnet. Außerhalb
der RU wird auf anderen belegten RUs die tatsächliche Energie
$\sum_{j\in g}|H[k,0,:]w_j[k]|^2/E_g[k]$ verwendet. Unbelegte
Spektralmaskenbins verwenden die Gleichleistungs-Antennenannahme. Eine Nutzdaten-SINR darf
nicht alle fremden Interferenzbänder skalieren.

Gemeinsame Präambel und Mithörer. Nur die adressierte
`HePpdu::PSD_HE_PORTION` verwendet die MU-Nutzdatenabstraktion. Die gemeinsame
`PSD_NON_HE_PORTION` enthält keine IUI- oder MCS-Beta-Kompression. Als
explizites physikalisches Modell gelten gleiche Leistung je TX-Antenne und
die erste RX-Verzweigung:

\[
 G_{\mathrm{common}}[k]=\frac{1}{N_{\mathrm{tx}}}\sum_t|H[k,0,t]|^2.
\]

Die vorhandene PHY-Verarbeitung behandelt anschließend Empfangsgewinn und
skalare Diversity. ns-3.40 sendet die Nicht-HE-Portion mit 312,5 kHz Abstand;
die installierte HE-CFR besitzt 78,125 kHz Abstand. Bei gleichem Zentrum und
Bandumfang wird jeder vierte CFR-Eintrag am entsprechenden Frequenzzentrum

verwendet. Gleich aufgelöste Cache-CFR bleiben ebenfalls zulässig. Der physikalische HE-MU-Pfad für Mithörer verwendet dieselbe Gleichleistungsannahme. Dies modelliert kein separates Präambel-Beamforming und keine räumliche Kovarianz fremder MU-Interferenz. Gemeinsame Portionen und Mithörer erhalten kein MU-Payload-CFR-Tag. Die HE-Portion umfasst im ns-3-Spektrumpfad auch HE-Training; eine weitere feldweise Trennung wird hier nicht eingeführt.

Inkonsistente Metadaten adressierter installierter Teilnehmer werden weiterhin abgelehnt. HE-SU und die bisherigen numerischen 1x1-/Precoderverträge bleiben kompatibel. EHT-MU und EHT-TB werden weiterhin abgelehnt. Die Gruppenrechnung wird noch pro Empfänger ausgeführt; diese quadratische Adapterarbeit bleibt eine bekannte Leistungsgrenze. Automatischer Scheduler, Helperfreigabe für MU, Uplink und wissenschaftliche EESM-Kalibrierung sind nicht Teil dieser Korrektur.

5.16 LAUFZEIT-CSI UND SCHEDULER-SCHNITTSTELLE DER UNTERPHASE 4.5

DlMuMimoRuntime bildet eine schmale, wertbasierte Brücke zwischen dem vorhandenen ns-3-MultiUserScheduler und dem Phase-4.4-Spectrumadapter. Ein Auswahlaufruf übernimmt dessen DlMuInfo einschließlich WifiTxParameters::m_txVector, HeMuUserInfoMap und WifiPsduMap. Eine explizite Menge von DlMuMimoStaUserBinding trennt weiterhin STA-ID und stabile Kanalnutzer-ID. PSDU-, HE-MU-TxVector- und Binding-Menge müssen identisch sein; jede PSDU muss existieren.

Der Laufzeitschlüssel lautet

$$
  K=\left(n_{\mathrm{AP}},d,
    \{(u,\mathrm{STA},\mathrm{RU},N_{\mathrm{SS}})\}\right).
$$

Die Nutzerfolge in K ist unabhängig von der Eingabereihenfolge streng nach stabiler Nutzer-ID sortiert. Unterphase 4.5 akzeptiert nur WIFI_PREAMBLE_HE_MU, Downlink, einen Stream und eine Empfangsantenne je Nutzer sowie mindestens zwei Nutzer pro Gruppe. Die ausgewählten Nutzer müssen dieselbe RU belegen. Der Scheduler erhält diese geordnete Auswahl über DlMuMimoRuntimeSelection::GetUsers; nur ein anschließend ausgegebenes DlMuInfo mit exakt denselben STA-IDs, RUs und NSS kann den Zustand durch Lookup oder Install treffen. So bleiben interne WifiTxParameters-Informationen im Scheduler und werden nicht nachträglich teilweise gefiltert.

Die feste Gruppierung übernimmt eine explizite Nutzermenge, ignoriert deren Eingabereihenfolge und verwendet danach die unveränderte ZF- oder explizite RZF-Fabrik aus Unterphase 4.3. Die alternativ explizit wählbare MINIMUM_CORRELATION-Gruppierung untersucht alle Teilmengen der angeforderten Größe in lexikographischer Reihenfolge. Für eine Gruppe G ist die ausgegebene Metrik

$$
  \mu(G)=\max_{u\ne v\in G}
  \frac{\left|\sum_{k,t}H_u[k,0,t]H_v^*[k,0,t]\right|}
    {\sqrt{\sum_{k,t}|H_u[k,0,t]|^2}
    \sqrt{\sum_{k,t}|H_v[k,0,t]|^2}}.
$$

Nur Nutzer mit identischer RU kommen gemeinsam in Frage. Überlastete, rangdefiziente oder anderweitig von der Phase-4.3-Fabrik abgelehnte Gruppen werden verworfen. Gewählt wird die verbleibende Gruppe mit kleinstem μ; innerhalb 10^{-15} gleicher Metriken entscheidet die

lexikographische Nutzer-ID-Folge. Gibt es keine belegbare Gruppe, wird kein Zustand erzeugt. Der fertige ZF-/RZF-Precoder behält $\lVert W[k]\rVert_F^2=|G|$ und damit die gesamte AP-Leistung.

Für den tatsächlichen Sendezeitpunkt t_{tx} wird unverändert der Phase-3-Vertrag MimoCsiTimingConfiguration::ResolveCsiTimestamp benutzt:

$$
t_{\mathrm{csi}}=
\begin{cases}
  t_{\mathrm{tx}}-t_{\mathrm{fb}}, & T_{\mathrm{upd}}=0,\\
  T_{\mathrm{upd}}\left\lfloor
   \dfrac{t_{\mathrm{tx}}-t_{\mathrm{fb}}}{T_{\mathrm{upd}}}
  \right\rfloor, & T_{\mathrm{upd}}>0.
\end{cases}
$$

Snapshot und alle Nutzer müssen exakt diesen gerichteten CSI-Zeitpunkt tragen. Auswahl und Diagnose speichern t_{csi}, t_{tx}, CSI-Alter, Aktualisierungsintervall und Rückmeldedauer. Ein Lookup für einen anderen Sendezeitpunkt, eine andere Gruppe, AP, RU, NSS oder Richtung schlägt fehl. Pro Schlüssel bleibt der neueste Zustand erhalten; ein älterer Ersatz sowie verschiedene CFRs unter derselben CSI-Identität werden atomar abgelehnt.

DlMuMimoFeedbackRepresentation unterscheidet vollständige perfekte MU-Kanalschätzung, quantisierte MU-Kanalschätzung und einen rekonstruierten SU-Precoder. In dieser Unterphase ist ausschließlich PERFECT_CHANNEL unterstützt. Die beiden anderen Varianten werden eindeutig abgelehnt; insbesondere wird ein SU-Precoder nicht als vollständige Mehrnutzer-CFR interpretiert. Install prüft zuerst den vollständigen Laufzeitschlüssel und Zeitpunkt und übergibt erst danach genau den gespeicherten Snapshot und Precoder an SionnaSpectrumPropagationLossModel::SetDlMuMimoState.

Alle Fehler der neuen wertbasierten API sind std::invalid_argument mit dem Präfix DlMuMimoRuntime. Konstruktion, Auswahl und Lookup prüfen Konfiguration, Richtung, Zeit, Endpunkte, Nutzer-, STA-ID-, PSDU-, RU- und NSS-Mengen, Frequenz-/Antennendimensionen, Endlichkeit, Last und Rang vor einer Zustandsänderung. Protocol Buffers, Serialisierung, Batching, Rohkanalcache, ZMQ, Server, Sionna RT, SU-Helper, EESM/PER und der Spectrum-Rechenpfad bleiben unverändert. Unterphase 4.6 ergänzt den konkreten End-to-End-Scheduler und die Helperfreigabe. Automatische Sounding-/Feedbackereignisse, EHT und Uplink bleiben ausdrücklich offen.

5.17 GEMEINSAME UPLINK-ZF-/MMSE-DETEKTION DER UNTERPHASE 4.7

Der gerichtete Uplink-Snapshot enthält je Nutzer genau einen Sendekanal

$$
h_u[k]\in\mathbb C^{N_{\mathrm{rx,AP}}\times1}.
$$

Alle Nutzer enden am selben AP, besitzen dasselbe Frequenzraster und denselben CSI-Zeitstempel. Unterphase 4.7 nimmt ausdrücklich ideale Zeit- und Frequenzsynchronisation, eine Sendeantenne und den lokalen Stream null je Nutzer an. Mehrere Nutzerstreams, Synchronisationsfehler sowie die Bindung an HE-TB-Trigger und tatsächliche gleichzeitige PHY-Empfänge bleiben Unterphase 4.8.

Die nutzerspezifische Referenz-SNR

$$
  \rho_u=\frac{P_{\mathrm{tx},u}10^{-L_u/10}}{N_{\mathrm{AP}}}
$$

enthält Sendeleistung, Wideband-Pfadverlust und AP-Rauschleistung genau
einmal. `UlMuMimoUserSnr` übergibt diese Werte getrennt; es gibt keine
Gruppenmittelung oder gemeinsame Leistungsnormierung. Die HE-Abstraktion
trägt dafür je Nutzer einen eigenen `WifiLinkBudgetResult`. Der im Snapshot
separat sichtbare Pfadverlust wird im Numerikkern nicht erneut angewendet.
Mit

$$
  H_{\mathrm{UL}}[k]=[h_0[k],\ldots,h_{U-1}[k]]
$$

lautet das rausch-normalisierte Signalmodell

$$
  y[k]=\sum_u\sqrt{\rho_u}\,h_u[k]x_u[k]+n[k],
  \qquad E[nn^H]=I.
$$

Der ZF-Detektor verlangt $U\leq N_{\mathrm{rx,AP}}$ und vollen
numerischen Rang auf jedem Subcarrier. Mit derselben relativen
Standardrangschwelle 10^{-12} wie die bisherigen SVD-Kerne werden seine
Kombinierer als Spalten von

$$
  F_{\mathrm{ZF}}[k]
    =H_{\mathrm{UL}}[k]
    \bigl(H_{\mathrm{UL}}^H[k]H_{\mathrm{UL}}[k]\bigr)^{-1}
$$

mittels dünner Jacobi-SVD berechnet. Überlast oder Rangverlust werden vor
einem Ergebnis als `std::invalid_argument` abgelehnt. Das lineare MMSE bleibt
auch für rangdefiziente Gruppen definiert:

$$
  R[k]=I+\sum_v\rho_v h_v[k]h_v^H[k],
  \qquad f_u[k]=R^{-1}[k]h_u[k].
$$

Für beide Detektoren werden die nicht beliebig normierten Kombinierer
vollständig ausgewertet:

$$
  D_u[k]=\rho_u|f_u^H[k]h_u[k]|^2,
$$

$$
  I_u[k]=\sum_{v\ne u}\rho_v|f_u^H[k]h_v[k]|^2,
  \qquad N_u[k]=\lVert f_u[k]\rVert_2^2,
$$

$$
  \gamma_u[k]=\frac{D_u[k]}{I_u[k]+N_u[k]}.
$$

Damit bleiben insbesondere Nah-Fern-Leistungen dem verursachenden Nutzer
zugeordnet. Für einen Nutzer reduzieren ZF und MMSE auf
$\gamma=\rho\lVert h\rVert_2^2$ und reproduzieren den vorhandenen

SU-SIMO-/MRC-Grenzfall.

`HeUlMuMimoLinkAbstraction` komprimiert ausschließlich die SINRs des jeweiligen Nutzers mit dessen explizitem MCS-Beta und fragt anschließend das vorhandene `ErrorRateModel` mit dessen `WifiTxVector` und Paketgröße ab. STA-ID, MCS, Paketlänge, Link Budget, effektive SINR und PER bleiben damit nutzerweise getrennt. Diese wertbasierte API installiert keinen PHY-Zustand.

Vor jeder Rechnung werden Uplinkrichtung, stabile Nutzerordnung, gemeinsamer AP und Empfangsarray, Zeitstempel, Frequenzraster, CFR-Endlichkeit, Ein-Stream-Grenze, vollständige Zuordnungen, getrennte endliche SNRs und gegebenenfalls ZF-Rang geprüft. Snapshot und Eingaben werden nur gelesen.

5.18 HE-TB-/TRIGGERINTEGRATION DER UNTERPHASE 4.8

`HeUlMuMimoPhyAdapter` bildet die schmale Grenze zwischen ns-3-Wi-Fi-PHY-Objekten und dem wertbasierten Uplinkkern. Der Aufrufer übergibt den dekodierten `CtrlTriggerHeader`, dessen PPDU-UID, stabile Bindungen $((u,\mathrm{STA-ID},n_\mathrm{tx}))$, den gerichteten Uplink-Snapshot sowie die am AP beobachteten `HePpdu`-Empfangsintervalle. Zulässig sind ausschließlich HE-Basic-Trigger und davon ausgelöste `WIFI_PREAMBLE_HE_TB`-PPDUs mit derselben UID. RU, MCS und NSS stammen ausschließlich aus den User-Info-Feldern des Triggers; das entspricht der ns-3-HE-TB-Semantik, bei der der AP diese nicht selbsttragenden PHY-Angaben aus seinem TRIGVECTOR ergänzt.

Für das explizite ideale Empfangsintervall $[t_0,t_1)$ und eine RU r lautet die tatsächlich gemeinsam detektierte Nutzermenge

$$
\mathcal A_r =
\left\{u\;\middle|\;
  \mathrm{uid}_u=\mathrm{uid}_\mathrm{TF},
  \mathrm{RU}_u=r,
  t_{\mathrm{start},u}=t_0,
  t_{\mathrm{end},u}=t_1
\right\}.
$$

Der Adapter konstruiert für jede nichtleere Menge $\mathcal A_r$ einen unveränderten, nach Nutzer-ID geordneten Teilsnapshot und ruft genau einmal `HeUlMuMimoLinkAbstraction` auf. Nutzer verschiedener RUs gehen nie in dieselbe Interferenzkovarianz ein. Ein im Trigger angekündigter Nutzer ohne PPDU erscheint in `GetMissingStaIds()`; ein gültiges HE-TB-PPDU mit abweichendem Start oder Ende erscheint in `GetNonSimultaneousStaIds()`. Beide Kategorien bleiben aus jedem Detektionszustand ausgeschlossen. Unbekannte oder doppelte STA-IDs, falsche Sender-/AP-Knoten, andere Trigger-UIDs, falsche Präambeln, ungültige Leistung oder Dauer, Antennendimensionsfehler und fehlende MCS-Betas brechen atomar als `std::invalid_argument` ab.

Je tatsächlich teilnehmendem Nutzer berechnet der Adapter das Link Budget mit der Bandbreite seiner RU und reicht Paketgröße, belegte CFR-Töne und Trigger-TxVector an die bestehende nutzerweise EESM-/PER-Abbildung weiter. Für eine nachfolgende skalare SpectrumWifiPhy-Einkopplung steht analog zum Downlink die äquivalente Empfangsleistung

$$
P_{\mathrm{eq},u} =
  \frac{\gamma_{\mathrm{eff},u}N_{\mathrm{AP},u}}

$$
    {G_{\mathrm{div},u}}
$$

bereit. $N_{\mathrm{AP},u}$ enthält thermisches Rauschen, RU-Bandbreite und Rauschzahl genau einmal; $G_{\mathrm{div},u}=N_{\mathrm{rx,AP}}$ korrigiert nur den zusätzlichen AWGN-Diversityfaktor des ns-3-ErrorRateModel. Die matrixwertige gewünschte Leistung, Inter-User-Interferenz, Rauschverstärkung, SINR und PER bleiben zusätzlich vollständig im Gruppenergebnis erhalten.

5.19 KALIBRIERWORKFLOW DER UNTERPHASE 5.1

Das neue CPU-Beispiel `ns3_wifi_mimo_awgn_curve` ruft für jede SNR-Stützstelle dieselbe `TableBasedErrorRateModel::GetChunkSuccessRate`-Schnittstelle wie `HeSuMimoLinkAbstraction` auf und exportiert `PER=1-P_succ` als CSV. Jede Zeile trägt HE-MCS, BCC/LDPC, Paketlänge, NSS, RX-Antennenzahl, Bandbreite und Guard Interval. Ein Fit gilt immer nur für genau diese Konfiguration. Die Messreferenz enthält zusätzlich AWGN-Paketfehler und für jeden frequenzselektiven Punkt die tatsächlich gemessenen Paketfehler sowie die zugehörige flache Liste linearer Post-Processing-SINRs in derselben Subcarrier-/Stream-Reihenfolge wie `CalculateEffectiveSinr`.

Die unabhängig gemessene AWGN-Kurve ist beim ersten 32-Byte-BCC-Lauf gegenüber dem vorhandenen ns-3-`TableBasedErrorRateModel` verschoben. Deshalb besitzt eine validierte PHY-Konfiguration c neben dem MCS-spezifischen EESM-Beta β_m einen expliziten AWGN-SNR-Offset δ_c in dB. Für einen frequenzselektiven Referenzpunkt i mit n_i Paketversuchen und e_i Paketfehlern lautet die Vorhersage

$$
  \widehat p_i(\beta_m,\delta_c)
  = F_{\mathrm{AWGN}}
    \left(10\log_{10}\gamma_{\mathrm{EESM},i}(\beta_m)+\delta_c\right).
$$

wobei F_{AWGN} die linear in dB-SNR interpolierte, unveränderte ns-3-AWGN-PER-Kurve ist. Zuerst wird δ_c ausschließlich auf den AWGN-Trainingspunkten bestimmt. Danach wird β_m mit festem δ_c auf den selektiven Trainingskanälen bestimmt. Für beide Schritte wird die binomiale negative Log-Likelihood minimiert:

$$
  L(\theta)=-\sum_{i\in\mathrm{Train}}
  \left[e_i\ln\widehat p_i(\theta)
  +(n_i-e_i)\ln(1-\widehat p_i(\theta))\right],
  \quad \theta\in\{\delta_c,\beta_m\},\quad \beta_m>0.
$$

Die Implementierung sucht zunächst 601 äquidistante Offsets im Bereich -3 bis +3 dB und danach 201 logarithmisch verteilte Betas im expliziten positiven Intervall. Jedes beste innere Intervall wird per Goldener-Schnitt-Suche verfeinert. EESM verwendet wie der C++-Kern den kleinsten SINR als Exponentialreferenz. SNR-Werte außerhalb der exportierten AWGN-Kurve werden nicht extrapoliert; ein Randoptimum wird zurückgewiesen. AWGN- und Frequenzselektiv-Referenzen müssen für dieselben PHY-Parameter vorliegen. Mindestens zwei AWGN-Trainingspunkte und ein AWGN-Validierungspunkt sowie zwei selektive Trainingskanäle und ein separater Validierungskanal sind erforderlich. Dieselbe `case_id` darf nicht in beiden Kanalgruppen liegen.

Das JSON der Link-Level-Referenz besitzt `source`, `source_version`, `source_kind` (`link_level` oder `synthetic`), `phy`, `awgn` und `selective`. `phy` enthält die sieben CSV-Konfigurationsfelder. Jeder AWGN-Punkt enthält

`snr_db`, `split`, `errors` und `trials`. Jeder selektive Punkt enthält `case_id`,
`split` (`train`/`validation`), `sinr_linear`, `errors` und `trials`.
`sinr_linear` enthält mindestens zwei nicht gleiche, endliche, nichtnegative
Werte; alle selektiven Punkte besitzen dieselbe Länge. Die Fehlerzahlen
stammen aus tatsächlichen Paketversuchen und sind keine gerundeten
Modell-PER-Werte. Je Punkt werden mindestens 1000 Paketversuche verlangt;
AWGN-Trainings-, AWGN-Validierungs-, selektive Trainings- und
selektive Validierungsmenge benötigen jeweils wenigstens einen
Punkt mit \(0<e_i<n_i\). Der Hash beider Eingabedateien wird im Ergebnis
gesichert.

Der erste reproduzierbare MCS-0-BCC-Lauf mit 32 Byte wird vom ns-3-Verzeichnis
aus erzeugt und gefittet:

```
    matlab -batch "addpath('contrib/sionna/tools');
    generate_he_link_level_reference('contrib/sionna/data/he_mcs0_bcc_32_matlab_r2026a.json',1000)"
    ./ns3 run 'ns3_wifi_mimo_awgn_curve --mcs=0 --packetSizeBytes=32 --ldpc=0 --snrStartDb=-10
    --snrStopDb=15 --snrStepDb=0.25' > contrib/sionna/data/he_mcs0_bcc_32_ns3_awgn.csv
    python contrib/sionna/tools/calibrate_eesm.py \
    --model-awgn contrib/sionna/data/he_mcs0_bcc_32_ns3_awgn.csv \
    --reference contrib/sionna/data/he_mcs0_bcc_32_matlab_r2026a.json \
    --output contrib/sionna/data/he_mcs0_bcc_32_calibration.json
```

`status=validated` erfordert standardmäßig einen mittleren absoluten
AWGN-PER-Fehler von höchstens 0,05 auf den zurückgehaltenen AWGN-Punkten
und einen mittleren absoluten PER-Fehler von höchstens 0,10 auf den
zurückgehaltenen selektiven Punkten. Nur dann trägt das Ergebnis
`beta_linear`, `beta_db` und `awgn_snr_offset_db`. Die Laufzeit wendet
\(\gamma_{\mathrm{ErrorRateModel}}=
\gamma_{\mathrm{EESM}}10^{\delta_c/10}\) an und erhält die ausgegebene
physikalische EESM-SINR unverändert. `HeAwgnCalibrationKey` bindet den
Offset an MCS, BCC/LDPC, Paketlänge, NSS, RX-Antennenzahl, Kanalbreite und
Guard Interval; unpassende HE-Konfigurationen und alle Legacyrahmen
behalten den unveränderten ErrorRateModel-Pfad. `SionnaMimoHelper` gibt
Betas und exakte Offsetzuordnungen an den Spectrumadapter weiter; die
direkten SU-/DL-MU-/UL-MU-Abstraktionen unterstützen denselben Vertrag.
Für den versionierten 32-Byte-BCC-Fall lautet die explizite Helperkonfiguration:

```
    SionnaMimoConfiguration config;
    config.eesmBetaLinear.emplace(0, 1.0383687667246768);
    config.awgnSnrOffsetsDb.emplace(
    HeAwgnCalibrationKey{0, false, 32, 1, 1, 20, 800},
    -0.49589212058827303);
    SionnaMimoHelper mimo(config);
```

Bei synthetischer Quelle oder überschrittenem Grenzwert lautet der Status
`rejected`; nur `candidate_beta_linear` und
`candidate_awgn_snr_offset_db` bleiben zur Diagnose sichtbar. Für andere
PHY-Konfigurationen sind neue Link-Level-Referenzen erforderlich.

5.20 MESSVERTRAG UND FORMELN DES BENCHMARKS AUS UNTERPHASE 5.2.1

Feste Szene ist `simple_room/simple_room.xml` einschließlich ihrer
OBJ-/Materialdateien. TX-Knoten 0 befindet sich bei (3,2,2) m und RX-Knoten 1
bei (4,3,1) m. Beide verwenden ein vertikal polarisiertes 1x2-PlanarArray;
TX verwendet `tr38901`, RX `dipole`, der ungenutzte vertikale Abstand ist
0,5 Wellenlängen. Der native Seed ist 42, Run 1; der Server und sein
`PathSolver` verwenden ebenfalls Seed 42. Mittenfrequenz ist 5180 MHz,
Bandbreite 20 MHz, FFT-Länge 256, Tonabstand 78125 Hz. Jeder Request gilt
bei Zeit null im P2P-/Positionsmodus mit fester Kohärenzdauer 5 ms.

Der feste CPU-Vertrag nutzt `llvm_ad_mono_polarized` mit einem Dr.Jit-Worker,
100000 Samples je Quelle, Tiefe 3, LOS, spekulare Reflexion und Refraktion.
Diffuse Reflexion, Diffraktion, Kantendiffraktion und synthetische Arrays
sind ausgeschaltet; tatsächliche Antennenelemente werden raygetraced.
Alle Fälle verwenden dieselben Solveroptionen. Die normalen Serverdefaults
werden durch diesen eigenständigen Workflow nicht verändert. Ein früher
Vier-Worker-Probelauf zeigte trotz gleicher Seeds bis zu etwa 4,4 ppm Drift
des schwachen Singularwerts. Die feste Ein-Worker-Ausführung beseitigte diese
Drift und lieferte für alle zwölf Paare byteidentische Antworten.

Die Arraykoordinaten vor der Knotendrehung lauten in Wellenlängen

$$
  \mathbf p_0=(0,-d/2,0),\qquad \mathbf p_1=(0,d/2,0),
  \qquad d\in\{0{,}25,0{,}5,1\}.
$$

Orientierungen werden explizit in Grad konfiguriert und gemäß Abschnitt 6.5
mit $\pi/180$ in Sionna-Radiant umgerechnet. Die Validierung liest
`scene.tx_array.normalized_positions`, `scene.rx_array.normalized_positions`
sowie Position und Orientierung der tatsächlich platzierten Objekte `tx`
und `rx1`; sie prüft daher mehr als die exportierten Konfigurationswerte.

Die bestehende gemeinsame CFR-Normalisierung aus Abschnitt 4.3 bleibt
unverändert. Der vom Raytracer erzeugte Wideband-Verlust darf sich durch
Geometrie und Elementmuster ändern; nur die Eingabekontrollen bleiben fest.
Bei 20 dBm Gesamtleistung, 290 K und 7 dB Rauschzahl gilt

$$
  N=1{,}3803\cdot10^{-23}\,290\,(20\cdot10^6)\,10^{7/10}
    =4{,}0123746837392\cdot10^{-13}\ \mathrm W,
  \qquad
  \rho=\frac{10^{(20-L_\mathrm{wb}-30)/10}}{N}.
$$

`WifiLinkBudget` berechnet diese Größen. Ohne Precoding trägt jede
TX-Antenne einen Stream, sodass $N_s=2$. Der native Kern berechnet

$$
  E[k]=\left(I+\frac{\rho}{2}H[k]^H H[k]\right)^{-1},
  \qquad \gamma_s[k]=\frac{1}{E_{s,s}[k]}-1,
$$

$$
  \gamma_{\mathrm{MI},s}
    =\exp\left(\frac1{256}\sum_k\ln(1+\gamma_s[k])\right)-1,
  \qquad
  \eta=\frac1{256}\sum_{k,s}\log_2(1+\gamma_s[k]).
$$

Die CSV-SINRs sind linear; nur die Zusammenfassung und Plots verwenden
zusätzlich $10\log_{10}\gamma_{\mathrm{MI},s}$. MI ist hier eine
kalibrierungsfreie Diagnose. Die SVD von $H[k]$ liefert
$\sigma_0[k]\geq\sigma_1[k]\geq0$, den Rang
$r[k]=\#\{i:\sigma_i[k]>10^{-12}\sigma_0[k]\}$ und den
zusätzlichen Konditionsindikator
$\operatorname{median}_k(\sigma_1[k]/\sigma_0[k])$.
Es wird kein SVD-Precoder auf den Kanal angewendet. Singularwerte gehören
zur normalisierten CFR; rohe Singularwerte lassen sich durch Multiplikation
mit $10^{-L_\mathrm{wb}/20}$ rekonstruieren. Der numerische Rang allein
beschreibt nicht die Nutzbarkeit einer schwachen Mode.

Alle 256 FFT-Bins werden mit gleichem Gewicht ausgewertet, einschließlich
DC und Guard-/Nulltönen. Das ist ein vollständig nachvollziehbarer
Kanalsensitivitätsvergleich und keine auf belegte HE-Nutzdatentöne beschränkte
PER-/Durchsatzprognose. \(\eta\) ist die ideale gaußsche spektrale Effizienz,
keine gemessene 802.11-Nutzdatenrate. Auch der feste kleine Sampleumfang ist
noch keine Konvergenzstudie.

`init_seconds` misst das Laden und Initialisieren der Szene;
`cfr_seconds` misst den Serveraufruf einschließlich Platzierung, Raytracing,
CFR-Konvertierung und Antwortaufbau. `analysis_seconds` misst im nativen
Programm Deserialisierung, Prüfung, Linkbudget, MMSE/MI und SVD, vor dem
CSV-Schreiben. `native_process_seconds` enthält zusätzlich Prozessstart
und CSV-Ausgabe; `case_seconds` umfasst einen einzelnen Lauf einschließlich
Geometrieprüfung, Ergebnislesung und Wiederholungsvergleich, nach dem
Konfigurationsexport. `sweep_seconds` enthält alle Fälle und
Konfigurationsexporte, nach Python-/Sionna-Import und vor Plot-/Manifestausgabe.
Diese Wanduhrzeiten enthalten gegebenenfalls JIT-Aufwand und sind weder
bitidentisch noch unabhängige Performance-Stichproben. IPC-/Cache-Latenz und
MAC-/PHY-Laufzeit werden in dieser Unterphase nicht gemessen.

`data/sumimo_frozen_reference.json` definiert zusätzlich fünf analytische
2x2-Töne: \(\sqrt2 I\), die Einsmatrix, eine phasengedrehte Diagonalmatrix
mit Betrag 2, eine Nullmatrix und eine komplexe asymmetrische Matrix. Ihr
gemeinsames mittleres Antennenpaarleistungsniveau ist eins. Die ersten vier
Fälle liefern pro Stream die SINRs \(\rho\), \(\rho/(1+\rho)\),
\(2\rho\) und null. Für den fünften Fall sind die Gram-Diagonalen 1,6 und
2,4 und das Betragsquadrat des Kreuzterms 1,28. Mit \(a=\rho/2\) folgt

\[
  \gamma_0=1{,}6a-\frac{1{,}28a^2}{1+2{,}4a},\qquad
  \gamma_1=2{,}4a-\frac{1{,}28a^2}{1+1{,}6a}.
\]

Damit entstehen unabhängige Referenzwerte für Rangverlust, Nulltöne,
Frequenzselektivität, komplexe Koeffizienten und flache CFR-Reihenfolge.
Die Tests vergleichen zusätzlich alle realen eingefrorenen Kanäle mit
NumPy-SVD und unabhängig invertierter MMSE-Fehlerkovarianz. Diese NumPy-
Rechnung ist ausschließlich Validierung; die exportierte Linkabstraktion
stammt aus dem vorhandenen C++-Kern.

5.21 MESSVERTRAG UND KONVERGENZFORMELN DER UNTERPHASE 5.2.2

Die Kontrollen, Normalisierung, MMSE-/MI-/SVD-Formeln und Laufzeitgrenzen
aus Abschnitt 5.20 bleiben gültig. Nur Samples je Quelle werden variiert;
die Seedmenge wird über alle Budgets gepaart. Pro Seed setzt der native
Konfigurationsexport `RngSeedManager::SetSeed(seed)` bei Run 1 und schreibt
denselben Seed in `SimInitMessage`. Der Server verwendet ihn für NumPy und
`PathSolver`. Eine Konfigurationsdatei je Fall/Seed wird unverändert für
alle drei Budgets verwendet. Die neue optionale native `--seed`-Angabe
hat weiterhin den Default 42; der alte 5.2.1-Export bleibt bytekompatibel.
`benchmark_sumimo.trace` akzeptiert ein explizites Samplebudget, ohne globale
Kontrollen oder Serverdefaults zu verändern. Die Linkabstraktion verbleibt
vollständig im bestehenden nativen ns-3-Programm.

Die Fallauswahl beruht ausschließlich auf dem früheren 100000-Sample-Lauf
mit Seed 42; die neuen Ergebnisse beeinflussen Auswahl und Toleranzen nicht:

Fall	Abstand / Lambda	TX/RX-Azimut / Grad	Auswahlgrund aus 5.2.1
03	0,25	90/90	kleinstes medianes sigma1/sigma0: 0,030958

05	0,5	0/90	gemischte Orientierung, Verhältnis 0,256484
06	0,5	90/0	größtes medianes sigma1/sigma0: 0,515959
10	1	90/0	höchste MI-Effizienz: 39,902643 bit/s/Hz

Vor dem ersten Raytrace wurden folgende technische Toleranzen festgelegt
und zusammen mit Zeitstempel in `plan.json` exportiert. Sie sind praktische
Stabilitätskriterien, keine aus PER abgeleiteten Unsicherheitsgrenzen:

Metrik	Kriterium für gleiches Fall-/Seed-Paar
Wideband-Verlust	absolute Differenz höchstens 0,5 dB
Numerischer Rang	identischer Rang an allen 256 Tönen
Beide Singularwerte	höchstens 5 Prozent relativ plus 1e-7 absolut, an jedem Ton
Post-MMSE-Stream-SINR	größte dB-Differenz über Töne/Streams höchstens 1 dB
MI-Effective-SINR	größte dB-Differenz über beide Streams höchstens 0,5 dB
MI-Effizienz	absolute Differenz höchstens 0,2 bit/s/Hz

Für Budget b und größeres Referenzbudget q bei gleichem Seed gilt mit
den normalisierten Singularwerten aus Abschnitt 5.20

$$
  D_\sigma(b,q)=\max_{k,i}
    \frac{|\sigma_i^{(b)}[k]-\sigma_i^{(q)}[k]|}
    {10^{-7}+0{,}05|\sigma_i^{(q)}[k]|}\leq1.
$$

Die relative Toleranz bezieht sich auf q; der absolute Term hält Nullmoden
definiert. Für SINR-Differenzen verwendet die Auswertung

$$
  g(x)=10\log_{10}(\max(x,10^{-12})),\qquad
  D_\gamma(b,q)=\max_{k,s}|g(\gamma_s^{(b)}[k])-g(\gamma_s^{(q)}[k])|,
$$

$$
  D_\mathrm{MI}(b,q)=\max_s|g(\gamma_{\mathrm{MI},s}^{(b)})
    -g(\gamma_{\mathrm{MI},s}^{(q)})|,\quad
  D_\eta(b,q)=|\eta^{(b)}-\eta^{(q)}|.
$$

Der Floor entspricht -120 dB und dient ausschließlich dem Vergleich
verschwindender SINRs; die exportierten nativen SINRs bleiben unverändert.
Es werden 10000/100000, 100000/1000000 und 10000/1000000 verglichen:
36 Zeilen für vier Fälle und drei Seeds. Ein Paar besteht nur, wenn alle
sechs Kriterien erfüllt sind. Der globale Konvergenzbefund verlangt alle
zwölf 100000/1000000-Paare. 10000 Samples werden nur dann als ausreichend
ausgewiesen, wenn zusätzlich beide Vergleiche mit 10000 für alle Paare
bestehen. Andernfalls sind bei bestandenem oberen Schritt 100000 Samples
das kleinste unterstützte Budget. Wenn dieser Schritt scheitert, wird kein
ausreichendes Budget angegeben; 1000000 wird nicht tautologisch als
konvergiert bezeichnet.

Verglichen werden gepaarte Seeds und sämtliche Tonwerte, keine nur über
Seeds gemittelten Singularwerte oder SINRs. Die Plots zeigen zusätzlich
Median und Min-/Max-Spanne über Seeds und Wiederholungen. Laufzeiten sind
separat exportierte Wanduhrdiagnosen und kein Konvergenzkriterium.
`plan.json` und `manifest.json` dokumentieren Solveroptionen, feste Kontrollen,
Seed-/Budgetmatrix und Toleranzen. Das Manifest enthält beobachtete reale
Array-/Radio-Geometrie und SHA-256-Hashes der Konfigurationen, Kanalantworten,
Szenendateien, relevanten Quellen und nativen Binärdatei.

1000000 Samples sind eine endliche numerische Referenz. Der Befund ist auf die vier statischen Links dieser Szene, drei Seeds, Tiefe 3 und die dokumentierten aktivierten Mechanismen begrenzt. Ein Plateau belegt weder asymptotische Konvergenz noch Vollständigkeit aller physikalischen Pfade. Diffuse Reflexion, Diffraktion und andere Szenen werden nicht erfasst. Die 256 FFT-Bins einschließlich DC/Guards und gaußsche MI-Effizienz bleiben derselbe Kanalvergleich wie in 5.2.1; daraus entsteht keine 2x2-PER- oder Wi-Fi-Durchsatzkalibrierung.

5.22 MESSVERTRAG UND FORMELN DER UNTERPHASE 5.2.3

Dieser Benchmark isoliert Datenhaltung und Transport. Sein eingefrorener Anker ist der 100000-Sample-/Seed-42-Kanal von Fall 05 aus Unterphase 5.2.2. Die 256x2x2-CFR wird für jede gewünschte Transportdimension deterministisch mit sc/rx/tx-Indizes modulo 256/2/2 wiederholt. Für Linknummer l gilt

$$
\begin{aligned}
&H_l[k,r,t]=\mathrm i^{(42+l-1)\bmod4}\\
&\quad H_\mathrm{Anker}[k\bmod256,r\bmod2,t\bmod2],\\
&\quad L_l=L_\mathrm{Anker}+(l-1)/8.
\end{aligned}
$$

Die vier Phasen werden mit exakten Vorzeichen-/Real-/Imaginärtauschoperationen erzeugt. Der Delay in ns erhöht sich um l-1; IDs sind TX 0 und RX 1 bis L. Alle Radios behalten die Ankerpositionen; der Frequenzvektor enthält (k-Nsc/2)*78125 Hz in der dokumentierten Reihenfolge. Die größere CFR ist eine Transportfixture, keine neue physikalische Array-/Bandbreitensimulation. Rang, SINR, PER und Wi-Fi-Durchsatz werden hier nicht ausgewertet.

Alle L Links kommen in einem gemeinsamen `ChannelStateResponse`-Snapshot innerhalb eines `Wrapper`. Die größeren Antworten werden bei Bedarf aus einer der 15 gespeicherten Einzel-Link-Vorlagen rekonstruiert. Manifest und Tests prüfen sowohl Vorlagenhashes als auch SHA-256 der vollständigen rekonstruierten Batchantwort. Der native Prüfer vergleicht jedes gecachte komplexe Element bitgenau und alle Metadaten exakt mit dieser Antwort. Zusätzlich berechnet NumPy unabhängig eine geordnete, nichtkryptografische IEEE-754-Bitprüfsumme über sämtliche Koeffizienten. Dabei wird mit globalem flachem Index j das XOR der beiden Produkte bits(real_j)*(2j+1) und bits(imag_j)*(2j+3), jeweils modulo 2^64, gebildet.

Mit K=Nsc*Nrx*Ntx pro Link und double-Präzision gilt für den logisch gespeicherten Nutzdatenspeicher des Caches auf der Referenzplattform

$$
\begin{aligned}
&B_\mathrm{CFR}=16LK,\qquad B_f=4LN_\mathrm{sc},\qquad\\
&B_\mathrm{logisch}=L\,N_\mathrm{sc}(16N_\mathrm{rx}N_\mathrm{tx}+4).
\end{aligned}
$$

Dieser Wert enthält komplexe Koeffizienten und int32-Frequenzen, aber keine CacheEntry-/Map-/Vektorköpfe, Allokatorausrichtung oder Socketpuffer. Die tatsächliche serialisierte Größe B_wire ist die Länge des kanonischen `Wrapper.SerializeToString`-Ergebnisses; der native C++-Encoder muss exakt dieselben Bytes erzeugen. Die doppelt gepackten CSI-double-Arrays tragen ebenfalls 16LK Bytes bei. Protobuf-Tags, Längenpräfixe, Positionen, IDs, Delay, Verlust und die gepackten Frequenz-Varints kommen hinzu. Negative int32-Frequenzen verwenden längere Wire-Varints; deshalb ist Wiregröße nicht identisch mit logischem Cache-Nutzdatenspeicher.

Die native glibc-Messung verwendet

$$
```
```

```
  A=\mathtt{mallinfo2().uordblks}+\mathtt{mallinfo2().hblkhd},\qquad
  \Delta A_\mathrm{fill}=A_\mathrm{live}-A_\mathrm{before},\quad
  \Delta A_\mathrm{release}=A_\mathrm{live}-A_\mathrm{released}.
\]

Der Socket wird vorher mit vollständigen Antworten aufgewärmt. Ein eigener
Cache wird anschließend einmal mit dem Batch befüllt, an allen Links
validiert und wieder zerstört. Zwischen den beiden Live-/Release-
Heapschnappschüssen erfolgen keine weiteren Benchmarkallokationen.
Delta A_release misst die beim Löschen dieses Caches sichtbare Änderung
der glibc-Buchhaltung einschließlich Verwaltungsdaten und Allokatorrundung.
Kleine freigegebene Blöcke im Threadcache können weiter in uordblks enthalten
bleiben; das Release-Delta kann deshalb kleiner als die logische
Payloadgröße sein. Es ist keine vollständige Zählung lebender Cacheobjekte.
ZMQ-Hintergrundaktivität kann weitere kleine Störungen verursachen.
Beide Deltas werden vorzeichenbehaftet exportiert und müssen nicht
bitidentisch wiederholbar sein.

RSS und Peak-RSS stammen aus `VmRSS` beziehungsweise `VmHWM` in
`/proc/self/status` des aktuellen Programmadressraums. Der zusätzliche
`getrusage`-Lebenszeitpeak wird separat als Diagnose exportiert; er kann
Speicherzustände vor exec beziehungsweise aus dem Elternprozess enthalten.
Baseline-, Live- und
Release-RSS werden getrennt exportiert. RSS enthält Fixture, Protobuf,
Sockets, Bibliotheken und behaltene freie Allokatorseiten. Ein nach
Cachefreigabe hoher RSS-Wert ist daher keine Messung des verbleibenden
Cacheinhalts. Weder Payloadgröße noch RSS werden als alleinige genaue
Cacheallokationsgröße ausgegeben.
Die Prozesspeaks umfassen sämtliche Messphasen einschließlich Fixture,
temporärer Dekodierungen und Expirationsprobe; der logische Cachebedarf
bezieht sich auf genau einen Snapshot je Link im separaten Speicherprüfcache.

Es gelten 16 Zeitstichproben pro Serialisierungs-/Parse-/CFR-Dekodierungs-
und Roundtripoperation, drei Warmups, 128 gemessene Getter-Hits und fünf
frische Cache-Misses je Prozess. Die drei Prozesse je Fall erzeugen
insgesamt 135 Läufe und 30915 Zeitstichproben. C++ misst mit steady_clock,
Python mit perf_counter_ns. Zusammenfassungen enthalten Median, 95.
Perzentil, Minimum und Maximum pro Operation/Prozess; die Plots zeigen
Median und Min-/Max-Spanne der drei Prozessmediane.

Die Timergrenzen sind bewusst explizit:

  - C++-Serialisierung und Parse: vollständiger Response-Wrapper,
  einschließlich lokaler Ergebnisfreigabe; einzelne Matrixdekodierung:
  `DeserializeChannelMatrix` nur für einen RX-Kanal einschließlich Freigabe.
  - Python-Serialisierung und Parse: vollständiger Wrapper; Freigabe des
  lokalen Ergebnisses erfolgt nach dem Timer. Diese Werte sind daher kein
  identischer Mikrobenchmarkvertrag zu den C++-Werten.
  - Roundtrip: tatsächliches Send/Recv über lokalen ipc://-Transport zu
  einem zweiten Prozess. Requestserialisierung, vollständiger Bytevergleich,
  Socketaufbau und Warmups liegen außerhalb der gemessenen Stichproben.
  Der REP-Prozess prüft Requests und sendet vorbereitete Antwortbytes.
  - Hit: öffentlicher `GetPropagationChannelMatrix`-Aufruf einschließlich
  CFR-Kopien und sofortiger Rückgabewertfreigabe. Der innere Cachepfad
  kopiert CacheEntry-Werte; gemessen wird daher mehr als reine Map-Suche.
  - Miss: derselbe Getter auf einem neu angelegten Cache, einschließlich
  Requestserialisierung, Send/Recv, Protobuf-/CFR-Dekodierung und Einfügen
  sämtlicher L Links. Cachekonstruktion und Cachezerstörung erfolgen
  außerhalb des Miss-Timers. Dadurch werden keine doppelten alten Snapshots
  in einem wiederholt deaktivierten Cache akkumuliert.
```

Alle Getter bleiben echte Produktionsaufrufe. Die Optimierung für weit entfernte Radios wird ausgeschaltet, damit auch Delay-/Loss-Prüfungen den Cache verwenden. Die anfängliche Gültigkeit ist [0,5000000] ns. Am Zeitpunkt 5000000 muss ein Hit ohne IPC erfolgen; bei 5000001 muss ein Miss einen neuen Snapshot [5000001,10000001] anfordern. Nur Zeitfelder ändern sich bei dieser vorbereiteten Antwort. Das prüft den inklusiven Gültigkeitsrand und die produktive Expiration ohne Raytracing.

Die erwartete Requestzahl je Prozess ist

$$
R=(3+16)_\mathrm{Roundtrip}+1_\mathrm{memory}+1_\mathrm{warmcache}
  +5_\mathrm{coldmiss}+1_\mathrm{expiry}=27.
$$

26 Requests gelten bei Zeit null, genau einer bei 5000001. Danach muss der bestehende Helper-Closepfad genau ein ACK erhalten. Alle 3645 Channel-Requests werden exportiert. Der Zeitcache hat 132 Hits (3 Warmups, 128 Stichproben, ein Randhit) und zwei Misses; seine Hitquote ist 132/134. Der separate Speicherprüfcache hat 4L Validierungshits (CFR/Frequenz/Loss/Delay) nach einem einzigen Batchmiss: Quote 4L/(4L+1).

Vor der ersten Sweepmessung wird `plan.json` mit folgenden Abnahmekriterien und Zeitstempel geschrieben: vollständige 45x3-Matrix, exakte Fixture-/Kanal-/Metadaten-/Byteerhaltung, exakte logische Speicherformeln, erwartete Requestfolgen und Hit-/Misszahlen mit 1e-12 Quotentoleranz, korrekte Expiration und identische deterministische Ergebnisse aller drei Prozesse. Jede Operation muss vollständig mit endlichen positiven Zeitstichproben vorliegen; RSS und Live-Heap-Schnappschüsse müssen positiv sein. Allokatordeltas und Zeitmediane sind Beobachtungen ohne starre Performance-Abnahmegrenze. Messungen erfolgen sequenziell; es gibt keine gleichzeitig ausgeführten Raytraces, Regressionen oder Benchmarkfälle.

5.23 MESSVERTRAG DER UNTERPHASE 5.2.4

Die beiden 1x2-Arrays haben unabhängige horizontale Abstände d_tx und d_rx in Wellenlängen. Ihre normierten lokalen Elementkoordinaten sind

$$
q_{e,0}=(0,-d_e/2,0),\quad q_{e,1}=(0,d_e/2,0),\qquad
x_{e,i}=p_e+\lambda R_z(\phi_e)q_{e,i},\quad e\in\{tx,rx\}.
$$

Die Zentren p_e bleiben bei (3,2,2)/(4,3,1) m. Elevation und Roll sind null, Azimute werden in Grad übergeben und am tatsächlichen Radioobjekt in Radiant geprüft. Beide tatsächlichen PlanarArrays müssen die jeweiligen lokalen Koordinaten liefern; der RX-Abstand wird unabhängig vom TX geprüft. Die Produktion unterstützt diese Geometrie bereits. Der Benchmark nutzt `spacingLambda` als bisherigen gemeinsamen Default; `txSpacingLambda` und `rxSpacingLambda` überschreiben ihn einzeln.

Die vorab fixierte kleine Matrix lautet:

Fall	d_tx / lambda	d_rx / lambda	TX-Azimut / Grad	RX-Azimut / Grad
00	0,5	0,5	90	0
01	1	0,25	90	0
02	0,25	1	90	0
03	1	0,25	45	-45
04	0,25	1	45	-45

Fall 00 entspricht Fall 06 aus 5.2.2. Die Fälle 01/02 isolieren die vertauschten Abstände bei festen Orientierungen; 03/04 ergänzen ausgewählte neue Winkel. TX verwendet weiterhin tr38901, RX dipole, beide V-Polarisation. Szene, Frequenz, Positionen, 256 FFT-Bins, 20 dBm Gesamtleistung, 290 K, 7 dB Rauschzahl und sämtliche Solveroptionen aus Abschnitt 5.20 bleiben fest. Die drei Seeds sind 42/43/44, Run ist eins. Zwei Wiederholungen pro Seed ergeben 30 Traces bei 100000 Samples und zwölf weitere für 03/04 bei 1000000, insgesamt 42. Der Runner schreibt `plan.json` vor dem ersten Trace.

Vergleiche und Formeln für Rang, Singularwerte, Open-Loop-MMSE, MI-SINR und Linkbudget werden unverändert aus 5.20/5.21 übernommen. Jeder Trace wird zusätzlich mit der bereits vorhandenen unabhängigen NumPy-SVD-/MMSE-/MI-Referenz geprüft; diese Prüfung wird mit den bestehenden Tests geteilt. Die CFR und alle deterministischen nativen Felder müssen innerhalb jedes Falls/Seeds/Budgets wiederholbar sein, die CFR zusätzlich byteidentisch. Die sechs gepaarten 100000/1000000-Spotchecks verwenden vorab dieselben Toleranzen wie 5.2.2: 0,5 dB Verlust, unveränderter Rang auf jedem Ton, 5 Prozent plus 1e-7 Singularwertfehler, 1 dB Ton-SINR, 0,5 dB MI-SINR und 0,2 bit/s/Hz MI-Effizienz. Ein bestandener Spotcheck belegt nur die ausgewählten Fälle und Seeds, nicht die Konvergenz der gesamten Matrix. PER und Wi-Fi-Durchsatz bleiben wegen der ausschließlich dokumentierten 1x1-Kalibrierung außerhalb dieses Messvertrags.

5.24 MESSVERTRAG UND FORMELN DER UNTERPHASE 5.2.5

Die Auswahl verwendet ausschließlich die früheren 5.2.1-Ergebnisse: Fall 03 hat das schwächste mediane zweite Eigenmodeverhältnis (0,030958), Fall 06 das stärkste (0,515959). Die kleine Matrix wurde vor dem ersten Trace zusammen mit Toleranzen und UTC-Zeitstempel in `plan.json` fixiert:

Fall	Gemeinsamer Abstand / lambda	TX/RX-Azimut / Grad	CSI-Alter / ms
03	0,25	90/90	0; 1; 2,5; 5
06	0,5	90/0	0; 1; 2,5; 5

Szene, initiale Zentren (3,2,2)/(4,3,1) m, TX-tr38901-/RX-dipole-1x2-Arrays, V-Polarisation, 5180 MHz, 256 FFT-Bins, 20 dBm Gesamtleistung, 290 K, 20 MHz, 7 dB Rauschzahl und Solveroptionen entsprechen Abschnitt 5.20. Elevation/Roll sind null. Beide Budgets 100000/1000000 werden für beide Fälle und alle Seeds 42/43/44 bei Run 1 mit zwei frischen RT-Wiederholungen gerechnet: 24 initiale RT-CFRs. Dieselbe CFR wird anschließend für alle Alter und beide CSI-Varianten genutzt; es entstehen 192 Auswertungszeilen und 49152 Tonzeilen. Die größeren Budgets prüfen diese repräsentative Matrix, keine anderen Geometrien oder Evolutionsmodelle.

Ein RT-Request bei t=0 liefert die normalisierte initiale Matrix H_0[k] und den Wideband-Verlust. Explizite Entwicklung für t in Millisekunden:

$$
\begin{aligned}
R(\theta)&=\begin{pmatrix}\cos\theta&-\sin\theta\\ \sin\theta&\cos\theta\end{pmatrix},\\
H[k,t]&=H_0[k]R(\omega t),\\
\omega&=9\pi/180\ \mathrm{rad/ms}.
\end{aligned}
$$

Dies ist eine kontrollierte analytische TX-Basismischung auf realen frequenzselektiven RT-Kanälen. Es ist kein aus Bewegung, Pfad-Doppler oder gemessener Kohärenzzeit abgeleitetes Modell. Die Geometrie bleibt konstant; die serverseitigen 5-ms-Gültigkeitsmetadaten steuern diese analytische Entwicklung nicht. Der Sendekanal wird für alle Alter bei t_tx=5 ms ausgewertet. Verzögertes CSI stammt exakt aus H[k,5-a], frisches aus

H[k,5]. Relative Rotationen sind 0/9/22,5/45 Grad. Der CSI-Zeitpunkt
beträgt 5/4/2,5/0 ms; frisches CSI hat stets Zeitpunkt 5 ms und Alter null.
`age_ms` bezeichnet die angefragte Vergleichsachse, `csi_age_ns` das
tatsächliche Alter der jeweiligen frischen/verzögerten Variante.

MimoCsiAgingModel::Select verwendet kontinuierliche Abtastung T_u=0 und
Feedbackverzögerung d_fb=a (frisch: null), prüft t_csi=t_tx-d_fb und erzeugt
unquantisiertes Gleichleistungs-SVD-W[k]=V[k,t_csi]. Der Empfänger kennt
H[k,t_tx] und den gewählten Precoder perfekt. CalculateLink erhält den
Sendekanal getrennt und berechnet mit dem festen Linkbudget aus 5.20

$$
  G[k]=H[k,t_{\mathrm{tx}}]W[k],\qquad
  E[k]=(I+\rho G[k]^H G[k]/2)^{-1},\qquad
  \gamma_s[k]=1/E_{s,s}[k]-1.
$$

MI-SINR und Effizienz verwenden dieselben Mittelungen wie Abschnitt 5.20.
Die Kanalrotation ist unitär. Deshalb bleiben Wideband-Verlust,
Singularwerte, Rang und mittlere Antennenpaarleistung unverändert.
Für zwei Streams gilt W^H W=I und ||W||_F^2=2, damit bleibt die gesamte
TX-Kovarianz P I/2 für frisches und verzögertes CSI gleich. Die gemeinsame
gaußsche MI

$$
  C=\frac1{256}\sum_{k,i}\log_2(1+\rho\sigma_i[k]^2/2)
$$

ist ebenfalls invariant. Frisches SVD liefert eta=C, verzögertes CSI kann
die Summe der getrennten MMSE-Stream-MI eta verringern. Dies misst den
Verlust bei separater Streamdetektion, keinen Verlust der gemeinsamen
MIMO-Kapazität. Stream null folgt der stärksten CSI-Zeitpunkt-Mode;
Rotation kann Stream eins verbessern. Deshalb werden Differenzen mit
Vorzeichen exportiert, nicht auf positive Werte begrenzt:

$$
  D_{\gamma,s}=g(\gamma^{\mathrm{fresh}}_{\mathrm{MI},s})
    -g(\gamma^{\mathrm{delayed}}_{\mathrm{MI},s}),\qquad
  D_\eta=\eta^{\mathrm{fresh}}-\eta^{\mathrm{delayed}},
  \quad g(x)=10\log_{10}(\max(x,10^{-12})).
$$

Ton-SINR-Differenzen verwenden dieselbe Formel ohne MI-Kompression.
Zusätzliche Kanaldiagnosen je Ton sind

$$
  c[k]=\frac{|\operatorname{tr}(H_{\mathrm{csi}}[k]^H H_{\mathrm{tx}}[k])|}
    {\|H_{\mathrm{csi}}[k]\|_F\|H_{\mathrm{tx}}[k]\|_F},\qquad
  e[k]=\frac{\|H_{\mathrm{tx}}[k]-H_{\mathrm{csi}}[k]\|_F}
    {\|H_{\mathrm{tx}}[k]\|_F}.
$$

Bei dieser Rotation sind beide Normen identisch. Der native CSV-Export
enthält initiale/CSI-/Sendezeiten, tatsächliches Alter, T_u/d_fb,
Rotationsrate, c/e, Rang, Singularwerte, W-Leistung, Ton-/MI-Stream-SINR,
Effizienz, Verlust, Referenz-SNR, RX-/Rauschleistung und Analysezeit.
Der Runner ergänzt Konfiguration, Seed, Samplebudget, Wiederholung,
MI-/SINR-Verlust, Init-/CFR-/native Prozesszeiten sowie Tonmediane.
Die initialen CFRs und die dokumentierte Formel ermöglichen natives Replay;
entwickelte CFRs werden nicht nochmals als Fixtures gespeichert.

Alle 48 gepaarten Fall-/Seed-/Alter-/CSI-Varianten vergleichen 100000 mit
1000000 Samples anhand sämtlicher sechs Kriterien aus 5.21:
0,5 dB Verlust, identischer Tonrang, 5 Prozent plus 1e-7 Singularwertfehler,
1 dB Ton-SINR, 0,5 dB MI-SINR und 0,2 bit/s/Hz Effizienz. Diese Grenzen
gelten für Budgetstabilität, nicht als Obergrenzen für Alterungsverluste.
Jeder native Export wird unabhängig mit NumPy-SVD, MMSE-Inversion,
MI und Linkbudget geprüft: SINR 1e-8 relativ/absolut, Singularwerte 1e-10,
MI 1e-9 relativ plus 1e-8 absolut. Wiederholungen verlangen byteidentische
initiale CFRs und die bisherigen deterministischen Feldtoleranzen
1e-6 relativ/1e-7 absolut. Ein analytischer Diagonalanker prüft frisches
SVD und 45-Grad-MMSE-Mischung; Rate null bei Alter 5 ms muss die
statischen frischen Linkmetriken exakt reproduzieren.

Frisches CSI und verzögertes Nullalter teilen dieselbe native Auswertung;
deren Zeitwerte werden auf der Vergleichsachse wiederverwendet.
`analysis_seconds` umfasst Lesen, Prüfungen, Entwicklung, SVD-Auswahl,
MMSE/MI und Singularwertdiagnostik vor CSV-Ausgabe. `native_process_seconds`
misst auch CSV-Lesen im Runner; Init-/CFR-Zeiten gehören zum geteilten
initialen Trace und dürfen nicht über die acht Vergleichszeilen aufsummiert
werden. `trace_and_evaluation_seconds` zählt jeden vollständigen Trace mit
Auswertungen einmal; `sweep_seconds` umfasst auch Konfigurationsexporte,
aber keine Imports/Plots/Manifestausgabe. 120 native Analysen einschließlich
24 statischer Kontrollen erzeugen bei zwei Wiederholungen 192 Exportzeilen.
Die beiden lokalen Plots zeigen SINR-/MI-Verlust sowie Korrelation und
native Laufzeit gegen Alter, als Median und Min-/Max-Spanne über Seeds und
Wiederholungen bei 100000 Samples. Das sind keine Konfidenzintervalle.

5.25 EIGENTUMS- UND MESSVERTRAG DES VIERTEN R3-INKREMENTS

Das Inkrement reduziert ausschließlich CFR-/Frequenzkopien in der nativen
Datenhaltung. Sionna, Protobuf, Kanalnormalisierung, MMSE/EESM/PER,
inklusive Zeitgültigkeit und die Reihenfolge überlappender Einträge bleiben
semantisch identisch. Für einen Link gelten weiterhin
$$
 C=N_{\mathrm{sc}}N_{\mathrm{rx}}N_{\mathrm{tx}},\qquad
 B_H=16C,\quad B_F=4N_{\mathrm{sc}},\quad B_{\mathrm{payload}}=B_H+B_F.
$$
Die Bytefaktoren beziehen sich auf die verwendete Plattform mit
`complex<double>` und `int`; der Produktionscode verwendet `sizeof`.

Zuvor kopierten die beiden Cache-Suchschleifen jeden besuchten `CacheEntry`,
auch bei skalaren Delay-/Loss-Zugriffen und noch nicht passenden Zukunftseinträgen.
Die Schleifen verwenden nun konstante Referenzen. Die private Suche liefert
entweder eine Referenz auf den gespeicherten Eintrag oder verschiebt bei
abgeschaltetem Cache das vollständig validierte Ergebnis in einen lokalen
Scratch-Eintrag des aufrufenden Getters. Nach einem Miss wird die Referenz erst
nach allen Cacheeinfügungen erneut ermittelt, weil Linkvektoren beim Einfügen
reallokieren können. Keine solche Referenz verlässt den öffentlichen Getter.
Der positionsaktualisierende Loss-Aufruf übernimmt seine skalaren Positions-/
Loss-Werte vor den Settern; ein Cachewechsel in einer abgeleiteten
Mobility-Implementierung kann damit keinen späteren Eintragszugriff invalidieren.

Öffentliche Ergebnisse behalten ihre bisherige Wertsemantik. Ein gecachter
Loss-/Delay-Aufruf benötigt keine CFR-/Frequenzkopie; Frequency kopiert nur
B_F, CSI und Matrix nur B_H. Die zurückgegebenen Werte bleiben nach
Mutation, Ablaufbereinigung, Cacheabschaltung und einer späteren Antwort gültig.
Dies ist eine logische Kopierbeschreibung, keine harte Allokator-/Latenzgrenze.
Der ungecachte öffentliche Vektor-/Matrixpfad erstellt weiterhin ein unabhängiges
Ergebnis aus dem getterlokalen Scratch; Protobuf-/Transportkopien bleiben bestehen.

Der Koeffizientenkonstruktor von `MimoChannelMatrix` nimmt seinen Vektor als

Wert entgegen und verschiebt dessen Speicher direkt in die Matrix. Rvalues
übertragen die Allokation; Lvalues erzeugen weiterhin eine unabhängige Kopie.
Die bisherige zusätzliche Nullmatrixallokation entfällt. Der Deserialisierer
füllt einen einmalig auf die vorgeprüfte Länge angelegten Koeffizientenvektor
über indizierte Schreibzugriffe und verschiebt ihn; der Cachekonstruktor
verschiebt die Matrix weiter. Der geplante Batch reserviert seine Eintragszahl
innerhalb der bestehenden Überlauf-/Budgetprüfung. Ein Neuaufbau der nativen
Bibliothek und ihrer Verbraucher ist wegen der Konstruktor-Signatur erforderlich.

Ein MU-Snapshot wird weiterhin vollständig einschließlich Richtung, RX-Geometrie,
Normalisierung und gemeinsamer Frequenz-/TX-Dimension geprüft. Seine einmalig
dekodierten, stabil nach Nutzer-ID sortierten Matrizen und Frequenzen werden
anschließend anhand der expliziten Nutzer-ID in die ursprüngliche
Antwortreihenfolge überführt und verschoben. Eine abweichende Wire-Reihenfolge
darf keine Kanäle vertauschen. Ein ungültiger späterer Nutzer oder Snapshot
veröffentlicht weiterhin kein gültiges Präfix. `bad_alloc` während der eigentlichen
Cacheübernahme besitzt unverändert keine vollständige Rollbackgarantie.

`benchmark_cache_ipc.py --copy-profile` erweitert denselben Frozen-REP-/
Produktionscachepfad. Die zwei vorab festgelegten Fälle sind 64 Töne/TX 2/RX 2/
ein Link und 1024 Töne/TX 8/RX 8/16 Links, jeweils drei unabhängige native und
Python-Prozesse. Synthetische Tensorgrößen bleiben Transportfixtures, keine
physikalischen HE-Ressourcengitter. `cache_hit` misst weiterhin den Matrixgetter;
`cache_hit_loss`, `cache_hit_delay`, `cache_hit_frequency` und `cache_hit_csi`
messen separat. Alle Getter verwenden dieselbe Round-Robin-Linkreihenfolge,
drei Warmups und 128 Messungen, einschließlich Ergebnisdestruktion.
Bitprüfung und komplette Linkvalidierung liegen außerhalb der Timer.

Jeder Prozess verlangt exakt
\[
 N_{\mathrm{hit}}=5(3+128)+1=656,\qquad N_{\mathrm{miss}}=2,
 \qquad r_{\mathrm{hit}}=656/658.
\]
Die Speicherprüfung benötigt fünf Getter je Link plus einen initialen Miss;
ihr Trefferverhältnis ist \(5L/(5L+1)\). Alle 27 tatsächlichen Kanal-RPCs
(26 bei t=0, einer nach dem inklusiven 5-ms-Endrand) sowie ein bestätigter
Close werden geprüft. Der vollständige Export enthält 4446 positive
Zeitstichproben und 162 Kanalrequests. `plan.json` steht vor der ersten Messung;
`run.json` trägt Quellen, Eingänge, Backend und den getrennten Ausführungs-/
Validierungsstatus. Ohne die Option bleiben bestehende CSV-/JSON-Schemata,
Messmatrix und historische Referenzen erhalten. Neue Messungen liegen lokal.

Vergleichszeiten verwenden die monotone Prozessuhr. Drei Prozessmediane pro
Operation werden nochmals durch ihren Median zusammengefasst; p95 und
Prozessstreuung stehen separat im Export. Die vorher festgelegten lokalen
Vergleichsziele sind mindestens 20 Prozent Medianverbesserung für den großen
Matrix- und skalaren Fall sowie höchstens zehn Prozent Regression beim kleinen
Default-Matrixfall. Nichterfüllung wird als solche berichtet. Diese Ziele
belegen weder universelle Latenzgrenzen noch Live-RT-/PHY-/GPU-Skalierung.

5.26 R3-VERTRAG FÜR MOBILITÄTSHISTORIEN UND NATIVEN STAGINGPOOL

R3: begrenzte Mobilitäts- und Pool-Speicherung (abgeschlossen).
`--mobility_history_limit H` begrenzt die gemeinsam gespeicherten Positions-/
Geschwindigkeitspaare pro Knoten (Default 1024; 0 oder `collect_history=False`
deaktiviert die Historie). Bei gleicher Zeit werden bestehende Arrays überschrieben;
nach Füllung werden die ältesten zwei Arrays mit `np.copyto` wiederverwendet.
Getter und Historienexport liefern unabhängige Werte; nicht mehr gespeicherte
RandomWalk-Zeitpunkte erzeugen `KeyError`. Direkte Historienarrays sind geliehene,
wiederverwendete Slots. Für N Knoten gilt für die NumPy-Nutzdaten
\(B_{hist} \le 48NH\): zwei float64-Vektoren mit je drei Komponenten.

Look-ahead schreibt keine Zukunftshistorie: `clone_for_prediction()` kopiert nur veränderlichen Bewegungszustand und lokale RNGs. Pro Empfänger und Epoche werden Position/Geschwindigkeit explizit gespeichert, unabhängig von H. Mit R Empfängern und D budgetgeprüften Epochen gilt \(B_{forecast}=48RD\), zusätzlich 48N Byte Positions-/Geschwindigkeitszustand der Klone. Nach jeder Vorhersage, auch nach Platzierungs-/CFR-Fehlern, werden die ursprünglichen Knoten, ihre Historien und RNGs wieder verwendet; das globale NumPy-RNG wird ebenfalls zurückgesetzt.
Bewegung bis zur tatsächlichen Anfragezeit wird vorher wie bisher ausgeführt; diese Aktualisierung ist keine Transaktion. Python-/RNG-/RT-/CIR-Overhead und RSS sind nicht in diesen Arraybytes enthalten. `mobility_storage_statistics()` trennt Historien, vorübergehende Epochen-/Klonzustände und Spitzen.

`SetBatchPoolLimit(bytes)` begrenzt unabhängig von Cachebudgets einen leeren nativen Stagingvektor (Default 1 MiB; 0 deaktiviert). Ganze Batches durchlaufen zuerst die bestehenden Budgetprüfungen, dann Datenvalidierung vor Übernahme. Ein RAII-Lease löscht alle Einträge auch bei Fehlern; nur leere strukturelle Kapazität wird wiederverwendet. Übergrößen umgehen den Pool, kleinere Grenzen geben übergroße freie Kapazität sofort frei. `GetPoolStatistics()` trennt Hits, Misses, echte Reserve-Vergrößerungen, Bypässe und freie Kapazität vom CSI-Cache. Es werden keine CFR-, Frequenz-, Protobuf- oder Transportpuffer gepoolt; ungecachte CSI-Belegung bleibt null. Dies ist kein RSS- oder Latenzversprechen.

6. HIGH-LEVEL-API-VERTRAG BIS UNTERPHASE 4.8

6.1 TYPEN, BESITZ UND DEFAULTS

SionnaMimoHelper ist ein wertbasierter, simulationsunabhängiger Konfigurationshalter. Sein Konstruktor erzeugt einen gültigen SISO-Vertrag; der Konstruktor mit SionnaMimoConfiguration und SetConfiguration prüfen eine vollständige Kopie, bevor sie den gespeicherten Zustand ersetzen. Eine fehlgeschlagene Änderung lässt den bisherigen Zustand damit unverändert. Alle Validierungsfehler sind std::invalid_argument mit einem SionnaMimoHelper-Präfix und benötigen weder Simulator, Node noch eine ZMQ-Verbindung.

SionnaMimoArrayConfiguration enthält Zeilen, Spalten, vertikalen und horizontalen Abstand in Wellenlängen, Elementmuster und Polarisation. Die repräsentierten Sionna-Muster sind isotrop, dipole, half-wave dipole und TR 38.901. Vertikale und horizontale Polarisation erzeugen je Element einen, duale Polarisation zwei Antennenports. Für Zeilenzahl \(R\), Spaltenzahl \(C\) und Polarisationsfaktor \(p\in\{1,2\}\) gilt deshalb

\[
  N_\mathrm{array} = R C p.
\]

SionnaMimoOrientation speichert Azimut, Elevation und Rollwinkel in Grad. Die SionnaMimoConfiguration führt beide Arrays, den Modus, TX-/RX-Antennenzahl, maximale NSS, Empfänger, Normalisierung, Empfängerrauschzahl, Effective-SINR-Modell, explizite EESM-Betas und eine Map von ns-3-Node-ID auf Orientierung sowie optional MimoPrecodingMatrix oder eine explizite SionnaMimoBeamformingConfiguration zusammen. Die festgeschriebenen Defaults sind:

- Modus SISO, eine TX-Antenne, eine RX-Antenne und maximale NSS eins;
- TX-Array 1 x 1, Abstände 0,5 Wellenlängen, TR-38.901-Muster, vertikale Polarisation;
- RX-Array 1 x 1, Abstände 0,5 Wellenlängen, Dipolmuster, vertikale Polarisation;
- keine explizite knotenspezifische Orientierung; der API-Lesezugriff liefert dafür neutrale Nullwinkel, während Apply kein IPC-Feld erzeugt und der

Server deshalb für alte und neue 1x1-Aufrufe exakt seine bisherige interne Orientierung [0, -180, 0] beibehält;
- linearer MMSE-Empfänger und gemeinsame Unit-Average-Power-Normalisierung über alle Subcarrier und Antennenpaare gemäß Abschnitt 4.3;
- 7 dB Empfängerrauschzahl entsprechend dem ns-3.40-WifiPhy-Default;
- EESM als HE-Effective-SINR-Modell, aber weiterhin keine impliziten MCS-Betas;
- weder ein fester Precoder noch Beamformingoptionen oder eine ausgewählte Precodingmomentaufnahme. Dadurch nehmen alle bisherigen 1x1- und offenen Mehrstromkonfigurationen weiterhin exakt den Phase-2-Pfad.

SetNodeOrientation, ClearNodeOrientation und GetNodeOrientation verwalten die knotenbezogenen Winkel. SetEesmBeta, HasEesmBeta und GetEesmBeta verwalten explizite lineare Betas für HE MCS 0 bis 11. SetPrecodingMatrix, ClearPrecodingMatrix, HasPrecodingMatrix und GetPrecodingMatrix verwalten den optionalen festen Precoder atomar. Die gespeicherte Konfiguration bleibt ein wertbasierter Vertrag und verändert Simulationsobjekte erst mit Apply.

6.2 UNTERSTÜTZTE BETRIEBSARTEN

GetSupportedModes und IsModeSupported melden vier akzeptierte Modi:

1. SISO mit genau einer TX- und einer RX-Antenne sowie NSS eins;
2. SU_MIMO_SPATIAL_MULTIPLEXING mit linearem MMSE-Empfänger. Ohne W[k] bleibt genau ein Stream je TX-Antenne aktiv; mit explizitem festem W[k] bestimmt dessen Spaltenzahl N_s;
3. SU_BEAMFORMING mit explizit gewähltem MRT oder SVD. SelectPrecoding erzeugt aus der übergebenen CSI-Matrix genau eine unveränderliche Auswahl, die Apply anschließend über den vorhandenen festen-W[k]-Pfad installiert.
4. MU_MIMO für den Helper-installierten HE-Downlink mit einem Stream und einer Empfangsantenne je Station, einer Vollband-242-Ton-RU und expliziter Laufzeitkonfiguration.

Festes benutzergegebenes Precoding bleibt bewusst SU_MIMO_SPATIAL_MULTIPLEXING zugeordnet, weil es keine aus H[k] abgeleitete Strahlformung implementiert. MRT ist weiterhin auf perfektes aktuelles 2x1-MISO-CSI und einen Stream begrenzt. SVD unterstützt Gleichleistung, quantisiertes HE-/EHT-SU-Feedback und explizites CSI-Alter. Räumliche Wasserfüllung ist mit unquantisiertem aktuellem CSI verfügbar; ihre Kombination mit Feedbackquantisierung oder Alterung bleibt aufgrund des semi-unitären Feedbackvertrags unzulässig. MU_MIMO wird nur ohne explizite `dlMu`-Konfiguration oder außerhalb von `InstallDownlink` eindeutig abgelehnt. Ebenso werden der repräsentierte Zero-Forcing-Empfänger, eine unnormalisierte CFR und Mutual Information als PER-wirksames Effective-SINR-Modell abgelehnt. Die MI-Kompression aus Abschnitt 5.4 bleibt eine Diagnosemetrik.

6.3 ZENTRALE KONSISTENZPRÜFUNG

Validate erzwingt gemeinsam die Grenzen von Sionna, ns-3-Wi-Fi und dem Phase-1-Abstraktionsmodell:

- Arraydimensionen und Abstände sind endlich und positiv; die abgeleitete Portzahl stimmt exakt mit numTxAntennas beziehungsweise numRxAntennas überein;
- TX-/RX-Antennenzahlen und maxNss liegen im ns-3.40-Bereich 1 bis 8;
- SISO besitzt exakt die Dimensionen 1 x 1 und maxNss eins;
- SU-MIMO ohne W[k] erfüllt weiterhin $2\leq N_\mathrm{tx}\leq N_\mathrm{rx}\leq8$ und $N_\mathrm{ss,max}=N_\mathrm{tx}$;
- SU-MIMO mit W[k] erfüllt

\(1\leq N_s=N_\mathrm{ss,max}\leq
\min(N_\mathrm{tx},N_\mathrm{rx})\). TX- und Streamdimension stimmen
exakt mit der Matrix überein; Subcarrierzahl, Koeffizienten und
Leistungsnorm sind bereits Bestandteil des unveränderlichen
MimoPrecodingMatrix-Vertrags;
- SU_BEAMFORMING besitzt mindestens zwei TX-Antennen, kein gleichzeitig
benutzergegebenes W[k] und eine explizite MRT- oder SVD-Konfiguration;
- MRT verlangt genau eine RX-Antenne, einen Stream, unquantisiertes aktuelles
CSI und unveränderte SVD-Standardoptionen;
- SVD prüft endliche positive Referenz-SNR und Rauschleistung, einen relativen
Rangschwellwert in (0,1), HE/EHT und unquantisiertes, grobes oder feines
SU-Codebook sowie nichtnegative CSI-Zeiten. Wasserfüllung schließt
Quantisierung und positives CSI-Alter aus;
- jede Änderung der Konfiguration oder einer Knotenorientierung verwirft eine
zuvor ausgewählte Momentaufnahme, damit Apply kein veraltetes W[k] nutzt;
- Azimut und Rollwinkel liegen endlich in [-180, 180] Grad, Elevation endlich
in [-90, 90] Grad;
- die Rauschzahl ist endlich und nicht negativ;
- jedes vorhandene EESM-Beta gehört zu HE MCS 0 bis 11 und ist endlich und
positiv.

ValidateForHeMcs ergänzt die nutzungsbezogene Vollständigkeitsprüfung: Für
Open-Loop-SU-MIMO und SU_BEAMFORMING muss das konkret verwendete MCS ein
explizites Beta besitzen. SISO verlangt wegen des unveränderten skalaren
Phase-1-Pfads weiterhin kein Beta. Damit werden keine unbelegten
Kalibrierwerte eingeführt.

6.4 GEMEINSAMER INTEGRATIONSAUFRUF

Apply erhält einen vorhandenen SionnaHelper, getrennte
SpectrumWifiPhyHelper-Instanzen für Sender und Empfänger sowie den
SionnaSpectrumPropagationLossModel-Adapter. Die Trennung der beiden
PHY-Helper hält auch \(N_\mathrm{tx}\ne N_\mathrm{rx}\) darstellbar. Apply
validiert zuerst erneut den vollständigen gespeicherten Vertrag und verändert
danach ausschließlich die explizit übergebenen Objekte:

```
    SionnaMimoHelper mimo(configuration);
    mimo.Apply(sionna, txPhy, rxPhy, spectrumModel);
```

Die Zuordnung ist eindeutig:

- txArray und rxArray werden auf die bestehenden
ConfigureTransmitArray-/ConfigureReceiveArray-Aufrufe abgebildet;
- numTxAntennas beziehungsweise numRxAntennas setzen das Antennenzahlattribut
des jeweiligen SpectrumWifiPhyHelper;
- maxNss setzt auf beiden Endpunkten MaxSupportedTxSpatialStreams und
MaxSupportedRxSpatialStreams;
- receiverNoiseFigureDb setzt sowohl WifiPhy::RxNoiseFigure als auch den
expliziten Rauschzahlwert des MIMO-Spectrumadapters;
- die vollständige EESM-Map ersetzt atomar die bisherige Adapterkonfiguration,
sodass beim erneuten Anwenden keine veralteten MCS-Betas zurückbleiben;
- eine durch SelectPrecoding gespeicherte MRT-/SVD-Auswahl ersetzt den festen
Adapter-Precoder vollständig und hat ausschließlich im Modus SU_BEAMFORMING
Vorrang;
- precodingMatrix ersetzt den festen Adapter-Precoder vollständig. Fehlt der
optionale Wert in einem Nicht-Beamformingmodus, wird ein zuvor gesetztes
W[k] gelöscht und der Phase-2-Pfad wiederhergestellt.

Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung und EESM
bleiben unverändert. Festes oder ausgewähltes W[k] wählt ausschließlich die
bereits vorhandene \(H[k]W[k]\)-Überladung. Eine

abweichende Empfänger-, Normalisierungs- oder Effective-SINR-Auswahl wird weiterhin vor Apply abgelehnt.

6.5 KNOTENORIENTIERUNG IM IPC- UND SERVERPFAD

NodeInfo besitzt das neue optionale Feld orientation mit Azimut, Elevation und Rollwinkel in Grad. SionnaHelper trägt nur explizit konfigurierte Knotenorientierungen ein. Der Python-Server prüft endliche Werte und dieselben Winkelgrenzen wie die High-Level-API, konvertiert Grad mit

$$
\boldsymbol\theta_\mathrm{rad}
= \frac{\pi}{180}\boldsymbol\theta_\mathrm{deg}
$$

und verwendet die drei Eulerwinkel beim Anlegen jedes Transmitter- und Receiver-Objekts im klassischen sowie im Look-ahead-Pfad. Das ausgelagerte PopulateMimoConfiguration schreibt Array- und Orientierungsfelder sowohl für Start als auch für den bytegenauen Integrationstest.

Fehlt orientation, verwendet der Server weiterhin exakt die bisherige Konstante [0, -180, 0]. Der Wert wird bewusst nicht als neue API-Orientierung interpretiert, sondern nur zur vollständigen Verhaltenskompatibilität alter Nachrichten beibehalten. Ein alter SionnaHelper-Aufruf konfiguriert weder Array- noch Orientierungsfelder und erzeugt deshalb weiterhin die bisherigen 1x1-Arrays und die bisherige Serverausrichtung.

6.6 BENUTZBARKEIT, BEISPIELE UND MIGRATION

Für eine 1x1-Verbindung reicht der Defaultvertrag. Er behält SISO, je eine TX-/RX-Antenne, NSS eins, 7 dB Rauschzahl, die bisherigen Arraymuster und den skalaren Legacy-Empfangspfad bei. Deshalb ist für diesen Pfad kein EESM-Beta erforderlich:

```
    SionnaMimoConfiguration configuration;
    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Das kleinste Vollrang-2x2-Beispiel verwendet zwei vertikale Elemente je Array, zwei offene Spatial Streams und ein vom Anwender kalibriertes Beta für den gewählten HE-MCS:

```
    SionnaMimoConfiguration configuration;
    configuration.mode = SionnaMimoMode::SU_MIMO_SPATIAL_MULTIPLEXING;
    configuration.txArray.numColumns = 2;
    configuration.rxArray.numColumns = 2;
    configuration.numTxAntennas = 2;
    configuration.numRxAntennas = 2;
    configuration.maxNss = 2;
    configuration.eesmBetaLinear.emplace(mcs, betaFromCalibration);

    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Eine feste 2x1-Streamauswahl auf einem 2x2-Kanal ergänzt ausschließlich maxNss und den normalisierten Precoder:

```
    configuration.maxNss = 1;
    configuration.precodingMatrix = MimoPrecodingMatrix::FrequencyFlat(
    numSubcarriers, 2, 1, {{1.0, 0.0}, {0.0, 0.0}});
```

```
    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Für frequenzselektives Precoding wird stattdessen der MimoPrecodingMatrix-Konstruktor mit $N_\mathrm{sc}N_\mathrm{tx}N_s$ Koeffizienten in sc/tx/stream-Reihenfolge verwendet. Beide Varianten durchlaufen dieselbe Validierung und denselben Adapterpfad.

ApplyForHeMcs führt zuerst ValidateForHeMcs aus. Ein ungültiger MCS oder ein fehlendes SU-MIMO-Beta bricht daher mit std::invalid_argument und einem SionnaMimoHelper-Präfix ab, bevor SionnaHelper, die beiden PHY-Helper oder der Spectrumadapter verändert werden. Apply bleibt für vorhandenen Code erhalten, wenn der verwendete MCS an dieser Stelle noch nicht bekannt ist.

Das Programm ns3_wifi_sumimo_minimal.cc enthält die vollständigen helperkonfigurierten 1x1-, offenen Vollrang-2x2-, Identitäts-Precoding- und 2x1-Streamauswahlszenarien. Es verbindet sie über eine feste normalisierte CFR mit dem echten UDP-/SpectrumWifiPhy-Pfad und behält zusätzlich den Phase-1-Rang-1-Vergleich:

```
    ./ns3 run "ns3_wifi_sumimo_minimal \
    --mcs=5 --referenceSnrDb=20 --packetSize=1200 \
    --eesmBetaLinear=<kalibrierter-linearer-Wert>"
```

Der feste Kanal ersetzt in diesem reproduzierbaren Beispiel nur die Kanalquelle. Array, Antennenzahlen, NSS, Rauschzahl und EESM werden nicht am Adapter oder PHY gesetzt, sondern ausschließlich mit SionnaMimoHelper.

Für die Migration eines bestehenden Programms werden die manuellen Aufrufe ConfigureTransmitArray, ConfigureReceiveArray, die PHY-Attribute Antennas, MaxSupportedTxSpatialStreams, MaxSupportedRxSpatialStreams und RxNoiseFigure sowie SetReceiverNoiseFigureDb/SetEesmBeta am Spectrumadapter durch genau eine SionnaMimoConfiguration und ApplyForHeMcs ersetzt. SionnaHelper::Configure, SetMode, Start und Destroy, der PropagationCache sowie die normale SpectrumChannel-Verdrahtung bleiben unverändert. Ein Programm, das weiterhin nur SionnaHelper verwendet, nimmt damit unverändert den bisherigen 1x1-Defaultpfad.

Funktional unterstützt der Stand bis Unterphase 3.5 SISO, offenen SU-MIMO, festes benutzergegebenes Precoding und explizites SU-Beamforming mit MRT oder SVD. Für festes W[k] gilt weiterhin $1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$. Die MRT-Fabrik erzeugt für eine explizit übergebene perfekte MISO-CFR ein normiertes Ein-Stream-W[k]. Die SVD-Fabrik erzeugt aus einer allgemeinen perfekten CFR ein festes W[k] für explizit gewählte, numerisch tragfähige Eigenmoden; gleiche Leistung bleibt Standard und räumliche Wasserfüllung ist eine explizite Option. Beide Fabriken verwenden anschließend unverändert den festen Precodingpfad. Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung der CFR und EESM bleiben unverändert. MimoCompressedBeamformingFeedback, MimoFeedbackReconstruction und MimoFeedbackDiagnostics bilden nun den getrennten Feedbackpfad. MimoCsiAgingModel liefert einen festen Precoder-Schnappschuss mit explizitem Zeitbezug und zwingt für die Linkberechnung eine separate Übertragungskanal-Matrix. Der Helper kapselt diese wertbasierten Ergebnisse unverändert; sie bleiben auch ohne Simulator testbar. Typische eindeutige Fehler sind:

```
  - SionnaMimoHelper: open-loop SU-MIMO uses one stream per TX antenna bei
  widersprüchlichem maxNss;
  - SionnaMimoHelper: SU beamforming requires an explicit MRT or SVD
  configuration bei fehlender Auswahlmethode;
```

- SionnaMimoHelper: water-filling cannot be combined with quantized feedback or CSI aging für nicht darstellbare Vertragskombinationen;
- SionnaMimoHelper: SU beamforming must select a channel-dependent precoder before Apply bei fehlendem CSI-Auswahlschritt;
- SionnaMimoHelper: ... requires an explicit EESM beta for HE MCS ... bei fehlender Kalibrierung für Open Loop oder Beamforming;
- SionnaMimoHelper: maximum NSS must equal the fixed-precoding stream dimension bei widersprüchlicher Helper-Konfiguration;
- WifiTxVector NSS must equal the transmitted stream dimension, falls WifiTxVector und W-Spalten beziehungsweise der alte offene TX-Pfad widersprüchlich sind;
- MimoPrecodingMatrix: MRT requires exactly one RX antenna beziehungsweise ... a non-zero channel on every subcarrier für mehrdeutige oder nicht normierbare MRT-Eingaben;
- MimoPrecodingMatrix: SVD stream count must be in ... beziehungsweise ... requested ... streams but subcarrier ... has numerical rank ... für ungültige Streamwahl oder Rangverlust sowie eindeutige Meldungen für nicht endliche CFR-, SNR-, Rausch- und Allokationseingaben.
- MimoCsiFeedback: SU compressed feedback requires a semi-unitary equal-power precoder beziehungsweise ... angle index exceeds the selected codebook width für nicht darstellbare Leistung oder ungültige Indizes;
- MimoCsiFeedback: CSI timestamp does not match update interval and feedback delay beziehungsweise ... no CSI feedback is available ... für einen off-grid oder noch nicht verfügbaren Schätzzeitpunkt.

6.7 KANALABHÄNGIGE AUSWAHL, DIAGNOSE UND PHASENBEISPIEL

SionnaMimoBeamformingConfiguration bündelt die explizite Methode MRT oder SVD, die unveränderten MimoSvdPrecodingOptions, MimoFeedbackConfiguration und MimoCsiTimingConfiguration. SionnaMimoPrecodingSelection besitzt den ausgewählten festen Precoder, CSI- und Sendezeitpunkt, CSI-Alter und, sofern vorhanden, die ursprünglichen SVD- und Feedback-/Alterungsergebnisse. Der Helper berechnet oder interpretiert diese Diagnosen nicht neu.

```
    configuration.mode = SionnaMimoMode::SU_BEAMFORMING;
    configuration.maxNss = 1;
    configuration.beamforming = SionnaMimoBeamformingConfiguration{};
    configuration.beamforming->method = SionnaMimoBeamformingMethod::SVD;
    configuration.beamforming->feedback.codebook = MimoFeedbackCodebook::SU_FINE;
    configuration.beamforming->csiTiming.updateInterval = MilliSeconds(5);
    configuration.beamforming->csiTiming.feedbackDelay = MilliSeconds(2);

    SionnaMimoHelper mimo(configuration);
    const auto& selection = mimo.SelectPrecoding(
    channelEstimateAtCsiTime,
    txNodeId, rxNodeId,
    csiTimestamp, transmissionTimestamp);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

SelectPrecoding validiert Konfiguration, CSI-Dimensionen und Zeitraster, erstellt das Ergebnis zunächst als lokalen Kandidaten und ersetzt den gespeicherten Zustand erst nach vollständigem Erfolg. SetConfiguration sowie Änderungen einer Knotenorientierung verwerfen die Auswahl. Apply lehnt SU_BEAMFORMING ohne vorherige Auswahl ab und installiert sonst ausschließlich selection.GetPrecodingMatrix() durch SetPrecodingMatrix. Der Adapter, das Signalmodell, die Leistungsnorm \(\Vert W[k]\Vert_F^2=N_s\) und die 1x1-/Open-Loop-/Festprecodingpfade bleiben unverändert.

Das neue Programm ns3_wifi_sumimo_precoding.cc führt vier getrennte UDP-/SpectrumWifiPhy-CPU-Läufe aus. Open Loop und SVD verwenden denselben

normalisierten 2x2-Kanal; fester Precoder und MRT verwenden denselben
2x1-MISO-Kanal, sodass der MRT-Gewinn direkt gegen die erste feste Antenne
vergleichbar ist. Alle Fälle verwenden dieselbe Gesamtleistung \(P\); die
unterschiedlichen Frobeniusnormen \(\sqrt{N_s}\) kompensiert das unveränderte
Signalmodell durch \(P/N_s\).

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --referenceSnrDb=16 --packetSize=1200 \
    --eesmBetaLinear=2"
```

Der Referenzlauf liefert für festes W[k] 16,000000 dB und 24,931200 Mbit/s,
für MRT den analytischen Gewinn auf 19,010300 dB und 68,582400 Mbit/s und für
Ein-Stream-SVD 21,563025 dB und 68,697600 Mbit/s. Die Precodingnorm bleibt in
allen Ein-Stream-Fällen eins; Open Loop verwendet bei zwei Streams
\(\sqrt 2\). Das Beispiel bricht ab, falls die Gesamtleistungsidentität, die
analytische MRT-SINR oder der ideale SVD-Gewinn verletzt wird.

Quantisierung und Alterung sind keine versteckten Defaults, sondern
ausdrückliche Optionen:

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --eesmBetaLinear=2 --quantization=coarse \
    --csiUpdateIntervalMs=5 --feedbackDelayMs=2 \
    --transmissionTimeMs=13 --agingRotationDegrees=25"
```

quantization akzeptiert none, coarse und fine.
csiUpdateIntervalMs, feedbackDelayMs und transmissionTimeMs bestimmen
den nach Abschnitt 5.12 zulässigen CSI-Zeitstempel;
agingRotationDegrees verändert ausschließlich den aktuellen Sendekanal.
Wasserfüllung wird separat und weiterhin nur mit unquantisiertem aktuellem
CSI aktiviert:

```
    ./ns3 run "ns3_wifi_sumimo_precoding \
    --mcs=5 --eesmBetaLinear=2 --svdStreams=2 --waterFilling=1"
```

Jede Ausgabezeile besteht aus whitespace-getrennten schlüssel=wert-Feldern.
Die configuration-Zeile enthält Referenz-SNR, konstante Gesamtleistung,
Quantisierung, Wasserfüllschalter, Aktualisierungsintervall,
Feedbackverzögerung, CSI-Alter, kontrollierte Kanalrotation und das explizite
EESM-Beta. Jede scenario-Zeile enthält mindestens:

  - throughput_mbps, expected_throughput_mbps und per;
  - stream_sinr_db als kommaseparierte Streamliste sowie
  effective_sinr_db;
  - channel_rank, precoding_norm und power_fractions;
  - csi_age_ns, wobei -1 einen nicht-CSI-basierten Modus bezeichnet;
  - feedback_bits, chordal_error und rms_coefficient_error.

Eine abschließende validation=PASS-Zeile macht den Prozessstatus zusätzlich
maschinenlesbar. Die ExampleAsTest-Referenzdateien fixieren alle Felder für
den idealen, quantisiert-gealterten und Wasserfüllungsfall.

Die Migration ist additiv: Bestehende SISO-, Open-Loop- und
Festprecodingprogramme ändern weder Modus noch Aufrufreihenfolge. Nur
kanalabhängige Programme wählen SU_BEAMFORMING, füllen beamforming, rufen
nach Vorliegen einer expliziten CSI-Matrix einmal SelectPrecoding auf und
verwenden danach das bestehende ApplyForHeMcs. ClearSelectedPrecoding
ermöglicht eine bewusste Neuwahl. Es gibt weiterhin keinen automatischen
Simulator-Timer, keine implizite CSI-Abfrage und keinen MAC-Feedbacktransport;
diese Grenzen verhindern, dass das explizite Forschungsmodell eine
vollständige 802.11-Soundingprozedur vortäuscht.

7. VALIDIERUNG BIS ZUM R3-ABSCHLUSS

Die native Suite sionna enthält die schnellen API-, Numerik- und Adapterfälle:

1. API-Defaulttest für den vollständigen legacy-kompatiblen SISO-Vertrag, leere EESM-Kalibrierung, fehlendes W[k] und alle vier gemeldeten Betriebsarten.
2. Gültiger 2x2-Vertrag mit dual polarisiertem TX-Element, knotenspezifischer Orientierung, explizitem Beta, Identitäts-Precoding, frequenzselektiver 2x1-Streamauswahl sowie zulässigem 2-TX-/1-RX-/1-Stream-Grenzfall.
3. Zentrale Negativmatrix für nicht endliche, unvollständige, widersprüchliche oder nicht unterstützte API-Konfigurationen einschließlich Precoding-TX-/NSS-Dimension, unzulässiger Beamforming-/Feedback-/Alterungs-Kombinationen, MU-MIMO und fehlendem MCS-Beta.
4. Manuell und über SionnaMimoHelper konfigurierte 1x1-Integration mit bytegleicher Array-/Orientierungs-IPC-Nachricht, identischer installierter SpectrumWifiPhy-Antennen-/NSS-Konfiguration, gleicher Rauschzahl und EESM-Kalibrierung sowie exakt gleichen MMSE-, EESM- und PER-Ergebnissen.
5. Derselbe Integrationsvergleich für eine vollständige offene 2x2-Konfiguration ohne W[k].
6. Manuell gesetztes und durch SionnaMimoHelper angewendetes frequenzselektives komplexes 2x1-W[k] liefern dieselben Koeffizienten, installierten NSS, Post-MMSE-SINRs, EESM-SINR und PER.
7. Layouttest eines 2 x 2 x 2-CFR-Tensors mit eindeutig nummerierten Koeffizienten.
8. Precoding-Vertragstest für sc/tx/stream-Reihenfolge, explizite Frequenzflach-Expansion, Frobenius-Leistung sowie Dimensions-, Endlichkeits- und Normierungsfehler.
9. Reversible dB-/Linear-Umrechnung einschließlich 10 dB gleich Faktor zehn.
10. Link-Budget-Test für Sendeleistung, Pfadverlust, B, T und NF sowie die erwartete Halbierung der SNR bei Verdopplung von B oder T.
11. Flacher orthogonaler 2x2-Kanal: EESM erhält identische Tone-/Stream-SINRs unabhängig vom positiven Test-Beta.
12. Frequenzselektiver SISO-Kanal: EESM stimmt mit der analytischen Zweitonformel für das konfigurierte Beta überein.
13. SISO-Parität zur direkten ns-3.40-ErrorRateModel-Abfrage für HE MCS 0, 3, 7 und 11, neun SNR-Punkte von -5 bis 35 dB sowie 32, 400 und 1458 Byte. Das sind 108 Kombinationen; MCS 11 prüft auch den TableBased-Fallback auf Yans.
14. Monotone PER auf einem normierten frequenzselektiven Vier-Ton-Kanal für dieselben vier MCS und drei Paketgrößen bei 26 SNR-Punkten von -10 bis 40 dB. Die 312 geprüften Werte bilden je Konfiguration eine nicht steigende Folge mit einem echten PER-Abfall.
15. Exakte Reproduzierbarkeit von Post-MMSE-SINR, EESM-SINR und PER über 20 Wiederholungen eines komplexen frequenzselektiven 2x2-Kanals.
16. SISO-Test mit $h=2+j$ und $\rho=10$; erwartet wird $\gamma=50$.
17. Orthogonaler 2x2-Kanal $\mathbf H=\mathbf I$ und $\rho=20$; jeder Stream erhält $\gamma=10$ und $\eta=2\log_2(11)$.
18. Zwei diagonale Subcarrier diag(1,2) und diag(2,1) bei $\rho=8$; erwartet werden (4,16), (16,4) und die analytische MI-Kompression.
19. Fester-Precoding-Referenztest für komplexes SISO, Identitätsparität, indexgenaues komplexes $H[k]W[k]$, identische MMSE-Ausgabe zum vorberechneten effektiven Kanal und 2x2 mit einem ausgewählten Stream.
20. Ein-Stream-MRT/MRC-Test für frequenzselektives komplexes 2x1-MISO, Einheitsleistung von W[k], analytischen Arraygewinn $\rho\Vert h\Vert_2^2$, gemeinsame Phasendrehung, äquivalentes 1x2-SIMO-MRC, exakte 1x1-Parität sowie die eindeutige Ablehnung von Nullkanal und allgemeinem 2x2-MIMO.
21. Deterministischer SVD-/Leistungsallokationstest für diagonale, unitär gedrehte, frequenzselektive, rangdefiziente und gleichsinguläre Kanäle.

   Er prüft explizite Stream-/Ranggrenzen, den relativen Rangschwellwert,
   Phasen-/Entartungskonvention, 1x1-Parität und analytische Eigenmode-SINRs.
   Zwei Zweimodenfälle unterhalb und oberhalb der Wasserfüllschwelle prüfen
   aktive Moden, endliche nichtnegative Anteile, exakte Leistungserhaltung,
   Wasserstand, KKT-Bedingungen und mindestens die Gleichleistungsrate.
22. Mindestkanalrang eins beziehungsweise zwei für Rang-1- und orthogonale
    2x2-Kanäle über alle Subcarrier.
23. Spectrumadapter-Test für exaktes Rauschleistungs- und AWGN-Diversity-
    Compensation-Mapping sowie SISO-, Identitäts- und 2x1-Precoding ohne
    doppelte Anwendung von Rauschen, Leistung oder Arraygewinn.
24. Deserialisierung einer alten Nachricht ohne Dimensionsfelder als
    \(N_\mathrm{sc}\times1\times1\).
25. Protocol-Buffer-Rundlauf einer synthetischen 2 x 2 x 2-CFR mit den
    komplexen Werten 1-j bis 8-8j und indexgenauer Prüfung.

26. Helperauswahl für exaktes MRT, quantisierte gleichleistungs-SVD mit
    vollständiger Zustandsidentität und drei Millisekunden CSI-Alter sowie
    räumliche Wasserfüllung. Der Fall vergleicht jedes W[k] mit der bereits
    abgeschlossenen Fabrik, prüft Feedbackbits, Leistungsanteile und
    Gesamtleistung, weist nach, dass Apply exakt dieselben Koeffizienten am
    Adapter installiert, und verwirft die Auswahl nach einer
    Orientierungsänderung.
27. Protocol-Buffer-Rundlauf eines absichtlich umgekehrt eingegebenen
    Zweinutzer-Snapshots. Er prüft die sortierte Nutzerfolge 3, 9, gerichtete
    TX-/RX-Identität, gemeinsamen Zeitstempel und Frequenzraster,
    sc/rx/tx-Indizes sowie heterogene 1x1- und 1x2-RX-Arrays bei gemeinsamem
    Zweiantennensender.
28. Atomare Negativmatrix für doppelte Nutzer, gemischte Zeitstempel und
    Richtungen, abweichende Frequenzraster sowie eine RX-Geometrie, die nicht
    zur CFR-Dimension passt. Jeder Fall wird vor Cacheverwendung als
    `invalid_argument` abgelehnt.
29. Fester-Downlink-MU-Referenzfall mit expliziter Nutzer-/Streamzuordnung,
    exakter Gesamtleistung bei gleichen und ungleichen Spaltengewichten,
    Einzelnutzerparität zu Phase 3 einschließlich 1x1, null Interferenz für
    orthogonale Nutzer und analytischer gewünschter Leistung,
    Inter-User-Interferenz sowie SINR für korrelierte Kanäle.
30. Nutzerspezifischer HE-Vertrag mit zwei MCS, EESM-Beta, Link Budgets und
    Paketgrößen. Beide effektiven SINRs und Paketwahrscheinlichkeiten werden
    gegen direkte Aufrufe des bestehenden EESM- und ErrorRateModel-Vertrags
    geprüft.
31. Atomare Phase-4.2-Negativmatrix für doppelte oder lückenhafte
    Nutzer-/Streamzuordnung, fehlende oder unbekannte Nutzer, Uplinkrichtung,
    abweichendes Frequenzraster, fehlende oder nicht endliche SNR,
    unvollständige HE-Eingaben, NSS-Widerspruch und Paketgröße null. Ein danach
    ausgeführter gültiger Referenzfall weist den unveränderten Vertrag nach.
32. Deterministische ZF-/RZF-Auswahl für orthogonale, korrelierte und
    schlecht konditionierte Kanäle. Der Fall prüft stabile Nutzer-/Spaltenordnung
    unabhängig von der Anforderungsreihenfolge, koeffizientengleiche
    Wiederholung, ZF-Interferenzunterdrückung, endliches explizites RZF,
    Gesamtleistung, Einzelnutzer-MRT-/Phase-4.2-Parität und den 1x1-Grenzfall.
33. Atomare ZF-/RZF-Negativmatrix für mehrere Streams je Nutzer, fehlende
    Nutzer, ungültige Regularisierung oder Rangtoleranz, Überlast, Rangverlust,
    nicht endliche CFR und den noch nicht definierten Mehrantennenempfänger.
    Ein gültiger Aufruf vor und nach allen Ablehnungen liefert identische
    Koeffizienten.

34. Nativer HE-DL-MU-Adapterfall mit umgekehrter Nutzerreihenfolge und von
    Nutzer-IDs verschiedenen STA-IDs. Er prüft zwei MCS, NSS eins und zwei,
    gleiche RU mit analytischer Interferenz und SINR \(5/6\), verschiedene
    RUs mit null Interferenz und SINR 10, nutzerweise PER, Link Budget,

    äquivalente PSD-Leistung sowie den 1x1-Grenzfall.
35. Atomare Adapter-Negativmatrix für unbekannte und nicht adressierte Nutzer,
    falsche Richtung und Präambel, EHT, TX-/RX-/NSS-Dimensionsfehler sowie
    unbekannte Ergebnisnutzer. Ein installierter gültiger MU-Zustand und der
    unveränderte SU-SINR-/PER-Pfad werden vor und nach Ablehnungen verglichen.
36. Echter `HePpdu`-Adapterfall mit AP- und zwei STA-MACs,
    `WIFI_PREAMBLE_HE_MU`, gemeinsamem 2x2-Precoder, unterschiedlichen
    Pfadverlusten und positivem Empfangsgewinn. Die je STA zurückgegebene
    skalare RU-PSD wird mit der zuvor gemeinsam berechneten Matrixreferenz
    verglichen.
37. Korrigierter numerischer RU-Vertrag mit gruppenweiser Precoderleistung,
    ungleichen Spaltennormen und Leistungsbudgets, ausgewählten EESM-Tönen,
    Invarianz außerhalb der Auswahl sowie Ablehnung ungültiger Tonlisten.
38. Unabhängiger Runtime-RU-Fall für 242-, 106- und mittlere 26-Ton-RUs. Er
    prüft Nulltöne, Frequenzzuordnung, fremde RU-Energie mit dem tatsächlichen
    Precoder, das 4:1-Präambelraster und normale Mithörer-/Fremdsenderpfade.
39. Echter Channel-/PHY-Lauf mit zwei adressierten STAs und einem Mithörer.
    Orthogonale Nutzer dekodieren mit analytischer SINR; korrelierte Nutzer
    erreichen die Nutzdatenphase und scheitern. Der Mithörer hält CCA busy,
    erhält aber kein PSDU.

Die eigenständige native Suite `ul-mu-mimo` ergänzt den analytischen
Unterphase-4.7-Fall. Sie prüft den Einzelnutzervergleich mit dem bestehenden
SU-MMSE-/MRC-Kern, orthogonale Nutzer, ZF-Rangverlust, rangdefiniertes MMSE,
den Nah-Fern-Fall mit Referenz-SNRs 100 und 1 sowie getrennte HE-EESM-/PER-
Abbildung für verschiedene MCS und Paketgrößen. ZF liefert im korrelierten
Fall 75 und 0,75; MMSE liefert 87,5 und 0,752475247524752.

Die eigenständige native Suite `he-ul-mu-mimo-phy-adapter` ergänzt zwei
Unterphase-4.8-Fälle mit echten `CtrlTriggerHeader`- und `HePpdu`-Objekten.
Der positive Fall prüft umgekehrte Empfangsreihenfolge, zwei MCS, gemeinsame
242-Ton-RU, analytische orthogonale MMSE-SINRs 4 und 9, PER und äquivalente
Empfangsleistung sowie die orthogonale Trennung zweier 106-Ton-RUs. Der
Teilnahmefall weist nach, dass ein fehlender und ein um eine Nanosekunde
verspäteter Nutzer nicht detektiert und getrennt gemeldet werden. Eine
fremde Trigger-UID wird atomar abgelehnt.

Die eigenständige native Suite `sionna-mimo-csi-feedback` ergänzt zwei Fälle
für Unterphase 3.4. Der Feedbackfall prüft HE und EHT, beide SU-Codebücher,
Bitbreiten und Payloadgrößen, exakt rekonstruierte Grenzindizes, deterministische
Wiederholung, den allgemeinen 3x2-Mehrstromfall, Leistungsrenormierung,
Phasen-/Subraumfehler sowie die Ablehnung nicht semi-unitärer Wasserfüllungs-
Precoder und ungültiger Indizes. Der Alterungsfall prüft Nullalter, getrennte
CSI- und Sende-CFR, Rasterung mit Aktualisierungsintervall und Rückmeldedauer,
statische und orthogonal gedrehte Kanäle, den daraus folgenden SINR-/PER-Abfall,
frische CSI, vollständige Zustandsidentität und den unveränderten 1x1-Fall.

Die sieben reinen Python-Vertragstests prüfen Standard- und explizite
Arraykonfiguration, bytegleiche 1x1-Serialisierung, die sc/rx/tx-Reihenfolge
einer synthetischen 2x2-CFR, die gemeinsame MIMO-Normalisierung sowie die
legacy-kompatible und explizite Knotenorientierung. Zwei Phase-4.1-Fälle
ergänzen denselben Zweinutzer-Rundlauf und die Negativmatrix für Identität,
Richtung, Zeitpunkt, Frequenzraster und heterogene RX-Geometrien. Diese
Tests benötigen weder Raytracing, ZMQ noch eine GPU. Die analytischen
Vergleiche des numerischen Kerns verwenden weiterhin eine absolute Toleranz
von 1e-10; dies gilt auch für Identitäts-Precoding und synthetische
\(H[k]W[k]\)-Referenzen sowie MRT-/MRC- und SVD-Eigenmode-SINRs.
Precoderleistung, Phaseninvarianz, effektiver MRT-Kanal, deterministische
SVD-Basis, Leistungsanteile, Wasserstand, KKT-Gleichheit und die neue
Feedback-Leistungsidentität werden innerhalb 1e-12 geprüft. Die
Link-Budget-/EESM-Vergleiche verwenden ebenfalls 1e-12.

Die Phase-4.2-Parität und Post-MMSE-SINR verwenden ebenfalls 10^{-10}; gewünschte Leistung, Inter-User-Interferenz, Precoderleistung, EESM und ErrorRateModel-Parität werden innerhalb 10^{-12} geprüft.
Phase 4.3 prüft ZF-Interferenz und Precoderleistung innerhalb 10^{-12} sowie Einzelnutzer-/1x1-SINR innerhalb 10^{-10}; deterministische Wiederholungen werden koeffizientengenau verglichen. Phase 4.7 prüft ZF-/MMSE-SINR und Einzelnutzerparität innerhalb 10^{-10} sowie Interferenz, EESM und PER innerhalb 10^{-12}.
Phase 4.4 prüft gleiche-RU-Interferenz, analytische SINR, EESM/PER, Link Budget und äquivalente Leistung innerhalb 10^{-12}. Die direkte HE-MU-PPDU-Integration verwendet relative 10^{-10}. Der unabhängige Runtime-RU-Test vergleicht EESM/Spektrum innerhalb 10^{-4}; diese Grenze bleibt deutlich unter dem Abstand verschiedener Ton- oder RU-Zuordnungen. Exakte Invarianz-, Nullenergie-, Adressierungs- und PHY-Ereignisbedingungen werden separat geprüft.

Die eigenständige native Suite `dl-mu-mimo-runtime` ergänzt zwei schnelle Fälle für Unterphase 4.5. Der Auswahlfall prüft feste und korrelationsbasierte Gruppen, stabile Nutzer-/STA-ID-/RU-/NSS-Zuordnung, lexikographische Gleichstandsauflösung, direkte ZF-/RZF-Parität, Gesamtleistung, Aktualisierungsraster, Rückmeldedauer, CSI-Alter und die exakte Installation im Phase-4.4-Adapter. Der Negativfall prüft EHT, Uplink, falschen AP, fehlende Bindings, unterschiedliche RUs, mehrere Streams, falsche und zukünftige Zeitpunkte, quantisierte MU-Rückmeldung, SU-Precoder-Rekonstruktion, Überlast, Rangverlust, Gruppen-/Zeit-Mismatch und atomare Ablehnung eines älteren Ersatzes.

Der reale Server-Integrationstest konfiguriert 1x2-PlanarArrays an TX und RX, führt Sionna RT auf der CPU aus und prüft die Form 2 x 2 x 256, 1024 komplexe Werte und mittlere normierte Leistung eins. Die übrigen sieben Servertests decken den unveränderten 1x1-Pfad, Mobilität, Kohärenzzeit und Mehrnutzer-Anfragen ab.

Die ExampleAsTest-Suites enthalten nun zwölf Integrationsprozesse. Die fünf bestehenden Prozesse führen 1x1, Vollrang-2x2, Rang-1-2x2, Identitäts-Precoding und 2x1-Streamauswahl über UDP/SpectrumWifiPhy aus und prüfen widersprüchliches NSS, fehlendes MCS-Beta sowie den CFR-/PHY-Dimensionsguard. Beamforming ist dabei nicht länger pauschal reserviert.

Drei neue positive Prozesse führen das Phasenbeispiel mit idealer Auswahl, grob quantisiertem und drei Millisekunden altem CSI sowie räumlicher Wasserfüllung vollständig auf der CPU aus. Die Referenzlogs vergleichen alle maschinenlesbaren Felder. Vier neue negative Prozesse prüfen fehlende SelectPrecoding-Auswahl, Wasserfüllung mit Quantisierung, MRT mit Alterung und einen nicht zum Aktualisierungsraster passenden CSI-Zeitstempel. Sie enden jeweils früh mit Status 2 und einer stabilen configuration_error-Zeile.

7.1 AUSGEFÜHRTE CPU-VALIDIERUNG

Am 15. August 2026 wurden nach Abschluss der Unterphase 3.5 folgende Prüfungen auf einem CPU-System erfolgreich ausgeführt:

- gezielter Build von Bibliothek, nativer Testsuite, neuem Precoding-Vergleich und bestehendem SU-MIMO-Beispiel unter GCC 13.3
- native ns-3-Suite sionna: 26 von 26 Fällen bestanden. Der neue Helperfall vergleicht MRT und SVD einschließlich Feedback, Alterung und Wasserfüllung direkt mit den bereits vorhandenen wertbasierten Fabriken, prüft die installierten Koeffizienten und Leistungsanteile sowie die Invalidierung einer alten Auswahl. Der bestehende SVD-/Leistungsallokationsfall prüft zwei frequenzselektive diagonale Moden,

unitäre Rotation, gleiche Singularwerte, 1x1-Parität, explizite Stream-/Rangfehler und nicht endliche Eingaben. Wasserfüllung aktiviert bei \(\rho=0{,}5\) nur die starke Mode und bei \(\rho=4\) beide Moden mit den analytischen Anteilen 0,59375 und 0,40625; Leistung, Wasserstand, KKT-Gleichheit und ideale Rate stimmen mit den Referenzen überein. Die bestehenden Phase-1-, Phase-2- und Phase-3.1-/3.2-Fälle bleiben bestanden
- native ns-3-Suite sionna-mimo-csi-feedback: zwei von zwei Fällen bestanden; deterministische HE-/EHT-Quantisierung und Rekonstruktion, Leistungs- und Fehlerdiagnostik, allgemeines 3x2-SU-Feedback sowie CSI-Alterung mit expliziter Zeit- und Zustandsidentität stimmen mit den analytischen Referenzen überein. Der Alterungstest weist zusätzlich den erwarteten SINR-/PER-Verlust bei orthogonal gedrehtem Sendekanal nach
- SISO-Paritätsmatrix: alle 108 Kombinationen stimmen innerhalb 1e-12 mit der direkt aufgerufenen ns-3-Fehlermodellschnittstelle überein
- frequenzselektive Monotonitätsmatrix: alle 312 PER-Werte bestanden
- Reproduzierbarkeit: 20 identische 2x2-Aufrufe mit exakt gleichen Ergebnissen
- ExampleAsTest sionna-example-sumimo-minimal: ein von einem Fall bestanden; Vollrang-2x2 erreicht 134,3904 statt 68,1408 Mbit/s bei 1x1, Rang-1-2x2 erreicht bei PER 1 erwartungsgemäß 0 Mbit/s; Identitäts-Precoding reproduziert die analytischen Vollrangwerte und die 2x1-Auswahl erreicht 23,010300 dB bei voller Gesamtleistung
- ExampleAsTest sionna-example-sumimo-precoding: drei von drei Prozessen bestanden; idealer Vergleich, grob quantisiertes und drei Millisekunden altes CSI sowie Wasserfüllung liefern die vollständigen stabilen Diagnostikzeilen und validation=PASS
- ExampleAsTest sionna-example-sumimo-precoding-negative: vier von vier Prozessen bestanden; fehlende Precoder-Auswahl, Wasserfüllung mit Quantisierung, MRT mit Alterung und ein off-grid CSI-Zeitstempel liefern jeweils die erwartete configuration_error-Zeile
- ExampleAsTest sionna-example-mimo-helper-configuration-negative: drei von drei Fällen bestanden; ungültiges NSS, nicht unterstütztes MU-MIMO und fehlendes MCS-Beta brechen jeweils früh mit Status 2 und eindeutiger Meldung ab
- ExampleAsTest sionna-example-mimo-helper-runtime-negative: ein von einem Fall bestanden; eine helperkonfigurierte 2x2-PHY mit 1x1-CFR erreicht den erwarteten Dimensionsguard im echten SpectrumWifiPhy-Laufzeitpfad
- reine Python-Vertragssuite: fünf von fünf Fällen in 0,001 s bestanden; der separate Legacy-Importlauf bestätigte Sionna 1.2.1, Mitsuba 3.7.1, NumPy 1.26.4, TensorFlow im CPU-Modus ohne GPU und ZeroMQ
- gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 153,015 s bestanden; der reale 2x2-Fall lief in 1,6164 s, lieferte 2 x 2 x 256 und erfüllte die Normalisierung innerhalb 1e-6
- P2P-End-to-End-Test example-sionna-sensing im unveränderten 1x1-Modus: drei 1 x 1 x 3072-CFR-Antworten, zwei empfangene UDP-Pakete zu je 1024 Byte und Cache-Hit-Rate 0,993056

7.2 MIGRATION UND VALIDIERUNG MIT SIONNA 2.0.1

Am 20. August 2026 wurde die Python-Serverumgebung auf Sionna 2.0.1 und Sionna RT 2.0.1 aktualisiert. Die reproduzierbare Abhängigkeitsbasis lautet:

- `requirements.txt` fixiert Sionna 2.0.1, Sionna RT 2.0.1, Mitsuba 3.8.0, Dr.Jit 1.3.1 und PyTorch 2.9.1; die von Sionna geforderte NumPy-Untergrenze ist 2.2.6.
- `requirements_cpuonly.txt` wählt zusätzlich das offizielle CPU-Rad PyTorch 2.9.1+cpu.
- TensorFlow wurde aus beiden Requirements und aus dem Server entfernt. Der Server erkennt den tatsächlich von Sionna RT gewählten Mitsuba-Variantennamen und unterscheidet darüber CUDA- und LLVM-Ausführung.
- Der ns-3-Seed wird bei beiden Raytracing-Pfaden nun mit `seed` explizit an `PathSolver` weitergegeben. Damit steuert der Simulationsseed auch die

  stochastische Strahlabtastung; zuvor blieb dort implizit Sionnas
  Standardwert aktiv.
  - `Paths.cir()`, `Paths.cfr()`, `PlanarArray` und `PathSolver` behalten
  in Sionna RT 2.0.1 die hier verwendete API. Die CFR-Dimension
  [num_rx, num_rx_ant, num_tx, num_tx_ant, num_time_steps, num_subcarriers]
  und damit der bestehende sc/rx/tx-Datenvertrag bleiben unverändert.

Die aktualisierte CPU-Umgebung bestand folgende Prüfungen:

  - strikte Importprüfung für Sionna 2.0.1 und Sionna RT 2.0.1 sowie
  Mitsuba 3.8.0, Dr.Jit 1.3.1, NumPy 2.5.2, PyTorch 2.9.1+cpu und ZeroMQ
  - Python-Vertragssuite: fünf von fünf Fällen in 0,003 s bestanden
  - gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 155,993 s bestanden;
  der reale MIMO-Fall lieferte auf dem LLVM-Backend erneut eine endliche,
  gemeinsam über alle Antennenpaare normalisierte 2 x 2 x 256-CFR
  - native ns-3-Suite `sionna`: 26 von 26 Fällen bestanden
  - native ns-3-Suite `sionna-mimo-csi-feedback`: zwei von zwei Fällen bestanden

Damit ist die Sionna-RT-Schnittstelle für 1x1 und 2x2 unter 2.0.1 validiert.
Der Wechsel von TensorFlow zu PyTorch betrifft den vorhandenen RT-Kanalpfad
nicht; TensorFlow ist für ns3sionna nach der Migration keine
Runtimeabhängigkeit mehr.

Der ältere Beispielpfad example-sionna ist in der unveränderten Basis defekt:
Er startet standardmäßig Mode 3 mit Submode 1, berechnet aber keinen
Look-ahead-Zeitpunkt. Dadurch wird kein Receiver angelegt und Sionna meldet
ValueError: Scene has no receivers. Der explizite P2P-Beispielpfad ist davon
nicht betroffen. Eine Korrektur gehört in eine eigene kleine Baseline-Subphase.

  7.3 VALIDIERUNG DER UNTERPHASE 4.1

Am 20. August 2026 wurden nach der Implementierung von Unterphase 4.1
folgende gezielte Prüfungen erfolgreich ausgeführt:

  - Build des Ziels `sionna-test` einschließlich neu erzeugter
  Protocol-Buffer-Bindings
  - native ns-3-Suite `sionna` mit allen 28 registrierten Fällen; die zwei neuen
  Fälle decken Rundlauf, Sortierung, Identität, Richtung, heterogene
  RX-Dimensionen und die atomare Negativmatrix ab
  - reine Python-Vertragssuite mit sieben von sieben Fällen in 0,004 s
  - gesamte Sionna-CPU-Serversuite mit acht von acht Fällen in 171,865 s,
  einschließlich der vorhandenen Mehrnutzer-, 1x1- und 2x2-Pfade
  - unveränderte SU-Suites `sionna-mimo-csi-feedback`,
  `sionna-example-sumimo-minimal` und
  `sionna-example-sumimo-precoding`; alle Fälle bestanden
  - `sionna-example-mimo-helper-configuration-negative` mit allen drei Fällen;
  insbesondere bleibt der MU-MIMO-Modus des Helpers abgelehnt und nimmt damit
  Unterphase 4.2 nicht vorweg

  7.4 VALIDIERUNG DER UNTERPHASE 4.2

Am 21. August 2026 wurden nach der Implementierung von Unterphase 4.2
folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

  - gezielter Build des Ziels `sionna-test` mit den vier neuen nativen
  Quelldateien
  - native ns-3-Suite `sionna` mit allen 31 registrierten Fällen; die drei
  neuen Fälle decken den festen MU-Precoder, Einzelnutzer- und 1x1-Parität,
  orthogonale und korrelierte Referenzkanäle, Gesamtleistung, nutzerweise
  EESM/PER und die Negativmatrix ab
  - reine Python-Vertragssuite mit sieben von sieben Fällen in 0,005 s; damit

bleiben Serialisierung, Snapshotordnung und heterogene RX-Dimensionen
unverändert
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`,
`sionna-example-sumimo-precoding` mit allen drei positiven Prozessen,
`sionna-example-sumimo-precoding-negative`,
`sionna-example-mimo-helper-configuration-negative` und
`sionna-example-mimo-helper-runtime-negative` erfolgreich
- `git diff --check` ohne Befund

Die Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt:
Unterphase 4.2 ändert weder Python noch Protocol Buffers, Serialisierung,
Batching, Cache, ZMQ oder Sionna RT. Die schnelle Python-Vertragssuite und die
nativen Phase-4.1-Fälle prüfen die unveränderten Grenzen direkt.

7.5 VALIDIERUNG DER UNTERPHASE 4.3

Am 21. August 2026 wurden nach der Implementierung von Unterphase 4.3
folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

- gezielter Build des Ziels `sionna-test`; die Änderung bleibt auf die
bestehenden Phase-4.2-Quell-/Headerdateien und die native Testsuite begrenzt
- native ns-3-Suite `sionna` mit allen 33 registrierten Fällen in 0,024 s;
die zwei neuen Fälle decken orthogonale, korrelierte, schlecht konditionierte,
überlastete und rangdefiziente Kanäle, ZF-Unterdrückung, endliches
deterministisches RZF, Leistung, Ordnung, Regularisierung, Mehrstrom-
ablehnung sowie Einzelnutzer-/1x1-/Phase-4.2-Parität ab
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,003 s; damit
bleiben Serialisierung, Snapshotordnung und heterogene RX-Dimensionen
unverändert
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`,
`sionna-example-sumimo-precoding` mit allen drei positiven Prozessen,
`sionna-example-sumimo-precoding-negative`,
`sionna-example-mimo-helper-configuration-negative` und
`sionna-example-mimo-helper-runtime-negative` erfolgreich
- `git diff --check` ohne Befund

Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt:
Unterphase 4.3 ändert weder Python noch Protocol Buffers, Serialisierung,
Batching, Cache, ZMQ, Sionna RT oder einen Serverpfad. Die schnelle
Python-Vertragssuite und die vollständige native Suite prüfen die berührten
und unveränderten Grenzen direkt.

7.6 VALIDIERUNG DER UNTERPHASE 4.4

Am 3. September 2026 wurden nach der Implementierung von Unterphase 4.4
folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

- gezielter Build des Ziels `sionna-test`
- direkte native Suite `sionna` mit allen 36 registrierten Fällen in 0,048 s;
die drei neuen Fälle decken STA-ID-/Nutzerzuordnung, gleiche und verschiedene
RUs, MCS/NSS, analytische SINR/PER, Leistung, Dimensions- und
Adressierungsablehnungen, SU-/1x1-Parität sowie einen echten HE-MU-PPDU ab
- `./test.py -s sionna --verbose` mit einem von einem Suite-Lauf in 3,77 s
- reine Python-Vertragssuite mit sieben von sieben Fällen in 0,005 s; damit
bleiben Serialisierung, Snapshotordnung und heterogene RX-Dimensionen
unverändert
- unveränderte Suite `sionna-mimo-csi-feedback` mit einem von einem Lauf in
3,54 s
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal` in
18,69 s, `sionna-example-sumimo-precoding` in 37,52 s,

  `sionna-example-sumimo-precoding-negative` in 4,28 s,
  `sionna-example-mimo-helper-configuration-negative` in 3,82 s und
  `sionna-example-mimo-helper-runtime-negative` in 5,08 s; jeweils ein von
  einem Lauf erfolgreich
  - `git diff --check` ohne Befund

Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt:
Unterphase 4.4 ändert weder Python, Protocol Buffers, Serialisierung, Batching,
Cache, ZMQ, Sionna RT noch einen Serverpfad. Die schnelle Python-Vertragssuite,
die native Suite und der echte HE-MU-PPDU-Adapterfall prüfen die berührten und
unveränderten Grenzen direkt.

  7.7 VALIDIERUNG DER UNTERPHASE 4.5

Am 3. September 2026 wurden nach der Implementierung von Unterphase 4.5
folgende Prüfungen auf der CPU-Referenzplattform erfolgreich ausgeführt:

  - gezielter Build des Ziels `sionna-test`
  - neue native Suite `dl-mu-mimo-runtime` mit beiden schnellen Fällen; feste
  und korrelationsbasierte Auswahl, ZF/RZF, Scheduleridentität,
  Zeitdiagnostik und die vollständige Negativmatrix bestanden
  - native Suite `sionna` mit allen 36 bestehenden Fällen erfolgreich
  - unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
  - unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`,
  `sionna-example-sumimo-precoding`,
  `sionna-example-sumimo-precoding-negative`,
  `sionna-example-mimo-helper-configuration-negative` und
  `sionna-example-mimo-helper-runtime-negative` jeweils erfolgreich
  - reine Python-Vertragssuite mit sieben von sieben Fällen in 0,006 s
  - `git diff --check` ohne Befund

Server- und Raytracing-Suites wurden bewusst nicht erneut ausgeführt:
Unterphase 4.5 ändert weder Python, Protocol Buffers, Serialisierung,
Batching, Rohkanalcache, ZMQ, Sionna RT noch einen Serverpfad.

  7.8 KORREKTUR DER FESTEN KOHÄRENZZEIT

Der Server konvertiert `fixed_coherence_time_ms` einmalig an der
Konfigurationsgrenze in `fixed_coherence_time_ns`:

    T_fixed,ns = T_fixed,ms * 1 000 000

Alle drei festen Zeitpfade verwenden diese Nanosekundendauer. Null aktiviert
weiterhin die dynamische Berechnung. Die neue AST-basierte Vertragssuite prüft
die tatsächliche Konfigurationszuweisung sowie alle drei festen und dynamischen
Zweige ohne Raytracing. Die vollständige Python-/CPU-Serversuite bestand am
11. September mit allen 19 Fällen in 229,102 s. Sie umfasst Datenvertrag,
deterministische Mobilität, feste und dynamische Kohärenzzeit sowie reale
1x1-/2x2-CFRs. Die Serverfixtures verwenden 5 ms statt des früheren Testwerts
100000 ms. Diese Korrektur wurde mit Commit `8c112b6` eingecheckt.

  7.9 ERNEUTE VALIDIERUNG DER KORRIGIERTEN UNTERPHASE 4.4

Am 12. September 2026 wurden nach Abschluss der Korrekturen folgende Prüfungen
auf derselben CPU-Referenzplattform erfolgreich ausgeführt:

  - Build des Ziels `sionna-test` einschließlich der neuen nativen Testdateien
  - `dl-mu-mimo-ru`: Spaltenleistungen 1,5/0,5 bei gleichen und ungleichen
  RU-Budgets, relative Gewichte innerhalb derselben RU, Ablehnung einer Gruppe
  ohne aktive Leistung, der analytische EESM-Gegenfall `[10,0]`, Invarianz
  außerhalb der RU sowie ungültige Tonindizes

- `he-dl-mu-spectrum-ru`: unabhängig festgelegte belegte Tonbereiche für Vollband-242-, getrennte 106- und zweigeteilte mittlere 26-Ton-RUs; DC-/Null-/Guardausschluss, Frequenzselektivität, Invarianz außerhalb der RU, tatsächliche `H W`-Energie einer fremden RU, 312,5-/78,125-kHz-Abbildung der gemeinsamen Präambel, atomare Zustandsablehnung sowie physikalische Pfade für Mithörer und einen anderen AP
- native Suite `sionna` mit allen 37 Fällen; der neue Fall sendet zwei normale HE-DL-MU-PPDUs über `MultiModelSpectrumChannel` an zwei adressierte Stationen und einen Mithörer. Beide Stationen dekodieren den orthogonalen Fall mit der unabhängig berechneten PHY-SINR, erreichen im korrelierten Fall die Nutzdatenphase und melden einen Empfangsfehler. Der Mithörer hält CCA busy, dekodiert aber kein PSDU. Gemeinsame Nicht-HE- und HE-Portionen werden damit im echten PHY-Ablauf erfasst.
- unveränderte Suite `dl-mu-mimo-runtime`; Phase-4.5-Auswahl, ZF/RZF, Scheduleridentität und Negativmatrix bestanden
- unveränderte Suite `sionna-mimo-csi-feedback` erfolgreich
- unveränderte ExampleAsTest-Suites `sionna-example-sumimo-minimal`, `sionna-example-sumimo-precoding`, `sionna-example-sumimo-precoding-negative`, `sionna-example-mimo-helper-configuration-negative` und `sionna-example-mimo-helper-runtime-negative` jeweils erfolgreich
- vollständige Python-/CPU-Serversuite mit allen 19 Fällen, einschließlich realer 2x2-CFR, erfolgreich
- `git diff --check` ohne Befund

Damit ist das Abnahmekriterium der Unterphase 4.4 nach der Korrektur erfüllt. Die Gruppenrechnung wird weiterhin einmal je adressiertem Empfänger ausgeführt; diese bekannte Leistungseinschränkung ändert die funktionale Abnahme nicht. Uplink und wissenschaftliche EESM-Kalibrierung bleiben spätere Unterphasen; der Downlink-Helper und Scheduler gehören seit Unterphase 4.6 zum getesteten CPU-Pfad.

7.10 VALIDIERUNG DER UNTERPHASE 4.6

Am 13. September 2026 wurden auf der CPU-Referenzplattform folgende Prüfungen erfolgreich ausgeführt:

- `./ns3 build sionna-test ns3_wifi_mumimo ns3_wifi_sumimo_minimal ns3_wifi_sumimo_precoding`
- die nativen Suites `sionna`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru` und `sionna-mimo-csi-feedback`
- die ExampleAsTest-Suites für SU-Minimal, SU-Precoding, dessen Negativfälle, Helper-Konfiguration, Helper-Laufzeit und `sionna-example-mumimo`; der neue MU-Vergleich bestand in 206,905 s mit dem stabilen Referenzlog
- `./ns3 run "ns3_wifi_mumimo --eesmBetaLinear=2 --rzfAlpha=0.25 --verify=1"`; die Ausgabe endete mit `validation=passed methods=4 channels=2 timings=2`
- `conda run -n sionna-venv python tests/test_imports.py` sowie die Python-Vertragstests für Datenvertrag, deterministische Mobilität, Kohärenzzeit und Server-Unit-Tests. Die Umgebung meldete Sionna/Sionna RT 2.0.1, Mitsuba 3.8.0 mit `llvm_ad_mono_polarized`, PyTorch 2.9.1+cpu und keine CUDA-Laufzeit.

Der MU-Pfad verändert weder Python, Protocol Buffers, Cache-Serialisierung noch den Server. Ein optionaler RTX-3060-Raytracing-Lauf bleibt daher eine Regression des bestehenden Pfads und keine Voraussetzung für die CPU-Abnahme.

7.11 VALIDIERUNG DER UNTERPHASE 4.7

Am 13. September 2026 wurden auf der CPU-Referenzplattform folgende Prüfungen erfolgreich ausgeführt:

- gezielter Build von `libsionna` und `libsionna-test`; die beiden neuen

Uplink-Quelldateien und die eigenständige Testsuite wurden übersetzt und gelinkt
- `./test.py --no-build -s ul-mu-mimo`: eine von einer Suite bestanden. Der analytische Fall prüft beide Detektoren im Einzelnutzergrenzfall, orthogonale Nutzer, ZF-Rangverlust, rangdefiniertes MMSE, Nah-Fern-SNRs 100 und 1 sowie getrennte HE-EESM-/PER-Abbildung
- die Regressionen `sionna`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru` und `sionna-mimo-csi-feedback`: jeweils eine von einer Suite bestanden, ohne Fehler, Überspringen oder Absturz
- `git diff --check` ohne Befund

Der neue Kern ändert weder Python, Protocol Buffers, Cache, Server, Scheduler noch SpectrumWifiPhy. Diese Pfade bleiben für die koordinierte Uplink-Integration der Unterphase 4.8 unverändert.

7.12 VALIDIERUNG DER UNTERPHASE 4.8

Am 14. September 2026 wurden auf der CPU-Referenzplattform folgende Prüfungen erfolgreich ausgeführt:

- `./ns3 build sionna-test ns3_wifi_mumimo`; Adapter, Tests und erweitertes Phasenbeispiel wurden übersetzt und gelinkt
- `he-ul-mu-mimo-phy-adapter`: beide Fälle bestanden. Echte HE-Basic-Trigger- und HE-TB-Objekte liefern für eine gemeinsame 242-Ton-RU die analytischen MMSE-SINRs 4 und 9; getrennte 106-Ton-RUs bilden zwei Gruppen. Fehlende und um eine Nanosekunde verspätete STAs werden getrennt gemeldet, eine fremde Trigger-UID wird abgelehnt.
- die nativen Regressionen `sionna`, `ul-mu-mimo`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru` und `sionna-mimo-csi-feedback`: jeweils eine Suite bestanden, ohne Fehler, Überspringen oder Absturz
- die unveränderten ExampleAsTest-Suites für SU-Minimal, SU-Precoding, dessen Negativfälle sowie Helper-Konfiguration und Helper-Laufzeit: jeweils erfolgreich
- `sionna-example-mumimo`: der normale Referenzloglauf bestand in 221,483 s. Das ideale orthogonale Uplinkbeispiel liefert bei MCS 5 je Nutzer die SINR 10000 und PER null. Sequenzielles SU erreicht 50 Mbit/s, gleichzeitiges UL-MU 100 Mbit/s und damit exakt den Faktor 2. Der Nah-Fern-Fall liefert 75 und 0,75; ZF lehnt Rangverlust ab, MMSE bleibt mit SINR 0,5 definiert.
- vollständige Python-/CPU-Serversuite mit allen 19 Fällen in 191,753 s, einschließlich Datenvertrag, deterministischer Mobilität, Kohärenzzeit und realer 2x2-CFR, erfolgreich
- `git diff --check` ohne Befund

Damit ist das Abnahmekriterium der Unterphase 4.8 erfüllt. Der Adapter bindet die reale Trigger-/HE-TB-Metadatenebene an den unveränderten Uplinkkern; die weiterhin idealisierte Synchronisation bleibt in API, Beispielausgabe und Modellgrenzen ausdrücklich sichtbar.

7.13 VALIDIERUNG DES KALIBRIERWORKFLOWS DER UNTERPHASE 5.1

Am 25. September 2026 wurden die folgenden lokalen CPU-Prüfungen ausgeführt:

- `./ns3 configure --enable-examples --enable-tests` und `./ns3 build ns3_wifi_mimo_awgn_curve` waren erfolgreich.
- Ein direkter Lauf des neuen Beispiels exportierte für MCS 3, BCC, 1458 Byte, NSS 1, eine RX-Antenne, 20 MHz und 800 ns GI genau 281 AWGN-Punkte von -20 bis 50 dB im 0,25-dB-Raster. Der Kalibrierparser las die Metadaten und die monotone PER-Kurve erfolgreich ein.
- `tools/generate_he_link_level_reference.m` führte mit MATLAB WLAN Toolbox R2026a für HE MCS 0, BCC, 32 Byte, 1x1, 20 MHz und 800 ns GI 20 unabhängige

  Punkte mit jeweils 1000 vollständigen codierten HE-Paketen aus. Fünf
  AWGN-Punkte, zehn Trainingspunkte auf zwei festen Zweitap-Kanälen und
  fünf Validierungspunkte auf einem dritten Kanal liegen mit ihren
  234 linearen Post-OFDM-SINRs je Punkt in `data/`.
  - Der unkalibrierte ns-3-AWGN-Pfad verfehlte die MATLAB-AWGN-Referenz mit
  einem mittleren absoluten PER-Fehler von 0,103956. Ein gemeinsam aus
  diesen Daten geschätztes Beta ohne AWGN-Korrektur verfehlte den
  Validierungskanal mit 0,102218 und wurde abgelehnt.
  - Getrennt gefittete AWGN- und Selektivparameter ergeben für die genaue
  genannte PHY-Konfiguration \(\delta_c=-0{,}4958921206\,\mathrm{dB}\)
  und \(\beta_0=1{,}0383687667\) in linearen SINR-Einheiten. Der
  zurückgehaltene AWGN-PER-Fehler beträgt 0,013024 und der Fehler auf dem
  zurückgehaltenen Kanal 0,054886. Beide liegen unter den vordefinierten
  Grenzen 0,05 und 0,10; der Status lautet `validated`.
  - `python -m unittest -v test_calibrate_eesm` bestand mit acht Fällen,
  darunter Recovery von Offset und Beta, getrennte AWGN-/Kanalauswahl,
  Kontext- und Zählervalidierung sowie Ablehnung synthetischer Freigaben.
  - `./ns3 build sionna-test` und der direkte native Test-Runner bestanden:
  `sionna`, `ul-mu-mimo`, `he-ul-mu-mimo-phy-adapter`,
  `dl-mu-mimo-runtime`, `dl-mu-mimo-ru` und
  `he-dl-mu-spectrum-ru`. Der neue C++-Test prüft die exakte
  PHY-Kontextbindung und eine veränderte SU-/DL-MU-PER; bestehende
  SISO-Paritäts- und MU-Regressionen bleiben erfolgreich.

Die validierten Werte sind keine allgemeinen HE-Defaults. Ihre Quelle nutzt
ideales Timing, perfekte RX-CSI und drei kontrollierte Zweitap-Kanäle;
Übertragung auf andere MCS, Paketlängen, Codearten, MIMO-Dimensionen oder
TGax-Kanalmodelle wurde nicht belegt. Deshalb bleiben Beta und Offset ohne
explizite, passende Benutzerkonfiguration ausgeschaltet.

  7.14 CPU-ERGEBNISSE UND VALIDIERUNG DER UNTERPHASE 5.2.1

Am 28. September 2026 wurde die feste Vergleichsmatrix mit Sionna/Sionna RT
2.0.1, Mitsuba 3.8.0, Dr.Jit 1.3.1, NumPy 2.5.2 und PyTorch 2.9.1+cpu
ausgeführt. Der Sweep benötigte 21,641 s nach den Imports. Alle zwölf Paare
frisch initialisierter Serverumgebungen lieferten byteidentische vollständige
Kanalantworten. Laufzeit bleibt von dieser Gleichheitsprüfung ausgenommen;
die nativen Diagnosefelder werden zusätzlich mit relativer Toleranz
\(10^{-6}\) und absoluter Toleranz \(10^{-7}\) geprüft. Diese Toleranzen sind
Numerikgrenzen und keine Messunsicherheits- oder Kalibrierangaben.

Alle 256 Töne aller Fälle besitzen numerischen Rang zwei. Die folgenden
Werte stehen für beide identischen Wiederholungen; Winkel sind Grad,
Abstände sind Wellenlängen, SINRs sind MI-komprimierte Streamwerte in dB.
Die vollständigen Singularwerte und Ton-SINRs bleiben in `tones.csv` erhalten.

| Fall | Abstand | TX-Azimut | RX-Azimut | Median sigma1/sigma0 | Stream 0 dB | Stream 1 dB |
eta bit/s/Hz |
| --- | --- | --- | --- | --- | --- | --- | --- |
| 00 | 0,25 | 0 | 0 | 0,084037 | 45,679 | 41,029 | 28,804 |
| 01 | 0,25 | 0 | 90 | 0,270370 | 56,947 | 55,114 | 37,226 |
| 02 | 0,25 | 90 | 0 | 0,091984 | 47,577 | 46,894 | 31,383 |
| 03 | 0,25 | 90 | 90 | 0,030958 | 39,189 | 38,617 | 25,847 |
| 04 | 0,5 | 0 | 0 | 0,218708 | 53,154 | 50,099 | 34,300 |
| 05 | 0,5 | 0 | 90 | 0,256484 | 57,090 | 57,460 | 38,052 |
| 06 | 0,5 | 90 | 0 | 0,515959 | 55,202 | 56,403 | 37,074 |
| 07 | 0,5 | 90 | 90 | 0,074755 | 46,810 | 46,407 | 30,966 |
| 08 | 1 | 0 | 0 | 0,327041 | 56,491 | 57,387 | 37,830 |
| 09 | 1 | 0 | 90 | 0,143210 | 51,537 | 48,973 | 33,389 |
| 10 | 1 | 90 | 0 | 0,488056 | 60,024 | 60,095 | 39,903 |
| 11 | 1 | 90 | 90 | 0,165965 | 53,823 | 53,034 | 35,497 |

Der Konditionsindikator variiert von 0,030958 bis 0,515959, obwohl der Rang überall zwei bleibt. Beim TX-/RX-Azimutpaar 0/90 sinken die Stream-SINRs zwischen Abstand 0,5 und 1 von 57,090/57,460 auf 51,537/48,973 dB. Der Abstandseffekt ist in dieser konkreten Szene somit nicht durchgehend monoton. Die gekoppelte Änderung von Wideband-Verlust und räumlicher CFR ist in den Exporten getrennt nachvollziehbar. Aus diesem einzelnen statischen Link folgt keine allgemeine optimale Arraygeometrie.

Über die 24 Läufe beträgt der Median der Szeneninitialisierung 3,104 ms, der CFR-/Raytracing-Zeit 0,716 s und der nativen Analyse 0,687 ms. Die CFR-Zeiten liegen zwischen 0,677 und 1,375 s; der Median inklusive nativem Prozessstart und CSV-Ausgabe ist 32,211 ms. Import, IPC und MAC-Verkehr sind gemäß Abschnitt 5.20 abgegrenzt. Der Benchmark erzeugt im lokalen Ausgabeverzeichnis die drei geprüften Vergleichsplots `channel_comparison.png`, `sinr_comparison.png` und `runtime_comparison.png`. Diese Dateien bleiben unversionierte Zusatzinformationen.

Die abschließende automatisierte Validierung war erfolgreich:

- `./ns3 configure --enable-examples --enable-tests` und gezielte Builds von `ns3_wifi_sumimo_benchmark`, `sionna-test`, SU-Minimal, SU-Precoding, MU-Beispiel und AWGN-Export waren erfolgreich.
- `python -m unittest -v test_benchmark_sumimo`: elf Fälle bestanden in 5,606 s. Sie prüfen sämtliche Sweepkontrollen und Exportlängen, Hashes, byteidentische Wiederholungen, die explizite Konfiguration aller drei Eulerkomponenten, tatsächliche Sionna-Array-/Radio-Geometrie, analytische Rang-/Singularwert-/MMSE-/MI-Referenzen sowie negative Geometrie- und Datenvertragsfälle. Alle zwölf eingefrorenen realen Kanäle wurden im nativen Programm erneut ausgewertet und gegen unabhängige NumPy-SVD und MMSE-Inversion geprüft. Drei repräsentative Fälle (00, 05, 10) wurden zusätzlich in einem frischen Python-Prozess erneut raygetraced und stimmen bytegenau mit der früher erzeugten Referenz überein.
- Die sieben nativen Suites `sionna` (38 Fälle), `sionna-mimo-csi-feedback`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru`, `ul-mu-mimo` und `he-ul-mu-mimo-phy-adapter` bestanden im vorhandenen Test-Runner.
- Die sechs ExampleAsTest-Suites für SU-Minimal, Helper-Konfiguration, Helper-Laufzeit, SU-Precoding, Precoding-Negativfälle und MU-MIMO bestanden. Der umfassende MU-Referenzloglauf benötigte 243,979 s.
- Der Server-Importcheck und die vollständige Python-/CPU-Serversuite aus Datenvertrag, deterministischer Mobilität, Kohärenzzeit und Server-Unit-Tests bestanden mit 19 Fällen in 192,187 s, einschließlich realer 1x1-/2x2-CFR und beider Platzierungspfade.
- `python -m unittest -v test_calibrate_eesm`: acht Fälle bestanden.
- `git diff --check` war ohne Befund. Nach der letzten Dokumentationsänderung wurde das Modell-PDF genau einmal erzeugt und mit Seitenzahlprüfung sowie vollständiger Textextraktion auf die neuen Abschnitte validiert.

Alle Regressionen wurden sequenziell ausgeführt. Die Befehle, Exitcodes, Wanduhrzeiten und Log-Hashes stehen in `data/phase5_2_spacing_orientation/validation.json`. Die elf neuen Tests verwenden standardmäßig die versionierten Referenzen und lassen sich ohne externen Server reproduzieren; die vier tatsächlichen Raytracing-Aufrufe der Geometrie-/Referenztests benötigen das dokumentierte CPU-Environment. Der erste Teil von Phase 5.2 ist damit abgeschlossen. Zusätzliche Winkel, getrennte TX-/RX-Abstände, CSI-Alter und Cache-/IPC-Skalierung bleiben weitere Arbeitspakete. Die Samplekonvergenz ergänzt Unterphase 5.2.2 in Abschnitt 7.15. Eine 2x2-PER-Kalibrierung wurde nicht abgeleitet.

7.15 CPU-SAMPLEKONVERGENZ UND VALIDIERUNG DER UNTERPHASE 5.2.2

Der vorab festgelegte Sweep wurde am 29. September 2026 auf derselben CPU-/LLVM-Plattform wie 5.2.1 ausgeführt. Die 72 frischen Traces lieferten innerhalb jedes Fall-/Seed-/Budgetpaares byteidentische Kanalantworten. Zwischen verschiedenen Seeds wird diese Identität nicht verlangt. Die Referenz enthält zwölf vollständige Helperkonfigurationen, 72 rohe Kanalantworten, 72 Zusammenfassungszeilen, 18432 Tonzeilen und 36 gepaarte Konvergenzvergleiche. Alle Töne behalten numerischen Rang zwei.

Konvergenz ist für die geprüfte Matrix bei 100000 Samples je Quelle innerhalb der vorab definierten Toleranzen erreicht. Alle zwölf 100000/1000000-Fall-/Seedpaare bestehen sämtliche Kriterien. 10000 Samples sind für die gesamte Matrix nicht ausreichend: Fall 03 mit Seed 43 scheitert gegen beide höheren Budgets an Singularwert- und Ton-SINR-Toleranz. Insgesamt bestehen 34 von 36 Vergleichen. Die übrigen 10000-Paare bestehen; das globale Urteil wird durch den schwachen Kanal bestimmt.

Die folgenden Werte sind die größten Fehler über alle vier Fälle und drei gepaarten Seeds. Der Singularwertfehler ist gemäß Abschnitt 5.21 bereits durch seine relative/absolute Toleranz geteilt; sein Grenzwert ist eins. Bei Rang bedeutet null, dass kein Ton seinen Rang ändert.

Vergleich b/q	Verlust / dB	Rangänderungen	D_sigma	Ton-SINR / dB	MI-SINR / dB	MI / bit/s/Hz	Bestanden
10000/100000	0,067268	0	4,554496	1,849055	0,276911	0,169142	11/12
100000/1000000	0,000574	0	0,084528	0,038117	0,007545	0,004979	12/12
10000/1000000	0,067264	0	4,562569	1,852972	0,276811	0,169080	11/12

Beim fehlschlagenden Fall 03/Seed 43 hat der schwache Singularwert am Tonindex 0 (-10 MHz) bei 10000 Samples den Wert 0,02895517 und bei 1000000 den Wert 0,02357629: 22,8148 Prozent relative Differenz. Der numerische Rang bleibt trotzdem zwei. Auch effektive SINR und MI-Effizienz liegen innerhalb ihrer Grenzen. Ein Vergleich nur von Wideband-Verlust, Rang oder effektiver SINR hätte daher die Ton-/Modeninstabilität übersehen.

Die nächste Tabelle zeigt Mediane der exportierten Laufzusammenfassungen über Seeds und Wiederholungen. sigma0/sigma1 sind je Lauf zunächst die Mediane über alle Töne; MI-SINR ist in dB angegeben. Die vollständigen einzelnen Ton-/Stream-/Seedwerte stehen in `tones.csv` und `summary.csv`.

Fall	Samples	Verlust / dB	sigma0/sigma1	MI-SINR 0/1 / dB	MI / bit/s/Hz
03	10000	49,948140	1,996718/0,060352	39,044913/38,469264	25,750037
03	100000	49,965776	1,995746/0,061211	39,188952/38,617453	25,847100
03	1000000	49,965775	1,995747/0,061213	39,189057/38,617575	25,847176
05	10000	50,516401	1,980547/0,503382	56,979714/57,376811	37,988421
05	100000	50,490308	1,980246/0,510831	57,089710/57,459474	38,052421
05	1000000	50,490313	1,980243/0,510846	57,089832/57,459695	38,052535
06	10000	55,828542	1,712816/0,890614	55,245325/56,493283	37,118770
06	100000	55,848764	1,713904/0,884302	55,201884/56,402762	37,074269
06	1000000	55,848763	1,713903/0,884302	55,201889/56,402764	37,074271
10	10000	51,711566	1,788855/0,872923	60,018006/60,095267	39,900769
10	100000	51,709226	1,789760/0,872812	60,024357/60,094553	39,902638
10	1000000	51,709224	1,789760/0,872813	60,024363/60,094549	39,902642

Zusätzlich beträgt die größte innerhalb eines Falls beobachtete Seedspanne der effektiven Stream-SINR 0,464537 dB bei 10000, 0,007693 dB bei 100000 und 0,000288 dB bei 1000000 Samples. Diese deskriptiven Spannen sind keine Konfidenzintervalle aus drei Seeds und kein Ersatz für die gepaarte Worst-Tone-Entscheidung.

Die Wanduhrzeiten je Budget über alle 24 zugehörigen Läufe lauten:

Samples	Init-Median / ms	CFR-Median / s	CFR-Min/Max / s	Native Analyse / ms
10000	3,543	0,151796	0,122829/2,069792	0,725
100000	2,774	0,736202	0,698936/1,486344	0,661
1000000	2,670	6,530119	6,301047/11,274462	0,687

Der Median des nativen Prozessaufrufs einschließlich CSV-Ausgabe liegt je Budget bei etwa 32 ms. Das dokumentierte `sweep_seconds` beträgt 202,463 s, ohne Import und Plot-/Manifestausgabe; der gesamte Runnerprozess benötigt 211,500 s. CFR-Zeiten enthalten Raytracing, CFR-Aufbau und gegebenenfalls JIT-Aufwand gemäß Abschnitt 5.20. Reihenfolge, JIT und Systemlast machen diese Messung zu einer Laufzeitdiagnose und nicht zu unabhängigen Performance-Stichproben. Der etwa 8,9-fache CFR-Median für 1000000 gegenüber 100000 Samples steht hier einer bereits bestandenen Stabilitätsprüfung bei 100000 gegenüber; daraus folgt keine allgemeine Budgetempfehlung für andere Szenen oder Solveroptionen.

Der Runner erzeugte die vier lokal geprüften Vergleichsplots `channel_comparison.png`, `sinr_comparison.png`, `runtime_comparison.png` und `convergence_comparison.png`. Der letzte Plot zeigt pro Fall die schlechteste gepaarte Seedabweichung gegen 1000000, geteilt durch die jeweilige Toleranz. Die horizontale Linie eins macht die beiden verletzten 10000-Kriterien sichtbar. Plotdateien bleiben gemäß den bestehenden Ignore-Regeln unversioniert.

Die automatisierten Ergebnisse und Regressionen werden zusammen mit Befehlen, Exitcodes, Wanduhrzeiten und Log-Hashes in `data/phase5_2_sample_convergence/validation.json` festgehalten. `test_benchmark_rt_convergence` enthält zehn Prüfungen: einzelne Grenzwertverletzungen einschließlich eines einzelnen schlechten Tons, Nullmoden-/SINR-Floor, ungültige Vergleichsmatrizen, vollständige fixe Kontrollen, reale Array-/Radio-Geometrie, Exporthashes, byteidentische Wiederholungen, unabhängige NumPy-SVD-/MMSE-/MI-/Linkbudgetreferenzen für sämtliche 72 Antworten, Neuberechnung des gesamten Urteils und nativen Replay der 36 eindeutigen Kanäle. Drei frische Traces in einem separaten Testprozess prüfen alle drei Budgets und Seeds. Die vier früheren 100000/Seed-42-Kanäle stimmen bytegenau mit der 5.2.1-Referenz überein.

Die abschließende sequenzielle Validierung war vollständig erfolgreich:

- Der gezielte Build von `ns3_wifi_sumimo_benchmark` und `sionna-test` bestand. Vier Policyprüfungen wurden bereits vor dem Sweep ausgeführt.
- `python -m unittest -v test_benchmark_rt_convergence`: zehn Fälle bestanden in 15,952 s (16,601 s einschließlich Prozessstart).
- Die elf bestehenden `test_benchmark_sumimo`-Fälle bestanden in 8,088 s; damit sind Defaultseed, Defaultbudget und alle früheren Referenzen erhalten.
- Die sieben nativen Suites `sionna` (38 Fälle), `sionna-mimo-csi-feedback`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`, `he-dl-mu-spectrum-ru`, `ul-mu-mimo` und `he-ul-mu-mimo-phy-adapter` bestanden.
- Alle sechs ExampleAsTest-Suites für SU-Minimal, Helper-Konfiguration, Helper-Laufzeit, SU-Precoding, Precoding-Negativfälle und MU-MIMO bestanden; der MU-Prozess benötigte 240,972 s.
- Server-Importcheck und alle 19 Python-/CPU-Serverregressionen für Datenvertrag, deterministische Mobilität, Kohärenzzeit und reale 1x1-/2x2-CFR bestanden. Die Serversuite benötigte 192,917 s Testzeit (196,161 s einschließlich Prozessstart).
- Die acht `test_calibrate_eesm`-Regressionen bestanden.

- Alle 21 gespeicherten Quell-/Szenenhashes stimmen mit der finalen
  Implementierung überein; alle vier Plotdateien wurden visuell geprüft.
- `git diff --check` war ohne Befund. Nach der finalen Dokumentationsänderung
  wurde das Modell-PDF genau einmal erzeugt und durch Seitenzahl,
  vollständige Textextraktion, neue Ergebnis-/Formelmarker und Sichtprüfung
  validiert. Die Hashes und Seitennachweise stehen in
  `data/phase5_2_sample_convergence/pdf_validation.json`.

Der obere Budgetschritt hat damit für diese vorab fixierte Testmatrix ein
Stabilitätsplateau erreicht. Das größte Budget bleibt eine endliche
Referenz. Eine 2x2-PER-Kalibrierung oder asymptotische Konvergenzaussage
wird aus diesen Daten nicht abgeleitet.

7.16 CPU-CACHE-/IPC-SKALIERUNG UND VALIDIERUNG DER UNTERPHASE 5.2.3

Die finale Referenz vom 29.09.2026 umfasst alle 45 Konfigurationen mit
je drei frischen nativen Prozessen und separaten Python-REP-Prozessen.
Alle vorab definierten Abnahmekriterien aus Abschnitt 5.22 sind erfüllt:
bitgenaue CFR-Erhaltung, exakte Metadaten und Wirebytes, vollständige
logische Speicherformeln, reproduzierbare deterministische Ergebnisse,
korrekte Hit-/Misszahlen und echte Expiration über den produktiven Cache.
Die Daten liegen in `data/phase5_2_cache_ipc/`: 15 Protobuf-Vorlagen,
`plan.json`, `manifest.json`, 135 Zusammenfassungen in `summary.csv`,
30915 Zeitstichproben in `timings.csv` und 3645 Channel-Requests in
`requests.csv`. Große Batchantworten bleiben aus den Vorlagen und den
festen Ableitungsregeln vollständig rekonstruierbar und hashgeprüft.

Die Plattform ist Linux x86_64 mit glibc 2.39, Python 3.12.12, NumPy
2.5.2, Python-Protobuf 7.35.1, C++-Protobuf 3.21.12 und libzmq 4.3.5
(PyZMQ 27.1.0). Szenen-/Solverlaufzeit ist ausgeschlossen. Seed 42,
Run 1, Fixture, Radio-Geometrie, Gültigkeit und Transport bleiben fest.
Der Sweep benötigte 125,273 s ohne Import, CSV-/Plot-/Manifestausgabe;
der vollständige Runnerprozess benötigte 131,951 s. Es gab keine
parallelen Benchmark- oder Regressionstestprozesse.

Die folgenden Latenzen sind Mediane der drei Prozessmediane. Die
256-Subcarrier-/2x2-Matrix entspricht den Transportdimensionen des
bisherigen SU-Benchmarks. Nutzbytes enthalten CFR und Frequenzvektoren;
Heapdelta bezeichnet Delta A_release aus Abschnitt 5.22.

Links	Wire / Byte	Nutzbytes	Heapdelta / Byte	Hit / us	Miss / ms	RTT / ms
1	18255	17408	16592	1,964	0,142230	0,127272
16	291420	278528	278672	2,044	0,603779	0,261919
64	1165548	1114112	1134992	2,235	2,310256	0,804902

Bei festem Tensor bleiben Getter-Hits damit nahe 2 us, während ein
Batchmiss alle Links transportiert, dekodiert und einfügt. Der Hit
ist ein öffentlicher Getter mit CFR-Kopien und Ergebnisfreigabe. Die
Beobachtung beschreibt diesen Cachepfad; sie ist keine Messung einer
isolierten Map-Suche. Seine Hitquote ist in allen 135 Läufen 132/134
(98,507463 Prozent). Der separate Speicherprüfcache liefert für
1/16/64 Links genau 4L/(4L+1): 80/98,461538/99,610895 Prozent.
Kein Warmhit erzeugt einen Channel-Request. Der Randhit bei 5000000 ns
und der neue Request bei 5000001 ns sind im Transkript nachgewiesen.
Je Prozess wurden 27 Channel-Requests und ein Close-ACK bestätigt.

Bei 1024 Subcarriern und 64 Links wächst der logische Bedarf exakt
linear mit dem Antennenprodukt. Die nächste Tabelle verwendet MiB
(2^20 Byte) und die Reihenfolge TX/RX, auch für den asymmetrischen Fall.

TX/RX	Wire / MiB	Nutz / MiB	Heapdelta / MiB	Hit / us	Miss / ms	RTT / ms
2/2	4,439802	4,25	4,274933	12,073	10,483940	3,181524
2/4	8,439802	8,25	8,274857	17,113	17,000560	5,558643
4/4	16,439802	16,25	16,274933	32,497	28,949737	9,720021
8/8	64,439802	64,25	64,274689	110,871	212,156401	127,523298

Die größte Antwort hat 67570030 Wirebytes. Ihr Cache enthält 64 MiB
komplexe Koeffizienten plus 0,25 MiB Frequenzen. Delta A_fill beträgt
67399616 Byte, Delta A_release 67396896 Byte; letzteres liegt rund
25888 Byte über dem logischen Nutzbedarf. Bei kleinen Blöcken zeigt
die glibc-Buchhaltung dagegen die dokumentierte Threadcache-Grenze:
für 64 Subcarrier, 1/1 und einen Link sind Nutzbedarf 1280 Byte,
Delta A_fill 1376 Byte und Delta A_release nur 192 Byte. Die übrigen
freigegebenen Blöcke müssen nicht aus uordblks verschwinden. Diese
Abweichung verletzt keine Daten-/Speicherformel und wird nicht als
negative Cachegröße oder als exakte Objektzählung interpretiert.

Für die größte Konfiguration sind die medianen nativen RSS-Werte
vor Befüllung/live/nach Freigabe 219,500/285,500/205,121 MiB. Der
aktuelle native VmHWM beträgt 414,680 MiB, der des REP-Prozesses
485,602 MiB. Beide Peaks umfassen alle Messphasen und temporäre
Objekte, nicht nur den isoliert gemessenen Cache. Der zusätzliche
native Lebenszeitpeak aus getrusage liegt bei 663,758 MiB und kann
den vor exec vorhandenen Elternspeicher einschließen; er wird separat
exportiert und nicht als aktueller Programmpeak berichtet. Ein erster
Sweep wurde nach dieser Peak-Abgrenzung durch den finalen vollständigen
Sweep ersetzt. Die finale Referenz verwendet durchgehend VmHWM.

Die getrennten Protobuf-Operationen zeigen für den größten Wrapper
folgende Mediane und Min-/Max-Spannen der drei Prozessmediane:

Operation	Median / ms	Min / ms	Max / ms
C++-Serialisierung	58,579679	55,996002	96,448522
C++-Parse	11,561986	9,398454	12,133919
Python-Serialisierung	247,430939	236,522898	250,878334
Python-Parse	10,742933	10,566953	10,793618
Einzel-CFR-Dekodierung	0,211423	0,204865	0,440899
IPC-Roundtrip	127,523298	105,808909	171,125000
Cache-Hit	0,110871	0,089786	0,159248
Cache-Miss	212,156401	207,712946	281,981298

Die Timergrenzen unterscheiden sich wie in Abschnitt 5.22 angegeben.
Der REP-Prozess sendet vorbereitete Bytes; die separate Python-
Serialisierung gehört deshalb nicht zur gemessenen RTT oder zum
gemessenen Cachemiss. Ein normaler Server würde zusätzlich seine
Kanalberechnung und Antwortserialisierung ausführen. Systemlast und
Allokationszustand erklären mögliche Streuung, ohne hier kausal
isoliert zu sein. Drei Prozesse liefern deskriptive Spannen, keine
Konfidenzintervalle und keine allgemeine Serverdimensionierung.
Es wurden vorab keine pauschalen Latenz- oder RSS-Grenzen festgelegt.

Die drei erzeugten und visuell geprüften Plots heißen
`memory_comparison.png`, `cache_comparison.png` und `ipc_comparison.png`.
Sie zeigen alle Tensorformen, Linkzahlen und Subcarrierbudgets mit
Median und Min-/Max-Spanne. PNGs bleiben gemäß den bestehenden
Ignore-Regeln lokal; der Runner rekonstruiert sie aus einem neuen Sweep.

Die sequenziellen automatisierten Prüfungen waren vollständig erfolgreich:

  - Konfiguration mit Examples/Tests und Build von
  `ns3_sionna_cache_ipc_benchmark` und `sionna-test` bestanden.
  - Die neun `test_benchmark_cache_ipc`-Fälle bestanden abschließend
  in 20,762 s (21,035 s mit Prozessstart). Sie prüfen unabhängige
  flache Ankerindizes aller 15 Formen, Phasen-/Linkmetadaten und
  IEEE-Bitprüfsumme, Bytewiederholbarkeit, sichere Ausgabeverzeichnisse,
  reale native Wiederholungen, Manipulationen/ungültige Fixtures,
  vollständige 45x3-Matrix, alle Referenzbytes/Quoten/Requestfolgen,
  sämtliche Rohzeitstatistiken und frische Replays dreier Randfälle
  einschließlich 1024/8x8/64. Asymmetrische 2/4-Layouts sind abgedeckt.
  - Alle 21 bisherigen SU-/Samplekonvergenz-Benchmarktests bestanden
  in 20,096 s. Die gespeicherten 5.2.1-/5.2.2-Referenzen bleiben erhalten.
  - Die sieben nativen Suites `sionna` (38 Fälle),
  `sionna-mimo-csi-feedback`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`,
  `he-dl-mu-spectrum-ru`, `ul-mu-mimo` und
  `he-ul-mu-mimo-phy-adapter` bestanden.
  - Alle sechs ExampleAsTest-Suites für SU-Minimal, Helper-Konfiguration,
  Helper-Laufzeit, SU-Precoding, Precoding-Negativfälle und MU-MIMO
  bestanden; der MU-Prozess benötigte 226,706 s.
  - Server-Importcheck und alle 19 Datenvertrag-/Mobilitäts-/Kohärenz-/
  CPU-Kanalregressionen bestanden in 199,305 s Testzeit; die acht
  `test_calibrate_eesm`-Fälle bestanden ebenfalls.
  - Alle 13 aufgezeichneten Quellhashes, der Ankerhash und der native
  Binärhash stimmen mit der finalen Implementierung überein.
  - `git diff --check` bestand. Nach den finalen Dokumentationsänderungen
  wurde das PDF genau einmal erzeugt und durch Seitenzahl, Textmarker,
  vollständige Seitenfußzeilen und Sichtprüfung validiert.

Befehle, Exitcodes, Laufzeiten und Log-Hashes stehen in `validation.json`;
PDF-/Quellhash und Seitennachweise in `pdf_validation.json` im selben
Referenzverzeichnis. Die Unterphase ist als Mess-/Validierungsarbeit
abgeschlossen. Cache-, Helper-, Server- und Protobuf-Produktion bleiben
unverändert; konkrete Speicher-/Kopier-/Transportoptimierungen können
anschließend gegen diese Referenz geprüft werden. Die Transportfixtures
begründen keine PHY-, PER- oder Durchsatzprognose für größere Arrays.

7.17 ERGEBNISSE UND VALIDIERUNG DER UNTERPHASE 5.2.4

Der CPU-Lauf vom 29.09.2026 erzeugte alle 42 vorab geplanten Traces und
10752 Tonzeilen. Sämtliche Wiederholungen waren byteidentisch; die
tatsächlichen Array-/Radio-Geometrien und unabhängigen SVD-/MMSE-/MI-/
Linkbudgetprüfungen bestanden für alle Traces. Die drei 100000-Sample-
Kontrollkanäle von Fall 00 stimmen für jeden Seed bytegenau mit dem
früheren Fall 06 aus 5.2.2 überein. Alle Töne aller Traces haben Rang zwei.

Die Tabelle zeigt Mediane über drei Seeds mit je zwei Wiederholungen bei
100000 Samples. sigma0/sigma1 sind zunächst Tonmediane je Lauf. MI-SINR
ist in dB, die Effizienz in bit/s/Hz angegeben.

Fall	Verlust / dB	sigma0/sigma1	MI-SINR 0/1 / dB	MI / bit/s/Hz
00	55,848764	1,713904/0,884302	55,201884/56,402762	37,074269
01	51,467147	1,904154/0,589488	58,649012/56,795143	38,349723
02	52,087101	1,972575/0,190528	46,584869/47,079595	31,114722
03	46,739740	1,866247/0,678221	64,526283/62,399438	42,163812
04	48,373161	1,995263/0,322120	55,386478/55,815199	36,940405

Bei gleichen Winkeln unterscheiden sich 01 und 02 im medianen
Wideband-Verlust nur um 0,620 dB, aber in der MI-Effizienz um
7,235 bit/s/Hz. Der mediane Tonquotient sigma1/sigma0 beträgt
0,309964 beziehungsweise 0,098778. Der Rang allein beschreibt diese

schwächere zweite Mode nicht. Vertauschte TX-/RX-Abstände sind für diesen
Link mit unterschiedlichen Arraymustern und Orientierungen nicht äquivalent.
Bei 45/-45 Grad beträgt der Effizienzunterschied 5,223 bit/s/Hz.
Diese Beobachtung ist auf die feste Szene und den Open-Loop-Messvertrag
begrenzt; die hohen gaußschen MI-Werte sind keine Wi-Fi-Payloadraten.

Alle sechs gepaarten Seed-/Fall-Spotchecks bestanden. Die schlechtesten
Abweichungen über diese Checks sind:

Kriterium	Maximum	Grenze
Wideband-Verlust / dB	0,000063766	0,5
Töne mit Rangänderung	0	0
Skalierter Singularwertfehler D_sigma	0,004976263	1
Ton-/Stream-SINR / dB	0,002644970	1
Effektive Stream-SINR / dB	0,000149771	0,5
MI-Effizienz / bit/s/Hz	0,000090278	0,2

100000 Samples sind damit für die beiden ausgewählten neuen Winkel-/
Abstandskonfigurationen innerhalb der bestehenden Toleranzen stabil.
Für die neuen Fälle 01/02 wurde kein zusätzlicher höherer Budgetlauf
ausgeführt; aus dem Spotcheck wird kein globales Konvergenzurteil abgeleitet.
Min-/Max-Spannen über drei Seeds sind deskriptiv, keine Konfidenzintervalle.

Die CFR-Mediane betragen 0,698006 s bei 100000 (30 Traces) und
6,451845 s bei 1000000 (zwölf Traces). Native Analysemediane liegen
bei 0,678 beziehungsweise 0,651 ms; native Prozessaufrufe bei rund
32 ms. `sweep_seconds` ist 107,656 s, der vollständige Runnerprozess
benötigte 112,533 s. Laufzeiten bleiben host-/JIT-/lastabhängige Diagnosen.
Software und Backend entsprechen 5.2.2: Sionna/RT 2.0.1, Mitsuba 3.8.0,
Dr.Jit 1.3.1, NumPy 2.5.2 und LLVM-CPU mit einem Dr.Jit-Thread.

Die vorhandenen 21 Benchmarktests bestanden in 19,969 s. Ihre bestehenden
Geometrie-, Wiederholungs- und Negativtests wurden um getrennte Abstände
erweitert; die reale Geometrieprüfung verwendet nun unterschiedliche
TX-/RX-Abstände. Es wurden keine neuen Testdateien oder Suites hinzugefügt.
Der gezielte Build von `ns3_wifi_sumimo_benchmark`/`sionna-test` und die
native `sionna`-Suite mit 38 Fällen bestanden. Die lokale Exportprüfung
validierte alle Traces, Rohmetriken, Hashes und Spotcheckentscheidungen,
bestätigte die alten Kontrollkanäle und replayte zwei neue Kanäle nativ.
Alle 21 aufgezeichneten Quell-/Szenenhashes und der Binärhash stimmen.
Die drei Vergleichsplots wurden visuell geprüft. Unveränderte teure
MU-Example-/Serverregressionen wurden nicht erneut ausgeführt.

Messwerte, Konfigurationen, Kanalfixtures, Vergleichsplots und Prüfprotokolle
liegen lokal unter `data/phase5_2_independent_spacing/`; das gesamte
Verzeichnis ist ignoriert. Der Commit enthält ausschließlich die kleine
Erweiterung vorhandener Beispiele/Tools/Tests und die Dokumentation.
Nach den finalen Dokumentationsänderungen wurde das PDF einmal erzeugt,
durch Seitenzahl, Textextraktion und Sichtprüfung validiert. Die Hashes
und Prüfprotokolle bleiben ebenfalls lokale Artefakte. Produktionsmodelle,
Helper, Server und Protobuf wurden für diese Unterphase nicht geändert.

7.18 ERGEBNISSE UND VALIDIERUNG DER UNTERPHASE 5.2.5

Der LLVM-CPU-Lauf vom 29.09.2026 erzeugte alle 24 geplanten initialen
RT-Traces, 192 Auswertungszeilen und 49152 Tonzeilen. Sämtliche initialen
CFR-Wiederholungen waren byteidentisch; alle 24 CFRs stimmen außerdem
bytegenau mit den entsprechenden Fällen/Seeds/Budgets/Wiederholungen aus
5.2.2 überein. Die tatsächlichen Array-/Radiogeometrien wurden geprüft.
Alle unabhängigen SVD-/MMSE-/MI-/Linkbudgetprüfungen und alle zwölf

statischen Kontrollpaare bestanden. Rang zwei bleibt auf jedem Ton erhalten, die Precoderleistung beträgt zwei. Nullalter reproduziert frisches SVD; positives Alter bei Evolutionsrate null ändert keine Linkmetriken. Verlust und Singularwerte bleiben bei der unitären Entwicklung innerhalb der numerischen Referenztoleranzen unverändert.

Frische Baselines bei 100000 Samples; Mediane über drei Seeds mit je zwei Wiederholungen. Singularwerte sind zunächst Tonmediane je Lauf:

Fall	Verlust / dB	sigma0/sigma1	MI-SINR 0/1 / dB	MI / bit/s/Hz
03	49,965776	1,995746/0,061211	67,005236/35,885346	34,179882
06	55,848764	1,713904/0,884302	60,053426/53,721877	37,795345

Verzögerte Auswertung desselben 5-ms-Sendekanals. D_0/D_1/D_eta sind Mediane der gepaarten Verluste gegen frisches CSI des jeweiligen Seeds; negative SINR-Verluste bedeuten einen Gewinn des betreffenden Streams:

Fall	Alter / ms	D_0 / dB	D_1 / dB	D_eta / bit/s/Hz	Verzögerte MI / bit/s/Hz
03	0	0	0	0	34,179882
03	1	12,701134	-0,058929	4,199649	29,980233
03	2,5	20,233071	-0,365202	6,599963	27,579919
03	5	25,531033	-1,398795	8,016558	26,163324
06	0	0	0	0	37,795345
06	1	0,046198	-0,011511	0,011523	37,783823
06	2,5	0,269186	-0,069421	0,066361	37,728985
06	5	0,856004	-0,242579	0,203776	37,591570

In dieser Matrix reagiert die stärkste Mode des schlecht konditionierten Falls 03 deutlich empfindlicher auf veraltete Ausrichtung. Stream eins gewinnt gleichzeitig SINR; der Gewinn gleicht den Verlust von Stream null in der getrennten MMSE-MI nicht aus. Bei 5 ms betragen die medianen Korrelationen c 0,850353 für 03 und 0,803003 für 06, obwohl 03 den größeren Effizienzverlust hat. Ein skalarer Korrelationswert beschreibt daher diese MMSE-Empfindlichkeit nicht allein; Modeverhältnis und Basisorientierung sind ebenfalls beteiligt. Die relative CFR-Änderung ist in beiden Fällen bei 1/2,5/5 ms 0,156918/0,390181/0,765367. Gemeinsame gaußsche Kapazität, Kanalenergie, Rang und Singularwerte bleiben in diesem Experiment konstant.

Alle 48 gepaarten Budgetchecks bestanden. Die Maxima über alle Fall-/Seed-/Alter-/CSI-Varianten betragen:

Kriterium	Maximum	Grenze
Wideband-Verlust / dB	0,000574092	0,5
Töne mit Rangänderung	0	0
Skalierter Singularwertfehler D_sigma	0,084527716	1
Ton-/Stream-SINR / dB	0,037378018	1
Effektive Stream-SINR / dB	0,007641967	0,5
MI-Effizienz / bit/s/Hz	0,004928625	0,2

Damit sind 100000 Samples für die getestete repräsentative Matrix gegen die endliche 1000000-Samplereferenz innerhalb der vorab festgelegten Toleranzen stabil. Das zertifiziert weder asymptotische Konvergenz noch andere Szenen, Bewegungen, Arrays oder Evolutionsraten. Der Verlust von 0,203776 bit/s/Hz durch Alter bei 06 ist kein Konvergenzfehler; die 0,2-Grenze gilt ausschließlich für Unterschiede zwischen Samplebudgets.

Die CFR-Laufzeitmediane betragen 0,690818 s bei 100000 und 6,147503 s bei 1000000 Samples (je zwölf Traces). Native Analysen benötigen median

1,332 beziehungsweise 1,338 ms; native Prozessaufrufe einschließlich
CSV-Lesen rund 46 ms. `sweep_seconds` beträgt 112,055 s. Geteilte
Frisch-/Tracezeitwerte werden nicht als unabhängige Laufzeitmessungen
behandelt. Software/Backend entsprechen 5.2.2: Sionna/RT 2.0.1,
Mitsuba 3.8.0, Dr.Jit 1.3.1, NumPy 2.5.2, PyTorch 2.9.1+cpu,
LLVM-CPU mit einem Dr.Jit-Thread. Diese Zeiten bleiben host-/JIT-/lastabhängig.

Der gezielte Build von `ns3_wifi_sumimo_benchmark` und `sionna-test` bestand.
Die vorhandenen Benchmarktests wurden um einen analytischen Vollranganker
mit frischem SVD, 45-Grad-Mischung, Statik, Wiederholung und ungültigen
Zeit-/Ratenangaben sowie eine optionale Prüfung des vollständigen lokalen
Exports ergänzt. Neue Testdateien oder gespeicherte Fixtures sind nicht
erforderlich. Der Exporttest prüft die komplette Matrix, Kanal-/Config-
Hashes, Übereinstimmung mit 5.2.2, jede unabhängige Referenzrechnung,
Wiederholungen, Zusammenfassungen, Zeitfelder, Degradation und die
erneut berechneten Budgetentscheidungen. Ein anfänglicher Fehler beim
Vergleich geplanter Validierungsregeln mit gemessenen Manifestverdicts
wurde ausschließlich im Test korrigiert; die Messimplementierung und
Messwerte blieben unverändert. Messzeitpunkt-Quellhashes bleiben im
Manifest erhalten, die finalen Test-/Quellhashes stehen separat lokal.

Die abschließenden 23 Python-Benchmarktests und die nativen Suites
`sionna`, `sionna-mimo-csi-feedback`, `sionna-example-sumimo-precoding`
und `sionna-example-sumimo-precoding-negative` wurden erfolgreich ausgeführt.
Die beiden Altersplots wurden visuell geprüft. Produktionsmodelle, Helper,
Server und Protobuf wurden nicht verändert; teure unveränderte MU-/
Serverregressionen wurden nicht erneut ausgeführt.

Raw-CFRs, Messwerte, Konfigurationen, Plots, Manifest, Prüfprotokolle und
finale Quellhashes liegen lokal unter `data/phase5_2_csi_aging/`, vollständig
von Git ignoriert. Der Commit enthält keine neuen Datensätze oder Fixtures.
Nach den finalen Dokumentationsänderungen wurde das PDF genau einmal
erzeugt und über Seitenzahl, Textextraktion, vollständige Seitenfußzeilen
und Sichtprüfung validiert; der Nachweis bleibt ebenfalls lokal.

Die Unterphase ist ein kontrollierter Empfindlichkeitsvergleich mit
perfekter RX-CSI und exaktem, lediglich verzögertem TX-CSI. Quantisierung,
Schätzfehler, physikalischer Doppler, bewegte Geometrie, Sounding-/MAC-
Overhead und automatische Updateereignisse fehlen. Die periodische
analytische Rotation rechtfertigt keine extrapolierte, für beliebige Alter
monotone Verlustkurve. Hohe gaußsche MI-Werte und alle 256 FFT-Bins
einschließlich DC/Guards sind Kanaldiagnostik, keine Wi-Fi-Payloadraten.
PER wird nicht berechnet: Die dokumentierte Kalibrierung gilt nur für
HE MCS 0/BCC/32 Byte/1x1 und wird nicht auf diese 2x2-Fälle übertragen.

7.19 R1/R3-INKREMENT: PORTABLE PROVENIENZ UND ERSTE CPU-CI

Dieses Inkrement erweitert die vorhandenen Runner und korrigiert ausschließlich
den deaktivierten Cachepfad. Die historischen Messmanifeste, Exporte und
Messzeitpunkt-Quellhashes bleiben erhalten; R1/R3 waren zu diesem Zeitpunkt offen.
Der vollständige Vertrag steht in `doc/provenance-cpu-increment.md`.

Neue Benchmarkläufe besitzen ein additives `run.json` mit
`schema=ns3sionna.run`, `schema_version=1`; Kalibrier-CLI-Läufe verwenden
`<output>.run.json`. Erfasst werden Konfiguration, Fall-/Seedidentitäten,
Revision/lokale Änderungen/Quellhashes vor dem Lauf, tatsächliches Backend,
CPU-/Paket-/Buildidentität, Befehle sowie Eingangs- und Artefakthashes.
Die bisherigen Exportschemata bleiben unverändert. Historische Läufe ohne
Sidecar haben keinen nachträglich behaupteten Ausführungsstatus; aktuelle
Replayquellen ersetzen niemals ihre historischen Messquellen.

Für Binärdaten gilt weiterhin der Rohbyte-SHA-256. UTF-8-Text erhält zusätzlich
einen versionierten Digest
\[
 d_{\mathrm{LF},1}(T)=\operatorname{SHA256}
 \left(\operatorname{UTF8encode}
 \left(\operatorname{CRLFtoLF}(\operatorname{UTF8decode}(T))\right)\right).
\]
Nur CRLF wird normalisiert; einzelne CR, Leerzeichen, BOM und Endzeilen bleiben
relevant. Der Rohdigest bleibt parallel sichtbar. `.gitattributes` erzwingt LF
für historische CSV-/JSON-Checkouts; Protobufs werden niemals textnormalisiert.
Die alten Rohhashes werden weder umgedeutet noch neu geschrieben.

Vor der Ausführung steht atomar `incomplete`; Ausnahmen liefern `failed` mit
lokalem Traceback und gegebenenfalls Subprozess-stdout/stderr. Regulärer
Abschluss schreibt atomar `success` mit Artefaktindex. Wissenschaftliche
Akzeptanz bleibt separat unter `validation`: erfolgreicher Programmablauf ist
keine Freigabe eines abgelehnten Kalibrierprofils. Unterbrechungen bleiben
`incomplete`; vorhandene Ausgaben werden geschützt, Resume wird nicht angeboten.
Der unvollständige GPU-Selektor wurde auf den bisherigen strikten CPU-Vertrag
zurückgeführt. Auch ein abgelehntes Backend wird im Fehlermanifest benannt.

Bei deaktiviertem Cache wurden zuvor neue Antworten gespeichert und ältere,
noch gültige Antworten erneut ausgewählt. Der neue native REQ/REP-Test
konstruiert unterschiedliche überlappend gültige Antworten ausschließlich im
Speicher. Vor der Korrektur lieferte er Verlust 41 statt 42, Verzögerung 1 ns
statt 3 ns und CFR `(1,-1)` statt `(4,-4)`; nach Wiederaktivierung wurde weiter
41 verwendet, und nur vier statt fünf Anfragen erreichten den Peer.
`SetCaching(false)` leert jetzt vorhandene Snapshots. Die gesamte neue Antwort
wird vor der Rückgabe validiert; der passende aktuelle Eintrag wird direkt
zurückgegeben, ohne andere Empfänger oder Look-ahead-Snapshots zu speichern.
Die inklusive Zeitgültigkeit und bestehende Richtungsprüfung bleiben erhalten.
Wiederaktivierung beginnt kalt und füllt den Cache regulär neu. Der Test prüft
auch den anschließenden Treffer ohne zusätzliche Anfrage.

Die neue CPU-CI baut den aktuellen Modulcheckout gegen die feste ns-3.40-
Revision `4fdc985b5d5b800992e0ce87df7a727e32033794`, mit frisch generierten
C++-/Python-Protobuf-Bindings. Python 3.12.12, pip 26.0.1 und 55 direkte/transitive
Paketversionen sind exakt festgelegt (`requirements_cpu_ci.lock`); `pip check`
und der Vergleich aller installierten Versionen bestanden. Der lokale frische
Linux-Build und die einmalige vollständige Auswahl über
`tools/run_cpu_regressions.py` bestanden: 50 native Testfälle in den sieben
Suites `sionna`, `sionna-mimo-csi-feedback`, `dl-mu-mimo-runtime`, `dl-mu-mimo-ru`,
`he-dl-mu-spectrum-ru`, `ul-mu-mimo` und `he-ul-mu-mimo-phy-adapter` sowie
56 erfolgreiche Python-Tests. Unter den insgesamt 57 ausgewählten Python-Tests
wurde ein optionaler lokaler Alterungsexporttest ausgelassen; drei teure frische
RT-Prüfungen wurden ausdrücklich aus dieser CI-Auswahl herausgenommen.
Vertrags-, Kalibrier-, Policy-, Provenienz- und eingefrorene Benchmarkprüfungen
verwendeten die bestehenden drei versionierten Exportstudien. Die tatsächlich
gewählte Mitsuba-Variante war `llvm_ad_mono_polarized`.

Zusätzlich bestanden ein lokaler IPC-Smoke mit einem Fall (64 Töne, TX 2/RX 4, ein Link,
drei frische Prozesse) und der Kalibrier-CLI-Replay mit Prüfung der neuen
Sidecars und Artefakthashes. Der historische selektive MAE bleibt 0,054886
bei Grenze 0,10; daraus folgt keine Freigabe für 0,05 oder weitere PHY-Profile.
Die GitHub-Workflowdatei und ihre festen Action-Revisionen wurden geprüft;
der gehostete Job selbst wurde hier nicht ausgeführt. Ubuntu-Systempakete
folgen weiterhin den Paketquellen, Wheelhashes sind noch nicht gesperrt.
Verfügbare CI-Installations-/Build-/Testlogs werden als Artefakt gesichert;
unerwartete native Testfehler enthalten auch erfasstes stdout/stderr. Nach der
abschließenden Erweiterung des Artefaktindex auf binäre Plotdateien bestanden
die neun gezielten Provenienztests erneut; PNG-Hashes bleiben rohbytegeprüft.

Lokale Logs, Builds und Smoke-Ausgaben liegen unter `/tmp/r1r3-*` beziehungsweise
`/tmp/ns3-r1r3-*`; es werden keine neuen Datensätze oder aufgezeichneten Fixtures
versioniert. Die PDF-Abschlussprüfung erfolgt nach der finalen Dokumentation
mit genau einer Neuerzeugung; Seitenzahl, Textextraktion, vollständige
Seitenfußzeilen und ausgewählte Seiten werden geprüft. Ihr Protokoll bleibt lokal.

Zum Abschluss des ersten Inkrements blieben insbesondere externer Artefaktzugang für
5.2.4/5.2.5,
Seed-/Run-/Streamunabhängigkeit, Resume, Wheel-/Systempaket-Locking und aktuelle
Containerrezepte aus R1 sowie breitere Cache-Zeit-/Richtungs-/Batchregressionen,
Belegungs-/Nullzugriffsstatistik, inaktive Linkbereinigung, Allokationsbudgets,
Langzeitspeichergrenzen und erweiterte Sweeps aus R3. Vollständige RT-, Live-,
GPU- und neue Referenzläufe sind kein Nachweis dieses Inkrements. Die Kalibrierung
bleibt HE MCS 0/BCC/32 Byte/1x1; 2x2-Ergebnisse bleiben MMSE-/SVD-/MI-Diagnostik.
Die nachfolgende Cachearbeit ist getrennt in Abschnitt 7.20 dokumentiert.

7.20 ZWEITES R3-INKREMENT: CACHEKORREKTHEIT UND BELEGUNGSSTATISTIK

Dieses Inkrement ergänzt den vorhandenen nativen Sockettest und IPC-Runner.
Die historischen Exportdateien, Fixtures, Manifeste und Messzeitpunkt-Hashes
bleiben erhalten; ihre früheren Laufzeiten werden nicht als neue Messung
interpretiert. R1/R3 waren zum Zeitpunkt dieses Inkrements nicht vollständig abgeschlossen.
Der ausführliche referenzierte Phase-5-Plan fehlt weiterhin im Checkout;
der eingebettete R1-R6-Plan bleibt die verwendete Anforderungsgrundlage.

Der Cache verwendet unverändert inklusive Gültigkeit
$$
 t_{\mathrm{start}} \leq t_{\mathrm{now}} \leq t_{\mathrm{end}}.
$$
Bei Überlappung gewinnt der erste passende Eintrag in Antwort-/Einfügereihenfolge.
Zukünftige Einträge werden gespeichert, aber vor ihrem Beginn nicht ausgewählt.
MU-Einträge mit Nutzeridentität gelten nur für ihre explizite TX/RX-Richtung;
der bestehende reziproke Legacy-Pfad bleibt erhalten. Eine kalte Anfrage muss
in ihrer neuen Antwort einen jetzt gültigen passenden Link besitzen.
Eine fehlende Zuordnung wird als `std::invalid_argument` abgelehnt; der frühere
Fallback auf einen nicht initialisierten Dummy-Eintrag entfällt.

Die vollständige Antwort wird vor Veröffentlichung der neuen Cacheeinträge
validiert. Ungültige Wrapper, Zeitintervalle und Zeit-/Delay-Konvertierungen,
leere Empfängerlisten, doppelte Empfänger, nichtendliche Verluste/CFR-Werte
und inkonsistente CFR-Längen oder Dimensionen liefern `std::invalid_argument`.
Die Multiplikation der CFR-Dimensionen wird vor Reserve/Allokation auf
`size_t`-Überlauf geprüft. Die bestehenden MU-Identitätsprüfungen bleiben aktiv.
Bei einem fehlerhaften späteren Eintrag wird auch ein gültiges Präfix nicht
gespeichert. Vorher gültige Einträge bleiben nutzbar; bereits abgelaufene
Einträge dürfen unabhängig davon bereinigt werden. Diese Atomarität gilt
für Datenvalidierung, nicht für einen Allokationsfehler während der Übernahme.
Transporttimeouts und absolute Antwort-/Allokationsbudgets bleiben offen.

`GetStats()` liefert vor dem ersten Lookup 0 statt des früheren `0/0`.
Fehlgeschlagene RPC-Versuche zählen als Miss; Statistiklesen ist kein Lookup.
Die additive API `GetStorageStatistics()` liefert aktuelle und lebenslange
Spitzenwerte für Links, Snapshots, Nutzbytes und Vektorkapazität. Für die
zurückgehaltenen Snapshots e gilt
$$
 B_{\mathrm{payload}} = \sum_e
 \left(|H_e|\,\operatorname{sizeof}(\mathrm{complex<double>})
 + |F_e|\,\operatorname{sizeof}(\mathrm{int})\right).
$$
`capacityBytes` zählt stattdessen die Kapazitäten der CFR-/Frequenzvektoren

und zusätzlich `capacity * sizeof(CacheEntry)` der Linkvektoren. Spitzen sind
das Maximum der veröffentlichten Belegung über die Objektlebensdauer.
`SetCaching(false)` setzt die aktuelle Belegung auf null und erhält diese
Spitzen; deaktivierte Anfragen speichern weiterhin keine Snapshots.
Mapknoten, Allokatoroverhead, RSS und temporäre Protobuf-/Dekodierkopien werden
nicht durch diese logischen Zähler erfasst. Teilweise belegte Linkvektoren
behalten ihre Kapazität; vollständiges Leeren entfernt den Bucket.

Nach Überschreiten der frühesten gespeicherten Endzeit entfernt ein stabiler
Sweep auf dem nächsten Cache-Lookup oder `GetStorageStatistics()` sämtliche
abgelaufenen Snapshots, auch in nie erneut angefragten Links. Er erhält
Reihenfolge und gültige zukünftige Look-ahead-Snapshots. Vor dieser Endzeit
entfällt der Sweep. Ein ganz ruhender Cache wird ohne erneuten Zugriff nicht
bereinigt; ein Sweep kostet linear viele Snapshotprüfungen und ist keine
harte Speicher-/Latenzschranke oder periodische Hintergrundbereinigung.

Validierung am 1. Oktober 2026: drei zusätzliche native Cachefälle prüfen
exakte Start-/Endränder und Zeit danach, Überlappungsreihenfolge, kalte Links,
Legacy-Reziprozität, gerichtete MU-Antworten, Zukunftserhaltung, inaktive
Bereinigung sowie aktuelle/Spitzenzähler und Leeren. Der dritte Fall lehnt
13 fehlerhafte Antworten atomar ab und prüft nach jeder Ablehnung den alten
Link; anschließend gelingt ein gültiger Retry. Alle Antworten entstehen im
Speicher, ohne aufgezeichnete neue Fixtures. Der bestehende IPC-Runner
prüft die Zähler zusätzlich gegen seine eingefrorenen CFR-/Frequenzlängen
und nach Ablauf/Refresh; sein JSON-/CSV-Exportschema bleibt unverändert.

Der inkrementelle Build im zuvor frisch erzeugten, festgelegten ns-3.40-
Checkout bestand. Die einmalige vollständige Auswahl über
`tools/run_cpu_regressions.py` bestand mit 53 nativen Fällen in sieben Suites
und 56 erfolgreichen Python-Tests unter den 57 ausgewählten Tests.
Ein optionaler lokaler Alterungsexporttest wurde ausgelassen; dieselben drei
frischen RT-Prüfungen bleiben separat. Verwendet wurden die zuvor geprüften
Python-/Paketpins und `llvm_ad_mono_polarized`. Die bestehenden eingefrorenen
IPC-Replays bestanden einschließlich der neuen Zählerprüfungen.
Logs und Builddaten liegen unter `/tmp/r3-*` und `/tmp/ns3-r1r3-*`.
Die PDF wird nach den finalen Dokumentationsänderungen genau einmal erzeugt;
Seitenzahl, Text, alle Fußzeilen und ausgewählte Seiten werden lokal geprüft.

Der gehostete CPU-Job bleibt unvalidiert: Der Remotezweig steht noch auf
`7dead3b`, vor der lokalen Workflowimplementierung. Eine Ausführung benötigt
die Veröffentlichung der lokalen Commits. Es wird kein gehosteter Erfolg
behauptet. Offen aus R3 bleiben insbesondere vor dem RT geprüfte Antwort-/
Allokationsbudgets, langfristige Grenzen für gültige/überlappende/zukünftige
Snapshots, Mobilitätshistorien-/Allokatormessungen, CFR-Kopieroptimierungen
und erweiterte Live-/Zeit-/Link-/Look-ahead-Sweeps. Die offenen R1-Pakete aus
7.19 sowie weitere PHY-Profile, GPU- und neue Referenzvalidierung bleiben offen.

7.21 DRITTES R3-INKREMENT: BUDGETS UND LOOK-AHEAD-TIEFE

Dieses begrenzte Inkrement ergänzt die vorhandenen Cache-/Serververträge,
Statistiken und Tests. R3 war zu diesem Messzeitpunkt offen. Der referenzierte ausführliche
Phase-5-Plan ist nicht vorhanden; der eingebettete R1-R6-Plan bleibt maßgeblich.
Historische Exporte, aufgezeichnete Fixtures und Provenienz werden unverändert
weiterverwendet. Neue Logs und Messungen bleiben ausschließlich lokal.

Native `SionnaPropagationCache::ResourceLimits` werden mit
`cache->SetResourceLimits(limits)` gesetzt. Python verwendet
`SionnaEnv(resource_limits=ResourceLimits(...))`; die Server-CLI bietet die
entsprechenden `--max_subcarriers`, `--max_rx_antennas`, `--max_tx_antennas`,
`--max_coefficients_per_channel`, `--max_batch_coefficients`, `--max_reply_bytes`,
`--max_snapshots` und `--max_payload_bytes`. Alle Grenzen sind positiv und

inklusiv und wirken gleichzeitig. Die gemeinsamen Standardwerte lauten:

Größe	Native Feld / Python Feld	Standardgrenze
Töne	`maxSubcarriers` / `max_subcarriers`	4096
RX-/TX-Antennen	`maxRxAntennas`, `maxTxAntennas` / `max_rx_antennas`, `max_tx_antennas`	je 64
Koeffizienten je Link	`maxCoefficientsPerChannel` / `max_coefficients_per_channel`	4194304
Koeffizienten je Antwortbatch	`maxBatchCoefficients` / `max_batch_coefficients`	16777216
Antwortbytes	`maxReplyBytes` / `max_reply_bytes`	268435456 (256 MiB)
Link-Snapshots	`maxSnapshots` / `max_snapshots`	65536
Zurückgehaltene Nutzbytes	`maxPayloadBytes` / `max_payload_bytes`	536870912 (512 MiB)

Die bisherigen nativen Deserialisierer bleiben verfügbar und delegieren an
neue Überladungen mit Standardgrenzen. Leere Legacy-CFRs bleiben unterstützt.
Ein Link-Snapshot ist ein RX-Eintrag eines Zeitpunkts, nicht ein gesamtes
`ChannelState` mit mehreren Empfängern. Die Snapshotgrenze begrenzt auch die
Anzahl aller zu dekodierenden RX-Einträge einer Antwort, einschließlich bereits
abgelaufener Einträge. Dimensionsgrenzen gelten für neue CFR-Dekodierungen.

Für jeden CFR gilt $C_e=N_{\mathrm{sc},e}N_{\mathrm{rx},e}N_{\mathrm{tx},e}$.
Native Multiplikationen, Summen und Allokationsbytes werden vor Reserve,
CFR-Konstruktion und Cacheeinfügung gegen Überlauf und Grenzen geprüft.
Python verwendet beliebig große Ganzzahlen und prüft die CFR-Bytezahl gegen
`sys.maxsize`. Der gesamte Antwortbatch wird vor seiner ersten CFR-Allokation
geprüft. Nach der bestehenden Ablaufbereinigung gilt für die geplante Aufnahme
$$
 S_{\mathrm{alt}}+S_{\mathrm{neu}}\leq S_{\max},\qquad
 B_{\mathrm{alt}}+\sum_{\mathrm{neue,\ nicht\ abgelaufene}\ e}
 (16C_e+4N_{\mathrm{sc},e})\leq B_{\max}.
$$
Dies verwendet die logischen Nutzbytes aus 7.20. Bei Budgetverletzung wird die
vollständige Antwort als `std::invalid_argument` verworfen, ohne gültiges
Präfix aufzunehmen oder gültige/zukünftige Snapshots zu entfernen. Auch ein
Absenken des Speicherbudgets unter die aktuelle gültige/zukünftige Belegung
wird abgelehnt; die bisherige Konfiguration bleibt bestehen. Ein späterer Retry
kann nach regulärem Ablauf Platz erhalten. Deaktivierter Cache behält keine
Snapshots, unterliegt aber weiterhin den Antwort-/Dekodiergrenzen.

Der native Empfang prüft die tatsächliche ZMQ-Antwortlänge vor Protobuf-Parsing
und dessen `int`-Längenkonvertierung. Das bereits empfangene ZMQ-Buffer ist damit
nicht begrenzt. Der Server prüft geplante Linkanzahl, Dimensionen und eine
konservative Wire-Obergrenze vor Mobilitätsfortschritt, Platzierung und RT.
Für R Empfänger, D Zeitpunkte, $C=N_{\mathrm{sc}}N_{\mathrm{rx}}N_{\mathrm{tx}}$
und I UTF-8-Bytes der Array-Identitätsstrings verwendet er bei CFR-Ausgabe
$$
 B_{\mathrm{wire,plan}}=64+256D+RD(512+I+16C+10N_{\mathrm{sc}}).
$$
Zehn Bytes pro Frequenz decken negative Protobuf-`int32`-Varints ab; die übrigen
Pauschalen decken Wrapper, Zeit-/Positions-/Identitätsfelder und Längen ab.
Ohne CFR-Ausgabe entfallen die beiden CFR-/Frequenzterme; die internen
CFR-Koeffizientengrenzen bleiben aktiv. Die Nutzbytegrenze prüft serverseitig
nur den geplanten Batch, nativ zusätzlich die bestehende Cachebelegung.
Die endgültige `ByteSize()` wird nochmals vor Serialisierung geprüft.
Serverablehnungen liefern eine bestehende negative `sim_ack` mit Fehlertext;
der native Client reicht ihn in seiner Ausnahme weiter. Es gibt keine neue
Protobuf-Nachricht oder automatische Aushandlung der beiden Konfigurationen.

Look-ahead erzeugt tatsächlich
$$

```
 D=\left\lfloor\frac{\mathrm{sub\_mode}}{\mathrm{receiver\_count}}\right\rfloor,
 \qquad \mathrm{receiver\_count}=|\mathrm{nodes}|-1.
\]
```
`sub_mode` zählt parallele Links, nicht die gewünschte Tiefe. Für drei Empfänger
liefern beispielsweise 5, 14 und 50 die Tiefen 1, 4 und 16. Keine Empfänger oder
\(D=0\) werden bei Initialisierung bzw. Anfrage klar abgelehnt, statt einen
leeren Antwortbatch zu erzeugen. Die Nulltiefenmeldung lautet
`Zero look-ahead depth: sub_mode must be at least receiver_count`.

Validierung am 2. Oktober 2026: Ein zusätzlicher nativer Socketfall prüft
inklusive Dimensions-/Koeffizienten-/Wire-/Speichergrenzen, Überlauf,
sechs atomare Budgetablehnungen, unveränderte Belegung/Spitzen, Ablehnung eines
zu kleinen neuen Speicherbudgets, erfolgreichen Retry nach Ablauf und einen
späteren Treffer im zuvor gespeicherten Zukunftssnapshot. Vier zusätzliche
Pythonfälle prüfen Grenzen, Allokationsüberlauf, unveränderte Nachrichten nach
abgelehnter CFR-Konvertierung und Recovery, Tiefen 1/4/16 mit Restdivision sowie
Ablehnung vor Mobilitätsaufruf/PathSolver ohne gefüllte Antwort.

Der bestehende IPC-Runner bietet zusätzlich `--storage-steps=1..128`; ohne diese
Option bleiben die bisherigen Exportschemata erhalten. Der kleine Lauf nutzte
64 Töne, TX 2/RX 4, vier Links und 16 zusätzliche Zeitfortschritte. Die 18 Fills
inklusive erster Befüllung und bestehender Ablaufprüfung erreichten zuletzt
85,000017 ms Simulationszeit. Aktuelle und maximale Belegung blieben bei genau
vier Snapshots und 33792 Nutzbytes, jeweils der konfigurierten Grenze.
Alle 43 tatsächlichen Kanal-RPCs, Zeitstempel und Koeffizienten wurden geprüft;
`run.json` meldet `success` mit getrenntem bestandenem Validierungsstatus.
Der Allokator-Deltaindikator betrug 33504 Byte; RSS mit dem gemessenen Cache
betrug 27602944 Byte, Prozess-Lebenszeitspitze 52342784 Byte. Diese Größen sind
keine Speicherbudget-Abnahmekriterien und keine CFR-/RSS-Gleichsetzung.

Der inkrementelle Build im festgelegten ns-3.40-Checkout und die bestehende
vollständige CPU-Auswahl bestanden mit 54 nativen Fällen in sieben Suites
sowie 60 bestandenen Python-Tests unter 61 ausgewählten Tests. Ein optionaler
Alterungsexporttest blieb ausgelassen; die drei frischen RT-Prüfungen bleiben
separat. Python 3.12.12, pip 26.0.1 und alle 55 Paketpins wurden geprüft;
`pip check` bestand, Backend `llvm_ad_mono_polarized`. Historische eingefrorene
Export-/IPC-Regressionsprüfungen bestanden unverändert. Logs liegen unter
`/tmp/r3-budget-*`, Build/Paketumgebung unter `/tmp/ns3-r1r3-*`.

Reproduktion vom vorbereiteten ns-3-Verzeichnis aus, mit leerem Ausgabeordner:

```
    python contrib/sionna/tools/run_cpu_regressions.py
    python contrib/sionna/tools/benchmark_cache_ipc.py --storage-steps=16 \
    --output=/tmp/r3-budget-storage
```

Nach der letzten Dokumentationsänderung wird die PDF genau einmal regeneriert;
Seitenzahl, Textextraktion, sämtliche Fußzeilen und ausgewählte Seiten werden
lokal validiert. Neue Datensätze, Fixtures, Logs und Plots werden nicht committed.

Offen bleiben Mobilitätshistorien, RT-/CIR-Zwischenspeicher, temporäre
Protobuf-/CFR-Kopien, Allokatorpools und RSS; die Budgetgrenzen sind keine harte
Prozessspeichergrenze. Budgetablehnung ist atomar, ein `bad_alloc` während der
Cacheübernahme besitzt weiterhin keine vollständige Rollbackgarantie.
Eine serverseitig passende Antwort kann wegen bereits zurückgehaltener
Snapshots oder unterschiedlich konfigurierter Clientlimits dennoch abgelehnt
werden; der zustandsbehaftete Server wird dadurch nicht automatisch
zurückgespult. Kein erforderlicher gültiger oder zukünftiger Snapshot wird
zur Freigabe von Platz verdrängt. Transporttimeouts, Budgetaushandlung,
Kopieroptimierungen, lange wechselnde Arbeitsmengen, breite Live-/GPU- und
Look-ahead-Sweeps sowie der gehostete CPU-Nachweis bleiben außerhalb dieses
Inkrements. R3 war zu diesem Messzeitpunkt nicht abgeschlossen; seine Abschlussabnahme steht in

7.24.

7.22 VIERTES R3-INKREMENT: CFR-KOPIERREDUKTION UND GETTEREIGENTUM

Dieses nächste begrenzte Phase-5-Inkrement setzt den Kopier-/Wiederverwendungspunkt von R3 um. Vier Produktionsdateien ändern ausschließlich Eigentum und Datenhaltung: `mimo-channel-matrix.{h,cc}` und `sionna-propagation-cache.{h,cc}`. Der vorhandene IPC-Benchmark und seine Tests werden erweitert; ein zweites Framework oder neue versionierte Messfixtures entstehen nicht. Der Vertrag steht in Abschnitt 5.25. R3 und Phase 5 insgesamt waren zu diesem Messzeitpunkt offen; R3 schließt in 7.24.

Die Baseline ist die vor den Produktionsänderungen gebaute native Bibliothek zum Modulstand `11fd05c`. Sie wurde lokal gesichert und ihr SHA-256 gegen die bereits aufgezeichnete Buildidentität geprüft. Für den vollständigen Gettervergleich verwendeten Baseline und Kandidat dasselbe neue Benchmarkprogramm; nur die Sionna-Bibliothek wurde über `LD_LIBRARY_PATH` gewechselt. `ldd` bestätigte den Baselinepfad. Die alte Bibliothek wird damit nicht als Ergebnis neuer Quellen bezeichnet. Baseline-Buildidentität, Kriterien und Rohmessungen stehen in `/tmp/r3-copy-baseline/`; Kandidatprovenienz und vollständige Exporte in `/tmp/r3-copy-profile-final/`. Ein früherer Kandidatenlauf bleibt separat unter `/tmp/r3-copy-profile/`, Vergleiche unter `/tmp/r3-copy-comparison*.json` erhalten. Keine dieser Messungen oder Bibliotheken wird in Git aufgenommen.

Gemessen wurde am 6. Oktober 2026 auf AMD Ryzen 5 PRO 2500U, x86_64, Linux 7.0.0-38 und glibc 2.39, mit dem lokalen ns-3.40-`default`-Build und aktiven Assertions. Die Vergleichsziele wurden vor Kandidatenmessungen festgehalten. Jeder Tabellenwert ist der Median dreier Prozessmediane; Zeitstreuung und p95 bleiben separat verfügbar. Für 1024 Töne, TX/RX 8/8 und 16 Links ergeben sich:

Operation	Baseline (µs)	Finaler Kandidat (µs)	Medianreduktion
Matrixgetter	75,289	71,667	4,81 Prozent
Konstanter Lossgetter	73,365	0,831	98,87 Prozent
Delaygetter	73,135	0,922	98,74 Prozent
Frequenzgetter	72,889	1,022	98,60 Prozent
CSI-Vektorgetter	126,807	73,716	41,87 Prozent
Einzel-CFR-Dekodierung	258,033	81,676	68,35 Prozent
Kalter vollständiger Cache-Miss	23321,386	17319,350	25,74 Prozent

Das vorab gesetzte 20-Prozent-Ziel für den großen Matrixgetter wurde ausdrücklich **nicht** erreicht. Dessen notwendige öffentliche CFR-Wertkopie bleibt bestehen; keine allgemein schnellere Matrix-API wird behauptet. Die skalaren Getter erfüllen das 20-Prozent-Ziel deutlich; die Frequenzausgabe kopiert nur noch Frequenzen. Der erste Kandidat machte gleichzeitig eine langsamere Einzeldekodierung sichtbar. Die abschließende Anpassung verwendet eine einmalig vorgeprüfte Vektorlänge und indizierte Schreibzugriffe; der finale Decodervergleich ist oben ausgewiesen. Alle früheren Beobachtungen bleiben lokal erhalten.

Beim kleinen Profilfall mit 64 Tönen, TX/RX 2/2 und einem Link sinkt der Matrix-Prozessmedian von 2,975 auf 1,563 µs, die Einzeldekodierung von 1,714 auf 0,536 µs. Ein separat gemessener unveränderter Default-Matrixpfad prüft das vorab gesetzte Höchstens-zehn-Prozent-Regressionsziel: 2,265 µs Baseline und 1,493 µs finaler Prozessmedian entsprechen Verhältnis 0,659 und bestehen die Grenze 1,10. Alle drei finalen Default-Prozesse bestanden dieselben Daten-/Requestprüfungen; der Vergleich steht in `/tmp/r3-copy-default-final.json`. Profilmodus und Defaultmodus werden nicht zu
einem gepoolten Zeitdatensatz zusammengefasst.

Die großen Matrix-Prozessmediane streuen von 70,510 bis 75,024 µs; die prozessweisen p95 liegen zwischen 90,807 und 153,668 µs. Die Lossmediane

liegen bei 0,812 bis 0,842 µs und p95 bei 0,980 bis 1,102 µs. Kleine CPU-Zeiten und die Verteilungen belegen keine harte Latenzgrenze. Die aufgezeichneten Live-Heap-Deltas des großen Falls liegen bei rund 16,068 MiB, Linux-RSS mit Cache bei 106,352 bis 106,492 MiB. Die logischen Nutzbytes betragen unverändert 16842752 Byte (klein: 4352 Byte). Allokatorpools, Map-/Protobuf-/Socketbedarf und RSS bleiben getrennte Beobachtungen; keine Gleichsetzung mit dem Cachebudget.

Der finale Profilrunner validierte sechs unabhängige native/REP-Prozesspaare, 4446 Zeitstichproben, 162 Kanal-RPCs und sechs bestätigte Close-Anfragen. Alle Koeffizienten, Dimensionen, Frequenzen, Delay-/Loss-Felder, Zeiträder, Trefferverhältnisse und Requesttranskripte bestanden. Die deterministischen Felder waren innerhalb jedes Falls identisch. `run.json` meldet `success`, Backend `frozen-protobuf-local-zmq` und separat `validation.status=passed`. Die zehn bestehenden/erweiterten IPC-Tests bestanden zusätzlich gegen diesen neuen Export einschließlich aller Statistiken, Hashes und frischer Replays.

Der neue native Eigentumstest prüft die unveränderte Allokationsadresse bei Rvalue-Matrixaufbau und die unabhängige Lvalue-Kopie. Er variiert die Wire-Nutzerreihenfolge mit unterschiedlichen gerichteten MU-CFRs, mutiert öffentliche Ergebnisse und prüft die alten Werte nach Ablauf, Bereinigung, Cacheabschaltung und Wiederaktivierung. Ein korrektes MU-Präfix mit fehlerhaftem späteren Snapshot wird vollständig verworfen; Belegung und Spitzen bleiben erhalten, vorherige Kanäle bleiben nutzbar. Wiederholte Getterhits verursachen keine zusätzlichen RPCs.

Der abschließende Zielbuild für `sionna-test` und die Benchmark-/SU-/MU-Beispiele bestand. Die bestehende vollständige Auswahl `run_cpu_regressions.py` wurde nach Stabilisierung einmal ausgeführt: 55 native Fälle in sieben Suites bestanden, sowie 65 bestandene Python-Tests unter 66 ausgewählten Tests. Nur der optionale lokale Alterungsexporttest wurde ausgelassen; die drei frischen RT-Prüfungen bleiben separat. Die früheren eingefrorenen Messungen und ihre Quellenhashes wurden nicht umgeschrieben. Python 3.12.12, sämtliche 55 Paketpins und `pip check` bestanden; der Serverimport bestätigte `llvm_ad_mono_polarized`.

Zusätzlich bestanden sechs vorhandene Beispiel-Suites mit 13 Fällen für SU-MIMO, Helper-Konfiguration/-Laufzeit, Precoding/Feedback/Alterung und MU-MIMO, einschließlich der bestehenden Negativtests. Der MU-End-to-End-Vergleich prüfte vier Downlinkmethoden, zwei Kanäle, aktuelles/verzögertes CSI und drei Uplinkmethoden; seine vorhandene Referenzausgabe blieb exakt erhalten.

Build-, CPU-, Export- und Beispielprotokolle liegen unter `/tmp/r3-copy-*.log`. Nach der letzten Dokumentationsänderung wird die PDF genau einmal erzeugt und auf Seitenzahl, vollständige Textextraktion, alle Fußzeilen und ausgewählte Seiten geprüft. Neue Rohdaten, Zeitmessungen und Fixtures bleiben ausschließlich lokal.

Zum damaligen Messzeitpunkt waren breitere Sweeps, Mobilitätshistorien und Allokatorpools offen. Die verbleibende R3-Abnahme wird in 7.24 geschlossen; RT-/CIR-/Protobuf-/Transportbedarf, Transporttimeouts, Budgetaushandlung sowie Live-/GPU-/gehostete CPU-Evidenz bleiben außerhalb dieser Speicherabnahme. Der ungecachte Getter behält seine unabhängige öffentliche Wertkopie. Dieses Inkrement reduziert keine MU-PHY-Gruppenrechnung und erweitert weder die HE-MCS-0/BCC/32-Byte/1x1-Kalibrierung noch den physikalischen CSI-Vertrag.

7.23 NATIVE R1-ABNAHME

Am 7. Oktober 2026 bestanden im bestehenden nativen Conda-CPU-Environment:

- gezielte Neubauten von Tests und beiden betroffenen Benchmarks nach Protobuf-/C++-Konfigurationsänderungen;
- strikte Umgebungsprüfung ohne Pinabweichung und tatsächliches `llvm_ad_mono_polarized`;
- sieben frische 10000-Sample-Traces: wiederholte Tupel (Run 1 und Run 2)

liefern byteidentische Kanäle; alternative Runs/Streambereiche leiten andere
Seeds ab; Run 1/RT-ID 0 erhält 42/43/44;
- verschobener repräsentativer Bundle: 20 Dateien verifiziert, frisches RT
und eingefrorener Replay mit unabhängiger NumPy-Referenz bestanden; der
ursprüngliche Szenenpfad war für den RT-Replay nicht verfügbar;
- Bundle der lokalen 5.2.4/5.2.5-Studien: 129 Dateien unverändert verifiziert,
je ein nativer Replay bestanden; historische Ausführung bleibt unbekannt;
- etablierte CPU-Regressionen sequenziell: sieben native Suites, 58 Fälle;
98 Python-Tests, davon 96 bestanden und zwei optionale Exporttests ausgelassen.

Die vollständigen lokalen Pakete und unveränderlichen Indexidentitäten stehen
im versionierten Artefaktkatalog. Neue Messdaten bleiben außerhalb von Git;
historische Messhashes, Datensätze und R2 wurden nicht verändert.

7.24 R3-ABSCHLUSS: BEGRENZTE MOBILITÄTS- UND POOL-SPEICHERUNG

Am 7. Oktober 2026 wurde die verbleibende R3-Abnahme als eine Unterphase im
bestehenden nativen Conda-CPU-Environment abgeschlossen:

- 15 vorab festgelegte Fälle: H = 0/1/8/64/1024, WALL/TIME/DISTANCE,
ns-3 Run 1, Seeds 42/43/44 und explizite Streams 0/1. Alle 300000
Bewegungsupdates stimmen exakt in Position, Geschwindigkeit, Kontrollzustand
und lokalem RNG mit einer historienlosen Referenz überein. 1200 Checkpoints
prüfen Slotidentitäten und Zähler; H = 1024 erreicht 49152 Byte und 1024
Paarallokationen, danach 18977 Wiederverwendungen. H = 0 bleibt leer.
- 512 ungecachte native RPCs: eine Stagingallokation, 511 Poolhits,
336 Byte freie Kapazität innerhalb 1048576 Byte; CSI-Belegung bleibt null.
Wachstum, kleinere Grenzen, Übergrößen, deaktivierter Pool, fehlerhafte
Batchsuffixe und unabhängige Getterwerte bestehen ebenfalls.
- 16-Epochen-Look-ahead liefert mit H = 0/1/1024 identische Antworten;
Fehler bei Platzierung/CFR stellen Knotenidentität, Historie und RNG wieder her.
Sechs zusätzliche echte LLVM-Traces (2x2, 256 Bins, 10000 Samples,
zwei Epochen bei Anfragezeiten 1/12 ms) stimmen rohbytegenau mit der
Vor-R3-Implementierung und einer Wiederholung überein.
- beide nativen Benchmarks und Sionna-Tests gezielt neu gebaut; bestehender
Dependency-Lock und tatsächlicher LLVM-Backendcheck bestanden.
- etablierte CPU-Regressionen sequenziell: sieben native Suites mit 60 Fällen;
122 Python-Tests, davon 120 bestanden und zwei optionale Exporttests ausgelassen.

Neue Pläne, Rohantworten, CSVs, Logs und Eingangsbytes sind lokal im
[R3-Artefaktkatalog](experiments/r3/catalog.json) mit externem Indexhash
zugänglich. Das verschobene 39-Dateien-Paket verifiziert vollständig; ein neuer nativer
CPU-Lauf reproduziert alle deterministischen Speicher-/Trajektorienfelder exakt. Historische
Messhashes und R1/R2 bleiben unverändert. R3 besitzt keine Live-RT/PHY-Latenz-,
physikalische Doppler-, CFR-/Protobuf-Allokator- oder RSS-Freigabe; R4–R6 bleiben offen.

8. BEKANNTE GRENZEN UND NÄCHSTE ENTSCHEIDUNGSPUNKTE

- SionnaMimoHelper überträgt eine gemeinsame TX- und RX-Geometrie für die
jeweilige integrierte Verbindung. Knoten können unterschiedliche
Orientierungen besitzen. Der Snapshotvertrag aus Unterphase 4.1
repräsentiert heterogene RX-Geometrien pro Nutzer; die aktuelle
Sionna-Raytracing-Konfiguration verwendet weiterhin ein gemeinsames RX-Array.
- Nicht explizit orientierte Knoten behalten die alte Serverorientierung
[0, -180, 0]. Explizite Winkel werden in beiden Servermodi an Sionna
übertragen.
- Der integrierte MIMO-Pfad unterstützt HE SU ohne Precoding, festes
benutzergegebenes \(W[k]\) und die explizite Helperwahl von MRT oder SVD.
Nicht-HE-Kontrollrahmen verwenden weiterhin das erste Antennenpaar.
SelectPrecoding benötigt eine bereits vorliegende CSI-Matrix und Zeitstempel;
automatische Kanalschätzung, Sounding, periodische Simulatorereignisse,

MAC-Feedbackframes und per-Paket-Aktualisierung sind nicht implementiert.
Unterphase 4.2 stellt den festen gemeinsamen MU-Precoder und die
Inter-User-Interferenz als direkte native API bereit. Unterphase 4.3 ergänzt
daran die explizit aufgerufenen ZF-/RZF-Fabriken für vollrangige Gruppen mit
einer Empfangsantenne und einem Stream je Nutzer. Unterphase 4.4 integriert
einen explizit installierten gemeinsamen Zustand für HE-DL-MU direkt in den
Spectrumadapter und liest die HE-MU-PHY-Metadaten aus dem TxVector.
Unterphase 4.5 ergänzt die wertbasierte Laufzeit- und Scheduler-Schnittstelle;
Unterphase 4.6 installiert sie mit `InstallDownlink` am AP und setzt am
tatsächlichen Sendezeitpunkt den getrennten Übertragungskanal. Automatisches
Sounding und MAC-Feedbackrahmen bleiben offen.
- Der direkte Uplinkkern der Unterphase 4.7 verlangt pro Nutzer eine
Sendeantenne und Stream null, einen gemeinsamen AP-Empfänger sowie ideale
Zeit- und Frequenzsynchronisation. ZF verlangt zusätzlich
$U\leq N_{\mathrm{rx,AP}}$ und vollen Rang; lineares MMSE bleibt bei
Rangverlust definiert. Unterphase 4.8 bindet den Kern an HE-Basic-Trigger
und echte HE-TB-PPDU-Metadaten, gruppiert exakt gleichzeitige Nutzer je RU
und meldet fehlende oder nicht gleichzeitige Teilnehmer explizit. Die
Gleichzeitigkeit wird über das vom Aufrufer beobachtete Empfangsintervall
eingebracht; automatische Trigger-Schedulerinstallation, Zeit-/
Frequenzoffsets und mehrere lokale Nutzerstreams bleiben offen.
- MRT bleibt der eindeutige perfekte 2x1-MISO-Ein-Stream-Fall. Quantisierung
und positives CSI-Alter gelten nur für gleichleistungs-SVD. Wasserfüllung
bleibt auf unquantisiertes aktuelles CSI beschränkt, weil die semantische
Winkelrückmeldung einen semi-unitären Gleichleistungs-Precoder verlangt.
- Die Auswahl speichert einen einzelnen unveränderlichen W[k]-Schnappschuss
pro Helper. Der Anwender muss die aktuelle Sendekanal-Matrix weiterhin
getrennt für SINR und PER bereitstellen und bei neuer CSI SelectPrecoding
erneut aufrufen. Das ist kein per-Link- oder per-Paket-Precodercache.
- Die Inter-User-Interferenz der gleichzeitig durch denselben festen
MU-Precoder gesendeten Streams geht in Unterphase 4.2 frequenz- und
antennenabhängig in die MMSE-Kovarianz ein. Im HE-MU-Pfad der Unterphase 4.4
gilt das ausschließlich innerhalb derselben RU; verschiedene RUs sind
orthogonal. Andere zeitvariable externe Interferenz wird weiterhin erst
nach der MMSE-/EESM-Abstraktion durch InterferenceHelper skalar
berücksichtigt; ihre räumliche Kovarianz ist nicht modelliert.
- SionnaMimoHelper übernimmt die Rauschzahl einmalig und setzt sie identisch am
Adapter sowie an beiden PHY-Helpern. Code ohne High-Level-API muss beide
Werte weiterhin manuell konsistent halten, da ns-3.40 das PHY-Attribut nicht
lesbar macht. Der Phase-4.4-Adapter verwendet deshalb seinen explizit
konfigurierten Wert für alle adressierten Empfänger.
- Es existiert absichtlich keine mitgelieferte HE-EESM-Beta-Tabelle.
Frequenzselektive absolute PER-Prognosen benötigen vor der wissenschaftlichen
Nutzung eine dokumentierte Link-Level-Kalibrierung. Ohne diese Konfiguration
bricht das Modell früh ab.
- TableBasedErrorRateModel ist eine AWGN-Referenz. Seine SNR-/PER-Kurven und
Paketlängenextrapolation stammen aus ns-3; ihre Grenzen gelten unverändert.
- Perfekte RX-CSI bleibt bewusst idealisiert. Für die Precoderwahl trennt der
neue Alterungspfad die geschätzte CFR zum CSI-Zeitpunkt explizit von der CFR
zum Sendezeitpunkt und weist Alter, Aktualisierungsintervall und
Rückmeldedauer aus; eine automatische Kanalschätzung oder Ablaufsteuerung im
Helper existiert noch nicht.
Der 5.2.5-Benchmark entwickelt eingefrorene reale CFRs durch eine
analytische unitäre TX-Basisrotation bei fester Geometrie. Seine Alters-
und MMSE-/MI-Verluste sind keine physikalisch kalibrierte Doppler- oder
Mobilitätsvorhersage; gemeinsame Kapazität bleibt hier invariant.
- Ohne W[k] gilt weiterhin genau ein Stream je TX-Antenne. Mit festem W[k]
sind kontrollierte Streamauswahl und
$1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$ für SU implementiert.
Im festen MU-Kern gilt zusätzlich
$N_{s,\mathrm{ges}}\leq N_\mathrm{tx}$ und $N_{s,u}\leq N_{\mathrm{rx},u}$.

Die ZF-/RZF-Auswahl der Unterphase 4.3 ist enger: $N_{s,u} = 1$, $N_{\mathrm{rx},u} = 1$, $U \le N_{\mathrm{tx}}$ und voller numerischer Rang auf jedem Subcarrier.
- Die MI-Kompression bleibt eine Diagnosemetrik; nur die explizit konfigurierte EESM-Ausgabe wird an ErrorRateModel übergeben.
- Explizit identifizierte Phase-4.1-Rohkanaleinträge werden nur in ihrer gespeicherten TX-/RX-Richtung getroffen. Nur identitätsfreie Legacy-Einträge behalten die bisherige reziproke Cacheverwendung. Die Precoder-Zustandsidentität enthält ebenfalls gerichtete TX-/RX-Knoten, beide Arraykonfigurationen, Modus, Feedbackkonfiguration und CSI-Zeitpunkt. Der Helper gibt diese Identität maschinenlesbar weiter, bindet sie aber bewusst nicht an einen automatischen Laufzeitcache.
- Der deterministische Beispielkanal ist eine gerichtete, normalisierte Vorwärts-CFR. Das Ad-hoc-UDP-Beispiel setzt deshalb den ARP-Eintrag am Sender statisch; so erzeugt die Infrastruktur keine rückwärts gerichtete HE-Datennachricht mit inkompatiblen 2x1-Dimensionen. Normale Sionna-Cachepfade sind davon nicht betroffen.

Phase 3 ist mit Unterphase 3.5 abgeschlossen; Unterphase 4.1 ergänzt den gerichteten, atomar validierten Mehrnutzer-Kanalsnapshot. Unterphase 4.2 verwendet ihn nun für einen festen, explizit zugeordneten gemeinsamen Downlink-Precoder, erhält die gesamte AP-Leistung und liefert gewünschte Leistung, Inter-User-Interferenz, Post-MMSE-SINR, EESM und PER je Nutzer. Der Phase-4.3-Kern leitet daraus für den dokumentierten Einantennen-/Ein-Stream-Fall deterministisches ZF oder explizit regularisiertes RZF ab, erhält die Gesamtleistung und lehnt nicht planbare Gruppen ab. Unterphase 4.4 bindet diesen gemeinsamen Zustand für echte HE-DL-MU-PPDUs an STA-ID, RU, MCS und NSS, begrenzt räumliche Interferenz auf dieselbe RU und übergibt erst nach der Matrixrechnung eine skalare PSD je adressiertem Empfänger. Der Helper führt parallel die unveränderten SU-Verträge durch den bisherigen festen-W[k]-Pfad. Einzelnutzer- und 1x1-Grenzfall sowie alle Phase-1-/2-/3-Pfade bleiben kompatibel. Die Laufzeit-CSI-/Scheduler-Schnittstelle der Unterphase 4.5 verwaltet jetzt den neuesten exakten AP-/Gruppen-/RU-/NSS-Zustand, übernimmt Phase-3-Zeiten und installiert ausschließlich denselben gerichteten Snapshot im PHY. Unterphase 4.7 ergänzt getrennte Nutzerleistungen, gemeinsame ZF-/MMSE-Detektion und nutzerweise EESM/PER im direkten Uplink-Numerikkern. Unterphase 4.8 schließt Phase 4 mit dem HE-TB-/Triggeradapter, expliziter Gleichzeitigkeitsauswahl und dem erweiterten CPU-Phasenbeispiel ab. Automatische Sounding-/Feedbackabläufe, Synchronisationsfehler und eine vollständige MAC-Schedulerinstallation bleiben Forschungsgrenzen für Phase 5.

9. AUSFÜHRUNG

Vom ns-3.40-Verzeichnis:

```
./ns3 configure --enable-examples --enable-tests
./ns3 build
./test.py -s sionna
./test.py -s dl-mu-mimo-ru
./test.py -s he-dl-mu-spectrum-ru
./test.py -s he-ul-mu-mimo-phy-adapter
./test.py -s ul-mu-mimo
./test.py -s dl-mu-mimo-runtime
./test.py -s sionna-mimo-csi-feedback
./test.py -s sionna-example-sumimo-minimal
./test.py -s sionna-example-mimo-helper-configuration-negative
./test.py -s sionna-example-mimo-helper-runtime-negative
./test.py -s sionna-example-sumimo-precoding
./test.py -s sionna-example-sumimo-precoding-negative
./test.py -s sionna-example-mumimo
./ns3 run "ns3_wifi_sumimo_minimal --eesmBetaLinear=2"
```

```
    ./ns3 run "ns3_wifi_sumimo_precoding --eesmBetaLinear=2"
    ./ns3 run "ns3_wifi_sumimo_precoding --eesmBetaLinear=2 \
    --quantization=coarse --csiUpdateIntervalMs=5 --feedbackDelayMs=2"

    ./ns3 run "ns3_wifi_mumimo --eesmBetaLinear=2 --rzfAlpha=0.25 --verify=1"
```

Reproduzierbarer Phase-5.2.1-Benchmark, vom ns-3-Verzeichnis aus, mit leerem Ausgabeverzeichnis und ohne laufenden ZMQ-Server:

```
    ./ns3 build ns3_wifi_sumimo_benchmark sionna-test
    conda run --no-capture-output -n sionna-venv python \
    contrib/sionna/tools/benchmark_sumimo.py \
    --output=/tmp/sumimo-spacing-orientation
    cd contrib/sionna/tools
    conda run --no-capture-output -n sionna-venv python \
    -m unittest -v test_benchmark_sumimo
```

Die Tests nutzen standardmäßig die versionierte CPU-Referenz. Mit `SUMIMO_BENCHMARK_RESULTS=/tmp/sumimo-spacing-orientation` wird die neue vollständige Ausgabe überprüft. Ein einzelner eingefrorener realer Kanal kann vom ns-3-Verzeichnis aus ohne Raytracing erneut ausgewertet werden:

```
    ./ns3 run 'ns3_wifi_sumimo_benchmark --action=analyze
    --input=contrib/sionna/data/phase5_2_spacing_orientation/case_00/channel_00.pb
    --output=/tmp/sumimo-replay.csv'
```

Reproduzierbare Phase-5.2.2-Samplekonvergenz, wieder vom ns-3-Verzeichnis aus mit leerem Ausgabeverzeichnis:

```
    ./ns3 build ns3_wifi_sumimo_benchmark sionna-test
    conda run --no-capture-output -n sionna-venv python \
    contrib/sionna/tools/benchmark_rt_convergence.py \
    --output=/tmp/sumimo-sample-convergence
    cd contrib/sionna/tools
    conda run --no-capture-output -n sionna-venv python \
    -m unittest -v test_benchmark_rt_convergence
```

Die neue Testsuite verwendet standardmäßig die versionierte 5.2.2-CPU-Referenz; `SUMIMO_CONVERGENCE_RESULTS=/tmp/sumimo-sample-convergence` prüft eine neue vollständige Ausgabe. Der Runner exportiert zwei frische Traces pro Fall/Seed/Budget, bewertet gepaarte Seeds und erzeugt vier unversionierte Vergleichsplots. Der bestehende 5.2.1-Runner bleibt mit seinen Defaultkontrollen unverändert nutzbar.

Reproduzierbare Phase-5.2.3-Cache-/IPC-Skalierung, vom ns-3-Verzeichnis aus:

```
    ./ns3 configure --enable-examples --enable-tests
    ./ns3 build ns3_sionna_cache_ipc_benchmark sionna-test
    conda run --no-capture-output -n sionna-venv python \
    contrib/sionna/tools/benchmark_cache_ipc.py \
    --output=/tmp/sionna-cache-ipc
    cd contrib/sionna/tools
    conda run --no-capture-output -n sionna-venv python \
    -m unittest -v test_benchmark_cache_ipc
```

Ein leerer Ausgabeordner ist erforderlich. Der Runner startet und beendet seinen lokalen Frozen-REP-Prozess selbst; ein Sionna-RT-Server ist nicht nötig. Die Tests verwenden die versionierte Referenz unter `data/phase5_2_cache_ipc/`; `CACHE_IPC_BENCHMARK_RESULTS=/tmp/sionna-cache-ipc` prüft einen neuen Export. Alle Fixture-, Cache- und IPC-Validierungen bleiben von PER-Kalibrierung unabhängig. Native/RPC-Zeitstichproben und speicherbezogene Diagnosen stehen

in CSV; die drei generierten PNG-Plots bleiben unversioniert.

Viertes Phase-5-R3-Inkrement, mit leerem Ausgabeordner:

```
./ns3 build -j 2 sionna-test ns3_sionna_cache_ipc_benchmark
python contrib/sionna/tools/run_cpu_regressions.py
python contrib/sionna/tools/benchmark_cache_ipc.py --copy-profile \
--output=/tmp/r3-copy-profile
cd contrib/sionna/tools
CACHE_IPC_BENCHMARK_RESULTS=/tmp/r3-copy-profile \
python -m unittest -v test_benchmark_cache_ipc
```

Es gilt das dokumentierte CPU-Environment. Ohne `--copy-profile` bleibt die
bisherige Messmatrix aktiv; ohne Exportoverride prüfen die Tests weiterhin
die historische eingefrorene Referenz.

Phase 5.2.4 verwendet denselben Runner mit optionalem kleinem Sweep:

```
./ns3 build ns3_wifi_sumimo_benchmark sionna-test
conda run --no-capture-output -n sionna-venv python \
contrib/sionna/tools/benchmark_rt_convergence.py \
--independent-spacing --output=/tmp/sumimo-independent-spacing
```

Ein leerer Ausgabeordner ist erforderlich. Die vorhandenen Tests werden mit
`python -m unittest -v test_benchmark_sumimo test_benchmark_rt_convergence`
im Tools-Verzeichnis ausgeführt. Der Runner prüft jede neue CFR gegen die
unabhängigen Referenzformeln und jeden höheren Budgetlauf gegen die vorab
festgelegten Toleranzen. Neue Messergebnisse und Fixtures bleiben lokal.

Phase 5.2.5 verwendet denselben Runner für aktuelles/verzögertes SVD-CSI:

```
./ns3 build ns3_wifi_sumimo_benchmark sionna-test
conda run --no-capture-output -n sionna-venv python \
contrib/sionna/tools/benchmark_rt_convergence.py \
--csi-aging --output=/tmp/sumimo-csi-aging
cd contrib/sionna/tools
SUMIMO_AGING_RESULTS=/tmp/sumimo-csi-aging \
conda run --no-capture-output -n sionna-venv python \
-m unittest -v test_benchmark_sumimo test_benchmark_rt_convergence
```

Ein leerer Ausgabeordner ist erforderlich. Ohne `SUMIMO_AGING_RESULTS` wird
nur der optionale lokale Exporttest übersprungen; der neue analytische
Alterungstest läuft weiterhin. Natives Replay einer initialen CFR:

```
./ns3 run 'ns3_wifi_sumimo_benchmark --action=aging
--input=/tmp/sumimo-csi-aging/case_03/seed_42/samples_100000/channel_00.pb
--output=/tmp/aging-replay.csv --csiAgeMs=5 --transmissionMs=5 --rotationRateDegPerMs=9'
```

Zeitstempel beziehen sich auf die initiale CFR bei t=0. Rate null hält den
Kanal statisch; die ältere Open-Loop-Aktion `analyze` bleibt unverändert.

Python-Vertrag und Server vom Verzeichnis contrib/sionna/model/ns3sionna:

```
python -m pip install --upgrade -r requirements_cpuonly.txt
python tests/test_imports.py
conda run -n sionna-venv python -m unittest -v tests.test_mimo_data_contract
conda run -n sionna-venv python -m unittest -v tests.unit_tests
```

R3-Abschluss, im vorhandenen Conda-CPU-Environment mit leerem Ausgabeordner:

```
./ns3 build -j 2 sionna-test ns3_wifi_sumimo_benchmark ns3_sionna_cache_ipc_benchmark
```

```
    python contrib/sionna/tools/benchmark_mobility_storage.py --native-check \
    --output=/tmp/r3-storage
    python contrib/sionna/tools/experiment_bundle.py export \
    --study=/tmp/r3-storage --output=/tmp/r3-bundle
    python contrib/sionna/tools/run_cpu_regressions.py

Den ausgegebenen Indexhash separat aufbewahren; das vollständige Bundle verschieben
und mit `verify --bundle <neuer-pfad> --index-sha256 <externer-hash>` prüfen.
`storage.csv` enthält alle 256 Schritte einen Checkpoint sowie Anfang/Ende.
Zeitmessungen umfassen gepaarte Bewegung und Validierung und sind rein deskriptiv.

PDF neu erzeugen:

    conda run -n sionna-venv python
    contrib/sionna/tools/render_model_pdf.py
    contrib/sionna/ns3sionna_mimo_model.md
    contrib/sionna/ns3sionna_mimo_model.pdf

Das Rendering-Skript benötigt nur Matplotlib und läuft im vorhandenen
CPU-Environment.
```